

**CHRONOSCHEDIO: IMPLEMENTATION OF
A WEB-BASED CLASS SCHEDULING AND FACULTY
LOADING MANAGEMENT FOR STI COLLEGE
ALABANG**

**A Capstone Project Proposal
Presented to the Faculty of the
Information and Communications Technology Program
STI College Alabang**

**In Partial Fulfilment of the
Requirements for the Degree
Bachelor of Science in Information Technology**

**Bianes, Stalen Hitler N.
Diamla, James Roland M.
Ibarreta, Richard Rodney T.
Labaclado, John Simon Lee B.**

November, 23 2024

ENDORSEMENT FORM FOR PROPOSAL DEFENSE

TITLE OF RESEARCH: **CHRONOSCHEDIO: IMPLEMENTATION OF A WEB-BASED CLASS SCHEDULING AND FACULTY LOADING MANAGEMENT FOR STI COLLEGE ALABANG**

NAME OF PROPONENTS: Bianes, Stalen Hitler N.
Diamla, James Roland M.
Ibarreta Richard Rodney T.
Labaclado, John Simon Lee B.

In Partial Fulfilment of the Requirements
for the degree Bachelor of Science in Information Technology
has been examined and is recommended for Oral Defense.

ENDORSED BY:

Ms. Darryl Pauline S. Nietes
Capstone Project Adviser

APPROVED FOR PROPOSAL DEFENSE:

Mr. Jeremiah D. Tablatin
Capstone Project Coordinator

NOTED BY:

Mr. King Jasser N. Andaya
Program Head

November 23, 2024

APPROVAL SHEET

This capstone project proposal titled: **CHRONOSCEHDIO: IMPLEMENTATION OF A WEB-BASED CLASS SCHEDULING AND FACULTY LOADING MANAGEMENT FOR STI COLLEGE ALABANG** prepared and submitted by **Bianes Stalen Hitler N., Diamla James Roland M., Ibarreta Richard Rodney T., and Labaclado John Simon Lee B.**, in partial fulfillment of the requirements for the degree of Bachelor of Science in Information Technology, has been examined and is recommended for acceptance and approval.

Ms. Darryl Pauline S. Nietes

Capstone Project Adviser

Accepted and approved by the Capstone Project Review Panel

in partial fulfillment of the requirements for the degree of

Bachelor of Science in Information Technology

Mr. Jerryfel M. Laraga

Panel Member

Kathlin Claire G. Torcelino

Panel Member

Mr. Romeo Lorenzo V. La Rosa

Lead Panelist

Noted:

Mr. Jeremiah D. Tablatin

Capstone Project Coordinator

Mr. King Jasser N. Andaya

Program Head

November 23, 2024

ACKNOWLEDGEMENT

The researchers would like to express their deepest gratitude to all those who have supported this Capstone Project.

First and foremost, the researchers would like to thank the Advisor of this project, **Ms. Darryl Pauline S. Nietes**, for her invaluable guidance, encouragement, and continuous support. Her expertise, constructive feedback, and unwavering patience have been essential to the success of this project.

The researchers, also would like to extend their sincere appreciation to **Mr. Ricson M. Ricardo**, the client for this project, providing the proponents the opportunity to work on such an engaging and meaningful project. His collaboration, insightful feedback, and trust in this work have been vital in shaping the direction and outcome of this project. The knowledge and resources he shared have been invaluable.

Additionally, the researchers would like to acknowledge the contributions of the colleagues, friends, and family members. Their feedback, moral support, and understanding have helped the proponents to stay focused and motivated during the challenging phase of this project.

Finally, the researchers are deeply grateful to **STI COLLEGE ALABANG** for providing resources and environment that have facilitated the academic and professional growth throughout this journey.

This Capstone Project would not have been possible without the encouragement and support of all these individuals and institutions.

ABSTRACT

Title of research: **CHRONOSCHEDIO: IMPLEMENTATION OF A WEB-BASED CLASS SCHEDULING AND FACULTY LOADING MANAGEMENT FOR STI COLLEGE ALABANG**

Researchers: **Bianes, Stalen Hitler N.**
Diamla, James Roland M.
Ibarreta, Richard Rodney T.
Labaclado, John Simon Lee B.

Degree: **Bachelor of Science in Information Technology**

Dare of Completion: **November 2024**

Keywords: **Class Scheduling, Faculty Loading Management**

Chronoschedio, is a class scheduling and faculty loading management are crucial aspect of academic administration in STI College Alabang, ensuring that educational institutions operate efficiently while meeting the needs of the faculty and students. This capstone project aims to explore the challenges and solutions in creating optimized schedules that balance course availability, room allocation, faculty loads, and conflicts resolutions.

The study also evaluates faculty load management to prevent overburdening instructors while insuring balance distribution of teaching hours.

By leveraging web hosting and securing a dedicated domain, the project aims to establish a widely and secure platform for day-day operations. The adoption of a web base application framework aligns with institution standards, providing a polished and professional interface for all users.

The primary objectives is to offer a system that support strategic tasks, improves utilization, enhance student learning experience, and addressing the academic needs of students.

TABLE OF CONTENTS

	Page
Title Page	I
Endorsement form for Proposal Defense	II
Approval Sheet	III
Table of Contents	IV
List of Figures	V
List of Notations	VI
Introduction	1
Project Context	1-2
Purpose and Description	2
Objectives	3-4
Scope and Limitations	4-10
Review of Related Literature/Systems	10
Related Literature	10-11
Related Studies and/or Systems	12-17
Synthesis	17-19
Technical Background	19
Overview of Current Technologies to be Used in the System	19-23
Calendar of Activities	23-24
Resources	25-26

Methodology, Results, and Discussion	27-29
Requirements Analysis	29-33
Requirements Documentation	33-40
Design of Software, System, Product, and/or Processes	40
Development Testing	54
Conclusions and Recommendations	73-72
Recommendation Letter	72
References	73
Appendices A	74
Resource Persons	75
Appendices B	76
Relevant Source Code	256
Appendices C	257
Evaluation Test Documents	258- 264
Appendices D	265
Sample Input/Output/Reports	266-270
Appendices E	271
Users Guide	272-278
Appendices F	279
Curriculum Vitae	280-283
Transcript of Interview	284-291

LIST OF FIGURES

Figure 1 Entity Relationship Diagram	41
Figure 2 Data Flow Diagram.....	42
Figure 3 Login Flowchart.....	43
Figure 4 Superuser Flowchart	44
Figure 5 Academic Head Flowchart.....	45
Figure 5.1 Academic Head Flowchart.....	46
Figure 5.2 Academic Head Flowchart.....	47
Figure 5.3 Academic Head Flowchart.....	48
Figure 6 Program Head Flowchart.....	49
Figure 7 School Registrar Flowchart.....	50
Figure 8 Faculty Flowchart.....	51
Figure 9 Student Flowchart.....	52
Figure 10 Profile Module Flowchart	53

LIST OF NOTATIONS

AJAX ver. 3.7	Asynchronous JavaScript and XML
CANVA	Multi-Media Platform
DATATABLE	A plug-in for the jQuery JavaScript library
IDE	Integrated Development Environment
FIGMA	A Browser-Based Web Design Tool
FULLCALENDAR	A JavaScript object that store information about Calendar Event
GODADDY	Domain registration, web hosting, and online service
LUCID FLOWCHART	A Web-Based Diagramming Application
MICROSOFT AZURE	Cloud computing platform
POSTMAN	Postman is a versatile API platform that simplifies the development, testing, and management of API's

INTRODUCTION

Project Context

Creating a well-organized academic environment is important, and effective class schedules are the key to achieving this goal. Class Scheduling is crucial, ensuring both students and faculty can attend all courses without conflicting with other schedules in different courses. As well as distributing manageable workloads for the faculty. However, manually creating these class schedules in excel is a complicated and time-consuming process. The challenge of creating a class schedule lies in managing a complicated set of competing demands.

Equally important is ensuring a fair distribution of workload among faculty, considering the faculty expertise and the demands of each course. The ability of the academic head to create or revise the class schedules to accommodate sudden changes, such as new course offerings or faculty adjustments. Finally, resource optimization is important. Maximizing room use and avoiding scheduling conflicts with faculty are essential for resource efficiency. Therefore, the researchers have explored solution to this challenge, the researchers come up with the "Chronoschedio: Implementation of a web-based class scheduling and faculty loading management" system.

This study aims to ensure the effectiveness of the class scheduling and faculty loading management system for STI College Alabang. This research highlights the need for an innovative approach whenever changes occur in student enrollment, course offerings, or faculty availability. The primary goal is to ensure the system's effectiveness by reducing the time spent, and conflicts in creating class schedules. These improvements not only enhance accuracy but also free up the academic head for other responsibilities. The system builds up collaboration among students, faculty, registrar, program heads, and academic head by facilitating communication regarding class schedule changes within the institution.

The system will be used by the academic head, faculty, registrar, and students, and by the program heads. Chronoschedio will not be limited only to the academic head when it comes to creating class schedules, especially in the growing population of STI College Alabang. Creating class scheduling in Chronoschedio will be decentralized, it will be efficient as the class scheduling will be distributed with the program heads. The continuous work of creating class schedule by the academic head and program heads will be effective to finish the schedule within a shorter period of time. Additionally, this decentralized approach among program heads leads to greater alignment between scheduling decisions and program objectives.

The system aims to improve the overall academic experience. That is why Chronoschedio will also utilize the office 365 account provided by the STI College Alabang. Therefore, registering manually to Chronoscedio is not necessary. This capstone project presents an exciting opportunity to apply software principles to solve a real-world problem in education. A class scheduling and faculty loading management system can improve operational efficiency and contribute to the overall success of the educational institution.

Purpose and Description

This project's purpose is to develop and implement Web-Based Class Scheduling and Faculty Loading Management for STI College Alabang. The manual class scheduling in excel is inefficient due to human-errors and conflicts on the schedule. In creating manual class scheduling in centralized approach is time-consuming as well, it is not effective particularly in the growing institution of STI College Alabang. The system aims to automate the class scheduling to lessen the time needed, reducing the conflicts, faculty loading, when creating a class schedule. The system uses a decentralized approach to distribute the class scheduling ability to the program heads for better and faster schedules.

Objectives

The goal is to implement a Web-Based Class Scheduling and Faculty Loading Management that organizes the class scheduling, faculty loading, and utilizing the dedicated Office 365 accounts in the institution to access the system. The registrar will also have access to the system to see the schedule made by the academic head.

- **To develop a module enabling login for the users through their provided credentials**

This module will allow the users except for the super-user to access the system by logging in using their credentials, ensuring a secure and personalized experience.

- **To develop a module that manages users within the system.**

This module will allow admin/super-user to manage the users.

- **To develop a module that monitors all user's activity within the system**

This module allows admin/super-user to monitor the activity logs of the users.

- **To develop a system that incorporate drag and drop functionality in scheduling**

The scheduling feature of the system will support drag and drop, enabling the academic head and program heads to easily change and modify class schedules by dragging and dropping course loads into new time slots. This feature enhances user experience, streamlining the scheduling process and enabling greater flexibility in creating class schedule and faculty loading.

- **To develop a module that monitors and manages different course schedules.**

This module will allow the academic head to monitor and manage the course schedules.

- **To develop a system that has real-time monitoring of class schedules.**

The system has real-time monitoring of class schedules that will easily be navigated by the users.

- **To develop a system that will view class schedules effectively.**
The system will allow the users to be able to view schedules effectively.
- **To develop a module that can ensure the courses have their designated time slot and allow adjustment when needed.**
This will allow the academic head to ensure the courses have their designated time slot and allow swift adjustment when needed.
- **To develop a module that will assign loads to the faculty.**
This module will allow the academic head to assign loads to the faculty.
- **To develop a module that allows Academic head, Program head, Faculty, and Registrar to export their class schedules.**
The module will ensure accessibility and ease for academic planning and time management by allowing specific users to export their class schedules.
- **To develop a module that will assign class schedules for courses that aligns to their departments.**
The system will allow the program heads to create class schedules that align to the specific program.

Scope and Limitations

This project aims to create an efficient system for organizing class schedules and managing administrative tasks in an educational setting. The intended users include superusers, academic head, program heads, school registrar, faculty, and students. All users will benefit from user-friendly access and management tools tailored to their specific needs. The system aims to enhance the accuracy, efficiency, and user experience of various administrative and academic processes.

Scope

Superuser Side (Back-end)

- **Login Module**

In this module, the Superuser must provide their user and password to access the system. The system is configured with predefined admin accounts by default, ensuring that only authorized people have access to administrative functions.

- **User Management Module**

This module enables the Superuser to manage user accounts, including adding, editing, archiving, and assigning roles. The Superuser can also reset passwords and batch upload multiple users. With a single interface for managing roles and access privileges, the module ensures that only authorized individuals access specific functionalities. The Superuser's oversight is essential for maintaining system security and integrity.

- **Activity Logs Module**

This module allows the superuser to monitor all users' necessary actions. Data updates, and configuration adjustments are recorded.

Academic Head Side (Back-end)

- **Login Module**

This module enables the Academic Head to securely access the system using their Office 365 account.

- **Dashboard Module**

This module serves as the main landing page, providing an overview of access for the academic head to view detailed room utilization reports.

- **Class Schedule Module**

This module enables the academic head to view and see all the created schedules.

This module provides the academic head with an overview of the whole timetable, including class times, rooms, and allocated teachers categorized by sections and programs. The module has the function to export schedules in an excel format, making them easier to distribute and record.

- **Curriculum Management Module**

This module manages the curriculum records. It provides the academic head with centralized control over curriculum-related tasks, enabling them to organize academic programs, manage curriculum details, and optimize course scheduling within the educational institution.

- **Academic Programs**

This submodule is responsible for overseeing the assigned curriculum within an educational institution. It provides the academic head with a centralized platform for adding academic programs and their corresponding curriculum years. This submodule provides an efficient and comprehensive tool for organizing academic classes in the system.

- **Program Curriculums**

This submodule facilitates the management of curriculums specific to selected academic programs and year. It offers a centralized platform for creating and maintaining curriculums, which include curriculum code, course code, course title, year-term, lecture room and laboratory duration. This submodule ensures efficient management of academic programs within the system by allowing academic head to customize curriculums based on the requirements given by the Commissioner on

Higher Education (CHED).

- **Course Scheduling Module**

This module allows the academic head to assign class schedules in rooms for loaded subjects based on the section chosen. The process is aided by a simple drag and drop function and conflict checker, which allows efficient allocation of classes to appropriate rooms throughout the institution.

- **Maintenance Module**

This module provides the academic head with tools to manage rooms, sections, and operational hours for the calendar utilized in the Course Scheduling Module. Tasks such as adding, archiving rooms and sections are completed in this module to ensure that the scheduling process meets institutional requirements. Additionally, the module allows the academic head to set operational hours within the calendar, giving more flexibility and control over class and academic activity scheduling.

- **Instructor Module**

This module provides the academic head with a centralized platform for managing instructors information and qualifications. It allows the academic head to view and update instructor profiles. This module improves administrative oversight by ensuring accurate records of instructor credentials across the institution.

- **Faculty Load Module**

This module allows the Academic Head to assign faculty members to specific program codes and curriculum codes. The Academic Head may effectively manage faculty assignments using this module, making sure each faculty member is assigned to the right program and curriculum. This procedure makes it easier to manage faculty workloads in an efficient and organized way, which guarantees that academic programs have enough faculty members and that curriculum

requirements are met.

- **Activity Logs Module**

This module allows the academic head to keep an eye on every activity within the system. When the schedule has been exported and update a contact number.

- **Profile Module**

This module enables user to view and update their personal contact number within the system.

Program Heads Side (Back-end)

- **Login Module**

This module enables the Program Heads to securely access the system using their Office 365 account.

- **Course Scheduling Module**

This module allows program heads to assign class schedules in rooms for loaded subjects that correspond to their departments. A drag-and-drop capability simplifies the procedure, allowing for efficient class distribution to appropriate rooms throughout the educational institution.

- **Profile Module**

This module enables user to view and update their personal contact number within the system.

School Registrar Side (Front-end)

- **Login Module**

This module enables the school registrar to securely access the system using their Office 365 account.

- **Class Schedules Module**

This module enables the school registrar to see all class schedules made by the academic head. This module provides the school registrar with an overview of the whole timetable, including class times, rooms, and allocated teachers categorized by sections and programs. The module allows the school registrar to export schedules in an excel format, making them easier to distribute and record.

Faculty Side (Front-end)

- **Login Module**

This module enables the faculty to securely access the system using their Office 365 account.

- **Class Schedule Module**

This module enables the faculty members to have comprehensive access to the class scheduling system, efficiently managing their teaching responsibilities. Faculty members not only view their assigned courses but also gain insights into their workload and class schedules, facilitating better time management and planning. The faculty members can also export their respective class schedules.

- **Profile Module**

This module enables user to view and update their personal contact number within the system.

Student Side (Front-end)

- **Login Module**

This module enables the faculty to securely access the system using their Office 365 account.

- **Class Schedule Module**

This module enables the students to view assigned schedules.

Limitations

- Creating class schedules are more compatible with larger screens (1920x1080p).
- The system is specifically for STI College Alabang.
- The system is exclusively for bona fide students of STI College Alabang.
- The academic year/term section projection or section prediction will be created outside the system.
- The system will use Microsoft 365 Login for account authentication.

REVIEW OF RELATED LITERATURE/SYSTEMS

Related Literature

Traditional Approaches to Scheduling

Historically, academic institutions relied on manual methods for scheduling and managing faculty workload. However, these traditional approaches often resulted in inefficiencies, scheduling conflicts, and poor allocation of resources. Some challenges associated with manual scheduling processes, including time constraints, subjective decision-making, and difficulty in accommodating changing academic demands. The integration of technology, automated algorithms, and transparent management practices has transformed traditional scheduling methods, offering unprecedented opportunities to optimize resource allocation, enhance faculty satisfaction, and promote student success. As academic institutions continue to adapt to changing educational landscapes, the role of in-class scheduling system and faculty loading management will remain essential in institutions.

Class Scheduling System and Faculty Loading Management

In-class scheduling systems and faculty loading management are critical components of academic institutions, ensuring efficient resource allocation and effective utilization of

faculty expertise. These systems encompass the organization of class schedules, room assignments, faculty workload distribution, and optimization of teaching and learning experiences.

Advantages of Using Web-Based Class Scheduling System

Web-based Class Scheduling System offers numerous advantages for educational institutions, faculty and students. Web-based scheduling systems simplify the scheduling process, reducing errors, and saving time for administrators, faculty and students. This provides centralized platforms for creating, updating, and managing schedules allowing for more efficient time management and improved organization.

Automated scheduling software minimizes room conflicts by allocating classrooms based on availability ensuring optimal use of campus facilities and preventing double-booking

These reduced schedule conflicts and providing clear, accessible schedules that cater individual preferences and availability.

The adoption of web-based class scheduling system can reform scheduling processes in educational institutions, leading to increased efficiency, improved satisfaction among faculty and students, and data-driven decision making to enhance the administrative decisions.

What is the difference of Web-Based Class Scheduling on other existing studies

The Web-Based Class scheduling system that the researcher built can be accessed from anywhere with internet connection, allowing for greater flexibility for users and faculty. This reduces the time and effort required for manual scheduling, which focuses on many scheduling studies that propose new algorithms. It also can store, and manage large amounts of data related to scheduling, such as course information, instructor availability, and room capacity. By having a simple drag and drop the users can easily create a schedule that doesn't require so much effort so that the Academic Head can do other tasks.

Having accurate class scheduling the user can have less time consuming and less human error and conflicting schedules so he can do some another extracellular activity.

Related Studies and/or Systems

Local Studies/or System

The research conducted on class scheduling systems highlights the importance and efficient scheduling in educational institutions. The studies emphasize the significance of accurate and reliable class schedules to prevent conflicts, ensure resource utilization, and meet the needs of the students and faculty members.

The studies reviewed the functionality of automated class scheduling system in terms of speed, data handling, accuracy, security, stability, and adaptability. They compared manual and automate systems, highlighting the advantages of automated systems in solving scheduling challenges and improving overall functionality.

Automation of Curriculum-Based Student-Subject Encoding: A Web-Application, Caraga (2022)

In this article, a web-based application using Django was developed to automate the student-subject encoding process, based on the curriculum, the faculty evaluates the student's eligibility to take the specific subject and creates a conflict-free timetable based on offered subjects and schedule plotted by the academic leaders.

Timetabling is a procedure of arranging examinations or class events into a specific time slot. Universities carries many timetables recurring problems, one of which is the demand to schedule subject offerings with no collisions in space, time and resources.

At Caraga State University, scheduling was done by the department chairs, and the faculty carried out the enrollment. The heavy involvement of faculty diverts their focus, accommodating enrollment requests instead of preparing their lecture for the upcoming opening of classes.

With this, a web-based application using Django was developed to automate the student-subject encoding process. (Mark P. & Vicente P., 2022)

A Proposed Faculty Loading Guide Framework for the Subject in the Senior High School in the Philippines, Laguna (2022)

In University of the Philippines, facilitation of K to 12 is also known as “Kindergarten to grade 12”, is an education system under the Department of Education that aims to produced Filipino citizens in the global market. New Subjects to the high school faculty members in the University of the Philippines encounters different issues and challenges. This paper proposed a faculty loading guide model for research subjects that can be used by school administrators to guide them in distributing research subjects to the faculty members. (Renell C., 2022)

Development of a Web-Based Pre-Loading System for Faculty in a University, Cebu (2019)

Assigning teaching loads is one of the most challenging tasks of deans in colleges and universities, especially during the enrolment period. They usually encounter problems in assigning teaching loads, faculty qualifications, and faculty evaluation by students. Therefore, the researchers made survey questionnaires that administered to the nine deans, 11 chairmen of disciplines, and six IT experts and end-users from the University of Cebu. The data were tabulated, analysed and interpreted using simple percentage, universal sampling and ranks. The results showed that the development of the Web- Based Pre-Loading System was effective and efficient since the system was highly accepted. (Bell C., Jonathan E. et al, 2019).

Application of Representation and Fitness Method of Genetic Algorithm for Class Scheduling System, Isabela (2019)

This paper presents a solution aimed at addressing the issue of poor schedule management, a significant challenge faced by Isabela State University-Main Campus. Scheduling is a crucial process, improved through the adoption of the Representation and Fitness Method of Genetic

The study demonstrated that this method effectively addressed the identified problem.

The Representation Method facilitated the creation of a scheduling template for schedule plotting, while Fitness Method determined the functionality and usability of the generated template. The researchers assessed the solution using criteria from ISO 9126 Standard. Result indicated that the Genetic Algorithm's methods enhanced scheduling accuracy, reduced time consumption, and minimized conflicts in schedules. For future studies, it is recommended to refine the fitness function to incorporate additional scheduling components, such as individual schedules for regular and irregular students, integration of campus extension, and consideration of other indicators form the assessment instrument. (Freddie L., Sheena T. et al., 2019)

Web Based Course Scheduling System Using Greed Algorithm, Manila (2019)

This study aims to develop a web-based course scheduling system for Colleges of Computer Studies under FEU institute of technology, Manila Philippines that make used of Greed Algorithm for managing schedules on courses and assigning to faculty. The method of this research was research and development (R&D). The researcher also discusses the scheduling process attributed to Scheduling Module System that follows the Greedy Algorithm. The system has been tested and supportability (John L., Roman A. et al., 2019)

Foreign Studies and/or System

Foreign studies on class scheduling systems highlight the crucial role of efficient scheduling in educational institutions. These studies underscore the importance of accurate and reliable class schedules to prevent conflicts, optimize resource utilization, and meet the needs of both students and faculty. Additionally, they emphasize the significance of effective communication tools and considering student preferences. The research provides valuable insights and recommendations for improving the scheduling process, enhancing stakeholder communication, and promoting better class scheduling practices.

Multi-Agent Negotiation System for Class Scheduling (2022)

In this paper, a multi-agent-based negotiation-based class scheduling system is proposed, which consists of student agents, professor agents, and negotiation agents, each agent acts in accordance with its respective human user's preference and performs the repetitive tedious process instead of the user.

The current class scheduling has difficulties in reflecting students' preferences for the classes they want to take and forecasting their demands. The system showed that it could produce a feasible scheduling solution minimizing the costs and reflecting the users' preferences, and the performance was evaluated by a class success ratio. (Seongju P. & Cheolhyun K., 2022).

Computer Intelligent Course Scheduling System Based on Deep Learning, China (2022)

In recent years, within the continuous development of computer technology, the successful application and penetration of computer technology in all aspect of society has made people more and more aware that the problem of school class scheduling can be solved by computer technology. This purpose is to study the design of a computer intelligent course scheduling system based on deep learning. Regarding the problems existing in the current intelligent course scheduling, starting from the needs of users, and based on many research results in the field of intelligent course scheduling at home and abroad, a BP neural network is proposed to provide decision-making for genetic algorithms to achieve system self-adaption method of scheduling. The practical problems of course arrangement is tested, the quality and efficiency of course, arrangement results are analyzed and evaluated, and the practicability of the system discussed. (Xuyue R. & Chongwei L., 2022).

Rigid Class Scheduling and It's Value for Online Learning in Higher Education, Russia (2022)

The goal of this study was to evaluate the effectiveness of using a planned class scheduling method in online learning while accounting for the limits imposed by the COVID-19

pandemic. The study involved 226 students and 54 teachers from the Moscow Aviation Institute and Kazan Federal University. The study employed a sociological survey to identify key obstacles in online learning from the perspective of students. According to the data, 35% of students recognized a lack of self-discipline as a major concern, which might be addressed by a clearly defined class schedule (37%), as well as a test schedule (29%). Subgroup research found that subgroups 1, 2, and 5 had the most efficient learning process. Recommendations for improving online education included establishing time limits. (Aleksandr G, Ekaterina S. et al., 2022).

Toward Achieving Quality in Faculty-Load Allocation: A developed Faculty-Load-Management System, Saudi Arabia (2021)

In, King Saud University, faculty members have multiple duties besides teaching and researching, community service, and administrative work. Maintaining balance among these different tasks are a major issue in academic institutions, as it is imperative for academic administrators to ensure that the faculty members work productively and be able to concentrate on their tasks. However, the administrative work cannot be avoided, and the workload depends on the number of people in the department. In this paper, the author describes the case on gathering the workload information and proposes to develop a faculty-load-management system (FLMS) for their university.

This proposed system manages workload allocation and time-conflict detection which can integrate and help organize the workload for the whole department effectively. (Duaa Hamed AlSeed, 2021).

Mathematical Model for Course Timetabling Problem with Faculty-Course Assignment Constraints, Thailand (2021)

University course timetabling problem includes the challenging task of generating an automated timetable for courses under resource limitations. Manual generated timetables might hold some errors and consume long time to create feasible solutions, thus there is a need

to find optimal and fast solutions for these problems, the further timetabling problem increased when tackling faculty-related-constraints.

The goal is to maximize events assignments and faculty-member preferences satisfaction while balancing the university requirements while minimizing student learning days and unassigned events. The model is tested with eight real-world instances. The proposed method can generate optimal timetables for all problem instances that satisfy faculty constraints. (Haneen A. & Wasakorn L., 2021)

Synthesis

The mentioned studies, under FEU institute of technology, a web-based course scheduling system for College of Computer that make use of Greed Algorithm. It follows the schedule module, and the system has been tested and supported, that manages schedules, courses and assigning faculty.

In Isabela State University-Main Campus, the paper aimed to address the issue of poor scheduling management. The researcher presents a solution to improve the issues through Application of the Representation and Fitness Method, the system methods enhanced scheduling accuracy, reduced time consumption, and minimized conflicts in schedules. For future studies, it is recommended to refine the functions to incorporate additional scheduling components, such as individual schedules for regular and irregular students, integration of campus extensions.

The article discusses the development of a web-based application using Django aimed at automating the student-subject encoding process. It focuses on addressing timetabling challenges faced by universities, particularly Caraga State University, where scheduling was traditionally done manually by department chairs. The developed application evaluates students' eligibility for specific subjects based on the curriculum, creates no conflicts of timetables and streamlines the enrollment process.

The University of Cebu addresses the challenges faced by deans in assigning teaching loads in colleges and universities, particularly during enrollment periods. The researcher conducted surveys among nine (9) deans, eleven (11) discipline chairmen, and six (6) IT experts and end-user from the University of Cebu. The data involved tabulation, simple percentage calculation, universal sampling, and ranking. The findings indicate that the developed Web-Based Pre-Loading System was highly effective and efficient, receiving wide acceptance.

The department of education aims to prepare Filipino students for global market. At the University of the Philippines, the integration of new subjects under this system has presented significant challenges for high school faculty members. The paper proposed a faculty loading guide model tailored for research subjects. The model is intended to help school administrators efficiently allocate research subjects to faculty members, addressing the issues encountered with the K to 12 implementations.

The advancement of computer technology has brought about a realization that school class scheduling challenges can be effectively addressed through technological solutions. This study aims to design a computer intelligence course scheduling system, this is to consider user needs. It proposed the use of BP neural network to inform decision-making genetic algorithms, enabling the system to adapt scheduling methods autonomously.

The study aimed to assess the effectiveness of planned class scheduling online learning during the COVID-19 pandemic. The surveys conducted of 226 students and 54 teachers from two universities revealed that 35% of students identified a lack of self-discipline as a major issue, thus they suggested that a clear schedule could help. The group analysis highlighted implementing to improve online education.

The paper addresses the challenges of balancing various duties of faculty members at King Saud University. Administrative tasks, community service, and other responsibilities often strain productivity. Then the author proposes a Faculty Load Management System (FLMS)

to gather workload information and effectively allocate tasks, integrating workload management and time-conflict detection for the entire department.

The university course timetabling problem involves the challenging task of creating automated schedules for courses under resource constraints. Manually generated timetables are often error-prone and time-consuming, necessitating the development of optimal and efficient solutions. This problem becomes even more complex when addressing faculty-related constraints. The goal is to maximize event assignments and satisfy faculty preferences while balancing university requirements, minimizing student learning days, and reducing unassigned events. This paper proposes a model tested with eight real-world instances, demonstrating its ability to generate optimal timetables that meet all faculty constraints.

TECHNICAL BACKGROUND

Overview of Current Technologies to be Used in the System

A combination of contemporary web development technologies is needed to create an intricate class scheduling system that guarantees a secure, seamless, and responsive user experience. Combining PHP, HTML, CSS (including Bootstrap), JavaScript, and AJAX, the system is constructed. Each of these technologies contributes to both the front-end and back-end functionality, making them essential to the entire architecture. The usage of each language and resource to create a productive and easy-to-use class scheduling application is explained in detail below. Canva and Figma for creating diagrams such as flow- charts, data flow diagram and entity relationship diagram.

It manages several server-side functions, including data processing, client request management, and MySQL database interaction. PHP is the best choice for creating dynamic web applications because of its many built-in capabilities and broad library support. Furthermore, PHP's session management features make it easier to authenticate and authorize users, enabling strong security controls and role-based access control. The system's web pages are structurally supported by HTML (Hypertext Markup Language). To show facts in a logical and understandable manner, it arranges content and incorporates multimedia components. The document is organized using HTML tags into sections, paragraphs, headings, forms, links, and other components. Search engine optimization (SEO) and improving the user experience depend on this semantic framework. Furthermore, the integration of FullCalendar and DataTables further enriches the system functionality. While DataTables permits dynamic and responsive data presentation, improving usability and engagement withing the system, FullCalendar allows for intuitive and interactive scheduling functionalities. These integrations help to create a comprehensive and effective web application, especially when combined with PHP's

adaptability.

AJAX (Asynchronous JavaScript and XML) ver. 3.7

AJAX (Asynchronous JavaScript and XML) allows for the asynchronous exchange of data between the client and server, improving the user experience by enabling partial page updates. This means users can interact with the application, such as adding or dropping classes, without experiencing full page reloads. AJAX enhances the responsiveness and efficiency of the application, making it more user-friendly.

Figma

Figma is a cloud-based design tool that allows for interface design, prototyping, and collaboration. It is especially popular with UI/UX designers and teams working on online and mobile app projects. Unlike traditional design tools, which require software installation and separate files, Figma is totally browser-based, allowing team members to collaborate in real time regardless of their location. There are few key features in Figma such as Vector Editing, Prototyping, Components and Styles, and collaborations.

Canva

Canva is a known online graphic design platform that lets users generate a variety of visual content, such as presentations, social media graphics, posters, papers, and more. It has a user-friendly interface with drag-and-drop capabilities and a large library themes, photos, graphics, and fonts, making it suitable for both beginners and pros. Canva also offers collaboration capabilities that allow several people to work on the same project at the same time, as well as free and paid subscription options that include additional features and resources. Overall, Canva has become a popular tool for consumers and organizations seeking to produce visually appealing material without requiring significant design abilities or tools.

FullCalendar

FullCalendar is a versatile JavaScript library for creating interactive and customizable

calendars on web pages. It supports various view types (month, week, day, list) and integrates seamlessly with different event sources like JSON, Google Calendar, and ICS files. Users can interact with the calendar through features like event creation, modification, drag-and-drop, and resizing. Customization options include themes, CSS classes, localization, and plugins for enhanced functionality. Its compatibility with popular JavaScript frameworks (React, Angular, Vue) and server-side technologies makes it a powerful tool for building robust scheduling and event management applications.

GoDaddy

GoDaddy is one of the world's leading platforms for domain registration, web hosting, and online services designed to help businesses, entrepreneurs, and individual establish and grow their digital presence. GoDaddy has evolved into a comprehensive provider of tools for website creation, hosting, online marketing, and e-commerce solutions.

DataTables

DataTables is a powerful jQuery plugin that enhances HTML tables with advanced interaction features. It provides functionalities like pagination, instant search, sorting, and filtering, making it easy to manage and display large datasets. Customizable through extensive options and flexible API, DataTables supports server-side processing for efficient data handling. It integrates well with various frameworks and libraries, allowing for seamless integration into web applications. DataTables also offers a range of extensions for additional capabilities such as responsive design, export buttons, and fixed columns.

Lucidchart

Lucidchart is a powerful diagramming tool designed for creating and sharing a variety of visual representations, such as flowcharts, organizational charts, mind maps, wireframes, and network diagrams. Its user-friendly drag-and-drop interface, extensive shape library, and customizable templates make it easy to design professional visuals for diver's

industries. Lucidchart supports real-time collaboration, allowing teams to work together seamlessly with features like comments, version history, and integrations with tools like Google Drive, Microsoft Office, Slack, and Jira. Advance functionalities include data linking, conditional formatting, and auto-generation of diagrams from imported data. With its cross-platform accessibility and secure role-based permissions, Lucidcharts is an excellent tool for planning, organizing, and presenting ideas effectively in both academic and profession environment.

Microsoft Azure

Microsoft Azure is a cloud computing platform and ecosystem provider by Microsoft that allows users to build, test, deploy, and manage applications and services through a network of Microsoft-managed data centers. Azure offers a broad range of services, catering to diverse business needs, including computing power, storage solutions, analytics, networking, and artificial intelligence. It supports various programming languages, framework, and tools, making it a versatile choice for organization ranging from startups to global enterprises.

Postman

Postman is a versatile API platform that simplifies the development, testing, and management of API's. It allows user to create and organized API requests, test endpoints, and automate workflows with its intuitive interface. Postman support various HTTP methods (GET, POST, PUT, DELETE, etc.) and enables user to configure parameters, headers, and authentication settings for comprehensive testing. It offers features like automated test scripts, monitoring, and debugging, which streamline the API cycle. Postman also facilities collaboration by allowing teams to share collections, environments, and documentation in real time.

Calendar of Activities

Brainstorming

In this phase, the proponents discuss comprehensively the difficulties of STI College Alabang. During the conversation and exchange of ideas, the proponents come up with unique and innovative ways on how we can help STI College Alabang with using the system that we will develop.

Researching

The proponents did research for the development of our system; we do intensive interviews especially around STI College Alabang campus, on how class scheduling and faculty load management operates and how we could help improve the school system.

Presentation of Project Proposal

The proponents presented different three (3) Titles that we will pursue and develop, we encounter challenges and difficult decisions which we had hard time. At the end of our presentation with the panelists, our group decided to develop class schedule system and faculty loading management for STI College Alabang.

Chapter 1

The developers created this chapter from the first of March to the last week of April. It includes the project context, purpose and description, objective, and scope and limitations.

Chapter 2

The developers created this chapter from March until April. Includes the review literature, related studies and/or systems, and synthesis.

Chapter 3

The developers created this chapter from March to May. It includes the overview of the current technologies used in the current system, calendar of activities, and resources.

Chapter 4

The developers continue to work on the methodology, results and discussion, these includes the requirement analysis, requirement documentation and the design of software, system, product and process.

ERD/Flowchart

The developers created the ERD/Flowchart from the third week of March until the first week of May. It contains the flow of the system and the attributes.

Context Diagram/Data Flow Diagram

The developers created Context Diagram/DFD from the third week of April until the third week of May. It contains the data flow of the system and the attributes.

Wireframing

The developers created the wireframing from the third week of March until the third week of May. It contains the outline and the flow of the system.

Gantt Chart of Activities



MONTH																																	
ACTIVITY	FEBRUARY	MARCH	APRIL	MAY	JUNE	JULY	AUGUST	SEPTEMBER	OCTOBER	NOVEMBER																							
Requirement Gathering																																	
Brainstorming																																	
Researching																																	
Presentation of Project Proposal																																	
Documentation																																	
Chapter 1: Introduction																																	
Chapter 2: Review Related Literature																																	
Chapter 3: Technical Background																																	
Chapter 4: Methodology, Results and Discussions																																	
<u>Chronoschedio</u> Creation																																	
Web Application																																	

Resources

Effective utilization of the available resources is critical to the success of a capstone project.

To create and run proposed system, the researcher used computer/laptop with the following specifications:

SOFTWARE DEVELOPMENT	
Operating System	Windows 11 Home Single Language 64-bit (10, 0 Build 22631)
Programming Language	PHP, HTML, CSS (including Bootstrap), JavaScript, AJAX
Integrated Development Environment	Visual Studio Code v1.89
Database	MySQL
Documentation	Microsoft Office 365
Server	Xampp 3.3.0

HARDWARE DEVELOPMENT	
Processor	AMD Ryzen 5 5600H
GPU	NVIDIA RTX 3060
RAM	24GB DDR 4
Storage Storage	500GB

Minimum Hardware Requirements (For Development):

	MINIMUM	RECOMMENDED
Processor	Intel Core i3-10100 @ 3.6- GHz (4 cores, 8 threads)/ AMD Ryzen 3 3200G @ 3.6GHz (4 cores, threads)	Intel Core i5-11400 @ 2.60GHz (6 cores, 12 threads) / AMD Ryzen 5 5600X
Memory	8GB DDR4	3.7HGz (6 cores, 12 threads) 16GB DDR4
Graphics	Integrated Graphics (e.g., Intel UHD Graphics 630 / AMD Radeon Vega 8)	Dedicated Graphics (e.g., NVIDIA GeForce GTX 1660 6GB / AMD Radeon RX 5600 XT 6GB)
Storage	128GB	256GB or more

Minimum Hardware Requirements (For Users):

Processor	Intel Core 2 Duo E8400 @ 3.0GHz / AMD Athlon II X2 250 @3.0GHz.
Memory	4GB RAM
Graphics	Integrated sound card (100% DirectX 10 compatible)

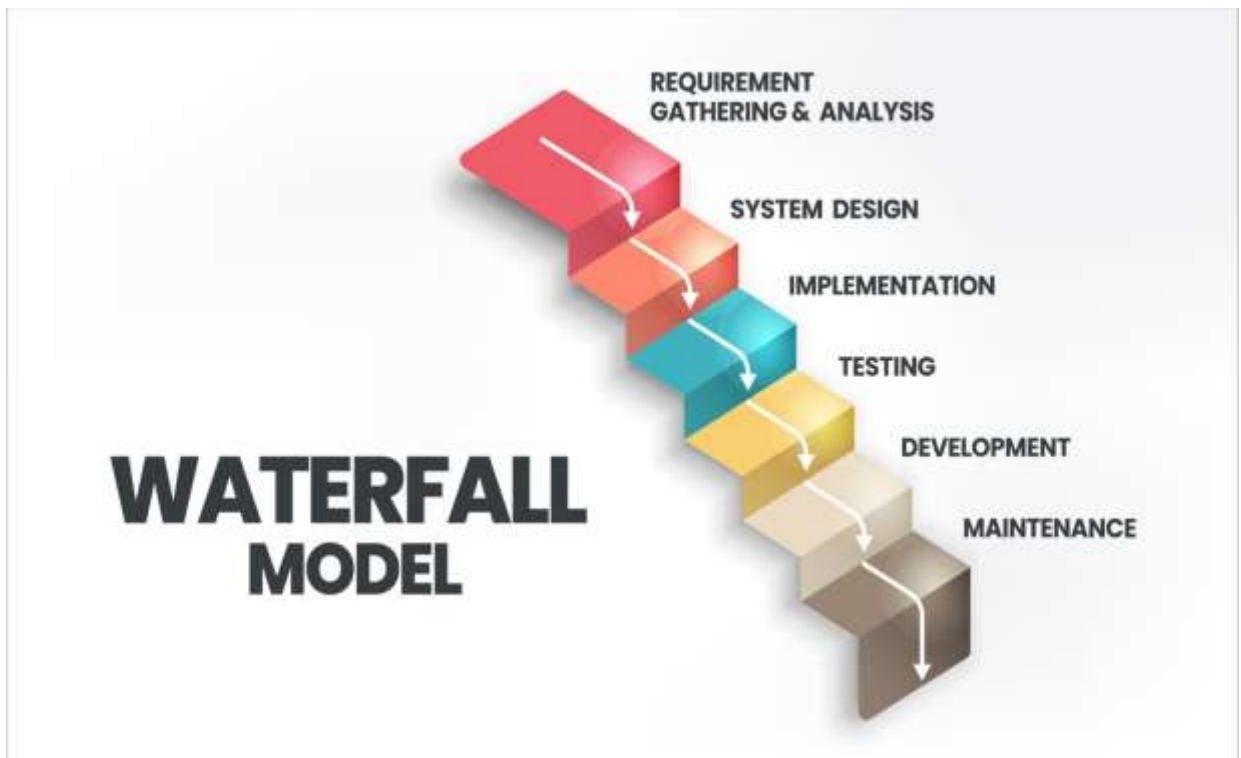
METHODOLOGY, RESULTS, AND DISCUSSIONS

In this part, various analytical techniques, software development processes, system design figures, and conducting outcome of the project is presented. The proponents have utilized a variety of ways to identify the specific capabilities and features of the system, then use these as a reference to build the system. This section includes the requirements analysis and documentation, design of software and processes, development and testing, system prototype and description, implantation plan, and results.

The system development approach that the opponents have used in this study is the Waterfall model. This method is a conventional, sequential paradigm used in the development of software and systems. It adopts a top-down, linear, sequential methodology in which the development processes move through several clearly define phases. Before going on to the next step, each one must be finished, and modification made in one phase could not be readily incorporated into succeeding stages without a significant amount of effort.

The researchers believe that the waterfall method would be the best fitting approach for the development of the system for class scheduling and faculty loading management in STICA, as the method employs the development of the system from requirements analysis up to system maintenance. The waterfall method is the preferred approach for the development of this project. Figure 1 shows the diagram illustration of the waterfall model.

The waterfall model is comprised of six (6) stages namely, requirements gathering, design, implementation, testing, deployment, and maintenance.



Stage 1: Requirement Gathering

During this stage, the project requirement is obtained and well documented. The objectives, scope, and requirements of the project are all included in this. It is also during this stage the researchers have gathered the necessary functional requirements of the system to be developed.

Stage 2: Design

The system design is made when the requirements are identified. This includes developing the system architecture, designing the user interface, and putting together comprehensive technical specification. In this phase, the proponents used the previously identified apexifications of the study based on chapter one (e.g., objectives of the study and scopes and limitations), as well as the devised flowchart and Entity Relationship Diagram (ERD). These specifications were the basis for the formulation of the system design that is high lightened in the storyboard and wireframes.

Stage 3: Implementation

Using the design guidelines as guide, the developer begins creating the system. Writing code, building databases, and developing the software components are all part of this step. In this phase, the developer has applied what previously devised from the design stage.

Stage 4: Testing

After the system is created, it goes through extensive testing to find any flaws or problems and repair them. The system testing, integration testing, and unit testing are all included in this.

Stage 5: Deployment

The system is deployed or put into use in the production environment following a successful test. This includes setting up hardware, installing software, and getting the system up and running.

Stage 6: Maintenance

Following system deployment, on-going support and maintenance are offered to address any problems, implements upgrades, and make enhancements as necessary.

Requirements Analysis

This is the requirement analysis for developing the Chronoschedio for STI College Alabang. The purpose of this part is to know what are the requirements for developing the Chronoschedio for STI College Alabang. In this step the researchers ran an interview with Mr. Ricson M. Ricardo the academic head of STI College Alabang.

The researchers asked about the present problem faced by the STI College Alabang in the current process in creating class schedule in excel.

The academic head state inefficiencies in the manual process of creating class schedules in excel, such as being time consuming, human errors, schedule conflicts, and handling sudden changes in the created schedules. These inefficiencies gave information to the researchers that is essential to develop a Web – Based Class Scheduling and Faculty Loading Management system that will address these issues.

For Software Development	
Operating System	Windows 11 Home Single Language 64-bit
Integrated Development Environment (IDE)	JavaScript AJAX Visual Studio Code
Database	MySQL is the database management system tool to be used as the back-end database of the system.
Programming Language	PHP, HTML, CSS (including Bootstrap)
Documentation	Microsoft

Recommended Hardware	
Processor	AMD Ryzen 5 5600H
Memory	24GB DDR
Disk Storage	500GB
Others	Standard keyboard USB Optical Mouse 15.6" Laptop

Recommended Peopleware	
Software Developers	<i>Job Responsibilities</i>
Programmer	The programmer is in charge of developing the system's functionalities, design, flow, and

	implementation. He or she should be far familiar with the IDE and programming languages such as PHP, HTML, CSS including Bootstrap. Furthermore, knowledge of MySQL is required to add, modify, archives, and change data within the database. Basic troubleshooting abilities are also required in the event that the system fails.
Designer	The designer, who is familiar with VS Code, HTML, and CSS, can assist the programmer in creating the system's design.
Users	<i>Functions</i>
Superuser	By controlling the Chronoschedio system, the superuser serves as the backbone. It guarantees that users are verified and authorized to use the system's features while also managing user accounts and data security.
Academic Head	The academic head play an important role in Chronoschedio system since the academic head is in charge of overall task in class scheduling and faculty loading. Academic head can contribute to the efficient of class

	scheduling. This allows the program heads, faculty members, registrars, and students to have accuracy, efficiency, and user experience of administrative and academic process.
Program Heads	The program head play also an important role in Chronoschedio system, since the program head can also create class schedule with their his/her designated departments.
Faculty Members	Faculty members can view their personal information details as well as the assigned class schedule. The faculty members are be able to comment their request of changing their personal details and concern about the class schedule, and feedbacks
Registrar	Registrar office can view the submitted class schedule by the academic head, with a standard template of STI College Alabang, and able to export class schedule.
Student	The student can view their updated class schedule with a user-friendly interface that makes it a great tool for improving their

	academic process.
--	-------------------

Requirements Documentation

This specific part defines the software features of web-system of Chronoschedio. The requirements that stated below are formulated based on the interview that was conducted with the Academic head of STI College of Alabang, as well as particularly selected information based on the objectives of this study. The defined requirements below are expected to cater to the needs of STI College Alabang that is to developed and are expected to be aligned with the required features that needed. The requirements below also describe how all the users of STI College Alabang can interact with the system.

4.1. Functional Requirements

4.1.1. Log In (Superuser)

REQ001. The superuser shall be able to log in by providing superuser credentials (e.g., email and password)

4.1.2. Maintain User (Superuser)

REQ002. The superuser shall be able to manage user accounts, adding, edit, archiving, and assigning roles.

REQ003. The superuser shall be able to reset password and batch upload multiple of users.

REQ004. The superuser shall ensure of maintaining the system security and integrity.

4.1.3. Activity Logs

REQ005. Thes superuser shall be able to monitor all users actions within the system such

as Data updates, and configuration adjustments.

4.1.4. Log In (Academic Head)

REQ006. The academic head shall be able to log in by providing Microsoft 365 account.

REQ007. The academic head shall be able to verify his identity with verification methods such as verification code, received text messages, or calls by Microsoft 365 account.

4.1.5. View Summary Dashboard (Academic Head)

REQ008. The academic head shall be able to view a brief overview of detailed room utilization report.

REQ009. The academic head shall be able to manage user accounts, edit, archiving, and assign class schedule.

4.1.6. View Schedules (Academic Head)

REQ010. The academic head shall be able to view all created class schedules.

REQ011. The academic head shall be able to view a brief overview of the whole timetable, class times, rooms.

REQ012. The academic head shall be able to have function to export class schedules in an excel format.

4.1.7. Overseeing Curriculum (Academic Head)

REQ013. The academic head shall be able control over curriculum-related tasks.

REQ014. The academic head shall be able to organized academic programs, manag curriculum details, and optimize course schedules.

4.1.8. Allocation of Class Schedule (Academic Head)

REQ015. The academic head shall be able to assign class schedules.

REQ016. The academic head shall be able to have function using drag and drop for class scheduling.

4.1.9. Flexibility and Control (Academic Head)

REQ017. The academic head shall be able to have tools to manage rooms, sections, and operational hours of class schedule.

REQ018. The academic head shall be able to add, archived rooms and sections.

REQ019. The academic head shall be able to set operational hours within the calendar schedule.

4.1.10. Activity Logs (Academic Head)

REQ020. The academic head shall be able to keep an eye on every activity within the system.

REQ021. The academic head shall be able to see when the schedule is been exported

REQ022. The academic head shall be able to see when the profile is edited, updated.

4.1.11. View and Profile (Academic Head)

REQ023. The academic head shall be able to view and update his profile.

4.1.12. Assign Workload (Academic Head)

REQ025. The academic head shall be able to assign faculty members to specific program codes and curriculum codes.

REQ026. The academic head shall be able to manage workload efficiently and organized.

4.1.13. Email Notifications (Academic Head)

REQ027. The academic head shall be able to receive an email notification (Outlook) with user manual.

4.1.14. Log In (Program Heads)

REQ028. The Program head shall be able to log in by providing Microsoft 365 account.

REQ029. The program shall be able to verify his/her identity with verification methods such as verification code, received text messages, or calls by Microsoft 365 account.

4.1.15. Allocation of Class Schedule (Program Head)

REQ030. The program head shall be able to assign class schedule that correspond to their assigned departments.

REQ031. The program head shall be able to have function of drag and drop for class scheduling.

4.1.16. View Profile (Program Head)

REQ032. The program head shall be able to view and update their contact number.

REQ033. The program head shall be able to view and update their personal contact number.

4.1.17. Email Notifications (Program Head)

REQ034. The program head shall be able to receive an email notification (Outlook) and user manual.

4.1.18. Log In (School Registrar)

REQ035. The school registrar shall be able to log in by providing Microsoft 365 account.

REQ036. The academic shall be able to verify his/her identity with verification methods such as verification code, received text messages, or calls by Microsoft 365 account.

4.1.19. View Class Schedule (School Registrar)

REQ037. The school registrar shall be able to view all created class schedules.

4.1.20. View Summary (School Registrar)

REQ038. The school registrar shall be able to have overview of the whole timetable.

REQ039. The school registrar shall be able to have export function.

4.1.21. Log In (Faculty)

REQ040. The faculty members shall be able to log in by providing Microsoft 365 account.

REQ041. The faculty members be able to verify his/her identity with verification methods such as verification code, received text messages, or calls by Microsoft 365 account.

4.1.22. View Class Schedule (Faculty)

REQ042. The faculty members shall be able to view all created schedule their assigned.

REQ043. The faculty members shall be able to export their respective class schedules.

4.1.23. View Profile (Faculty)

REQ044. The faculty members shall be able to view their personal details.

REQ045. The faculty members shall be able to view and update their contact number.

4.1.24. Email Notifications (Academic Head)

REQ046. The faculty member shall be able to receive an email notification (Outlook) and user manual.

4.1.25. Export PDF

REQ047. The faculty member shall be able to received exported schedule in excel and PDF file.

4.1.26 Log In (Student)

REQ048. The student shall be able to log in by providing Microsoft 365 account

REQ049. The student shall be able to verify his/her identity with verification methods such as verification code, received text messages, or calls by Microsoft 365 account.

4.1.27 View Class Schedule (Student)

REQ050. The student shall be able to view their assigned class schedule.

4.2. Non-Functional Requirements

4.2.1 Performance

REQ001. The system should process and display class schedules or changes real-time.

REQ002. The system should handle class scheduling and faculty loading operations for students and faculty members simultaneously without performance degradation.

REQ003. The system should support users actively making changes or viewing schedules without experiencing delays.

4.2.2 Availability

REQ004. The system should be accessible 24/7 except for planned maintenance in Microsoft 365 office.

4.2.3 Security

REQ005. The system must support multi-factor authentication (MFA) for academic head, program heads, faculty members, registrar, and students to prevent unauthorized access.

REQ006. The system must support server hosting in a secure facility with access control measures.

4.2.4 Usability

REQ007. The system must feature an intuitive and visually appealing interface with clear navigation for all user types (Academic head, Program heads, Faculty, Registrar, and Student)

REQ008. The system should respond to user inputs for standard configurations such as viewing schedules or generating schedules.

REQ009. The system should maintain consistent design elements, such as buttons, menus, and labels, across all modules.

REQ010. The system should be able to learn and operate effectively.

REQ011. The system provides wide search features that allows users to quickly locate schedules, faculty, or classrooms by keywords or filters.

4.2.5 Maintainability

REQ012. The system should maintain compatibility with other browsers (e.g., Chrome, Firefox, Edge, Safari) and be adaptable to future browser updates.

4.2.6 Portability

REQ013. The system must function consistently across major web browsers, including

Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari, without requiring additional configuration.

REQ014. The system should be accessible on devices running common operating systems such as Windows, macOS, Linux, iOS, and Android without compatibility issues.

REQ015. The system should be deployable on major cloud platform in Microsoft Azure, allowing flexibility in hosting environments.

REQ016. The system should have the ability to export schedules.

Design of Software, System, Product, and/or Processes

In designing our software, system, and processes, we utilized visual tools for a broader and user -centric approach. Entity – Relationship Diagram (ERD) modeled our database structure, illustrating entity relationship and contributing to coherent data architecture.

Use-Case Diagram captured user-software interactions, representing goals, system goals, and system functionality. Flowcharts mapped sequential processes, aiding task analysis and identification. Integrating these visuals ensured a comprehensive understanding of our software’s architecture, system behavior, and process flow, resulting in an efficient and user-friendly product.

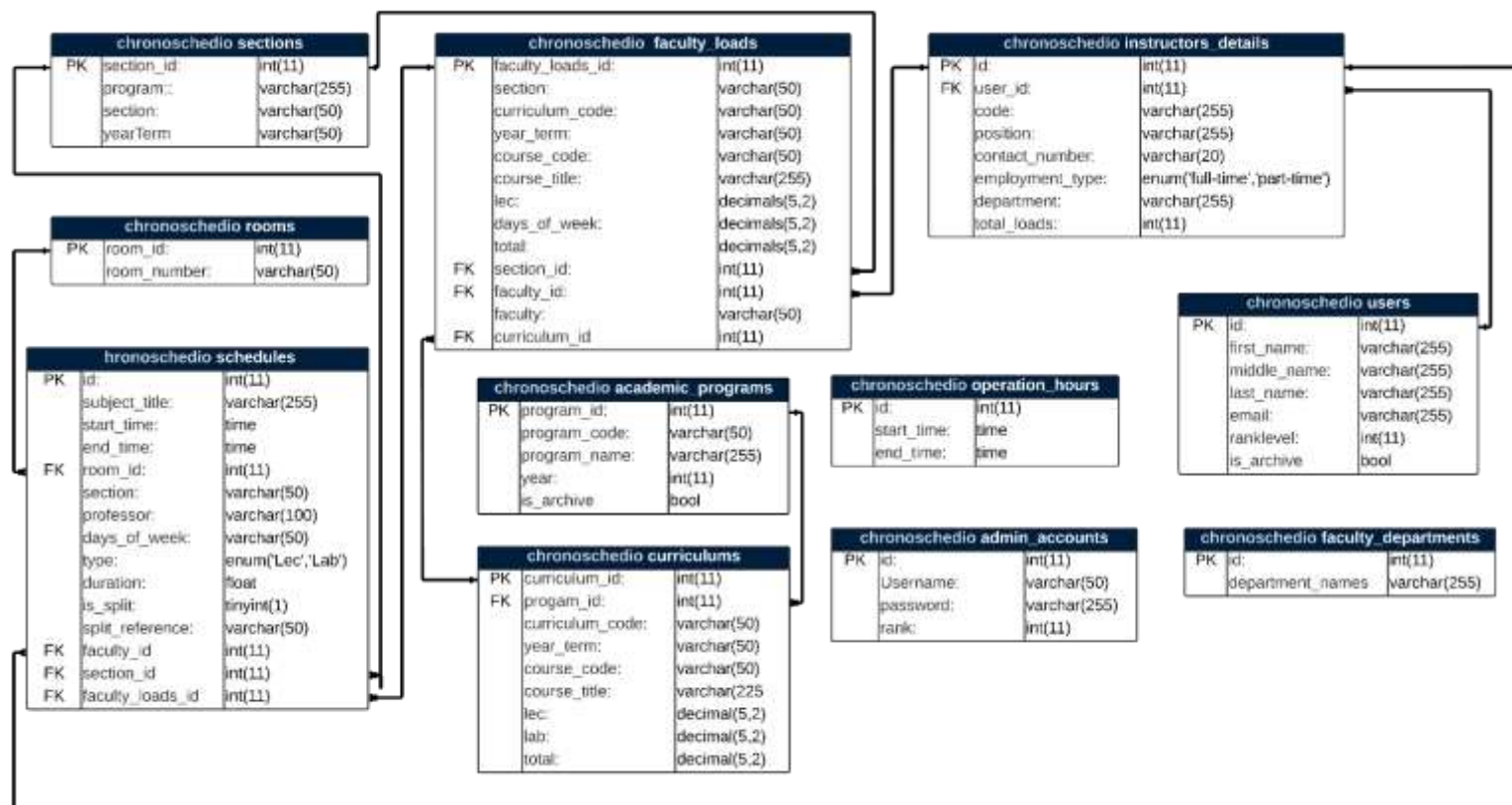


Figure 1 Entity Relationship Diagram

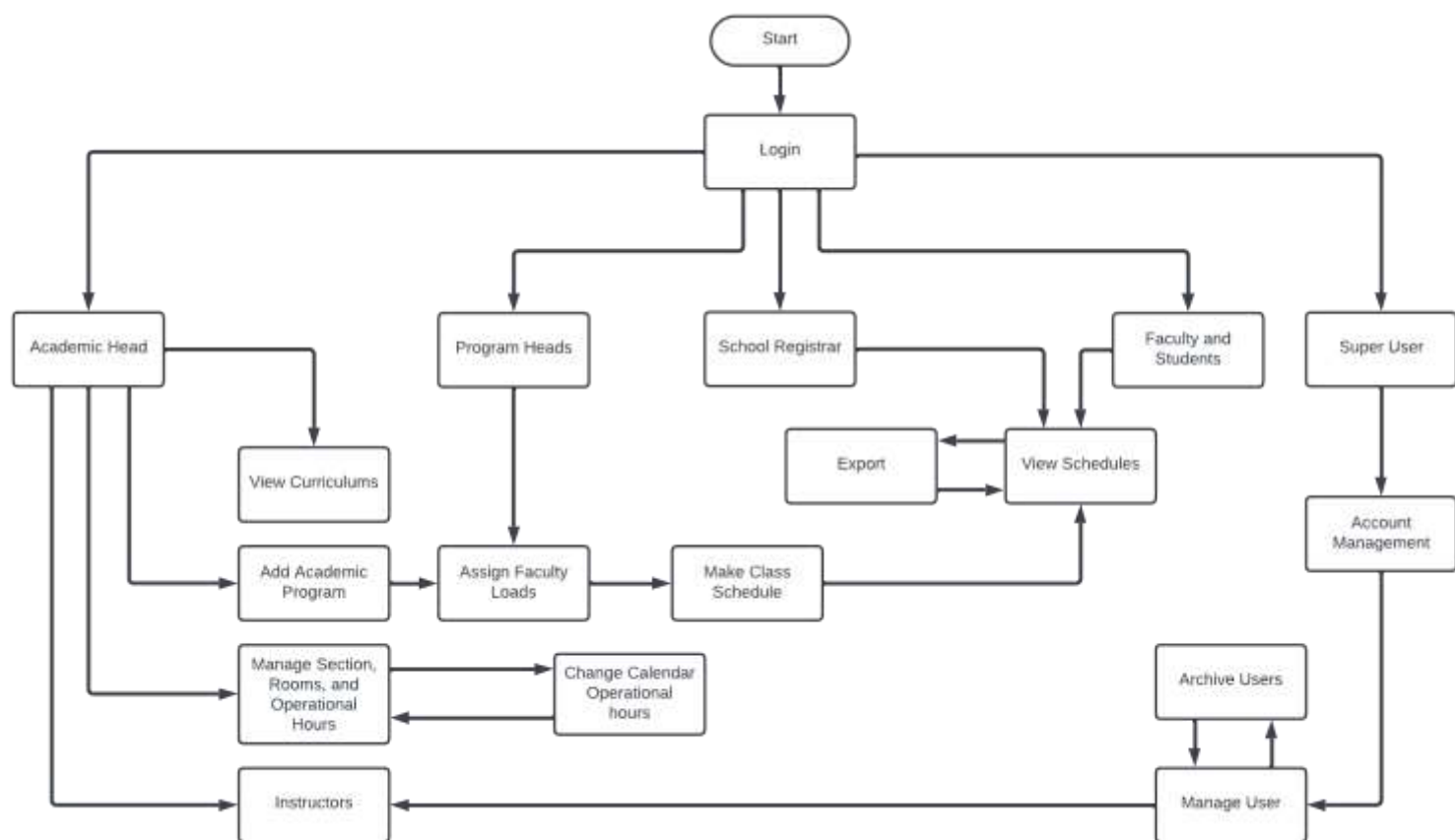


Figure 2 Data Flow Diagram

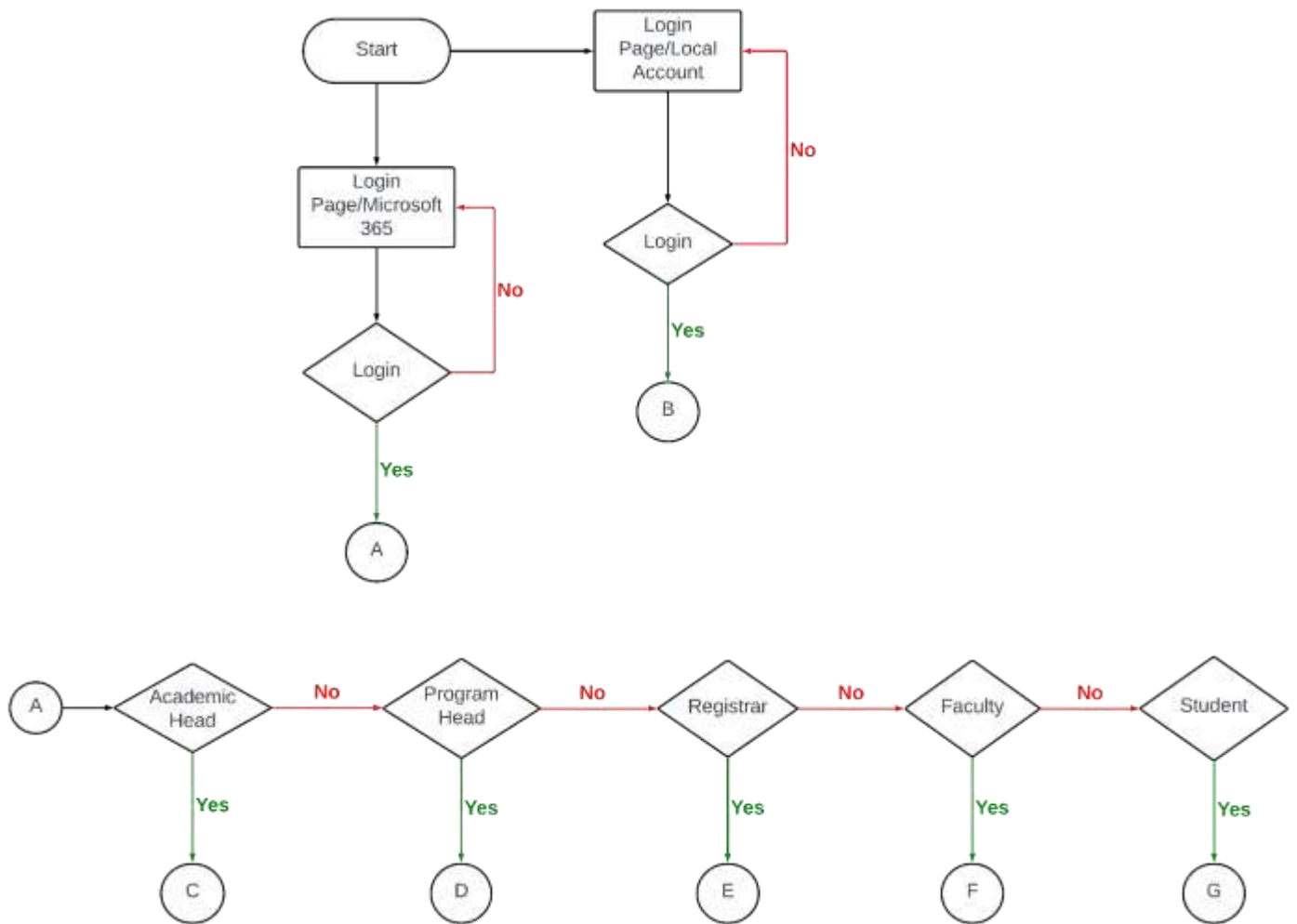


Figure 3 Login Flowchart

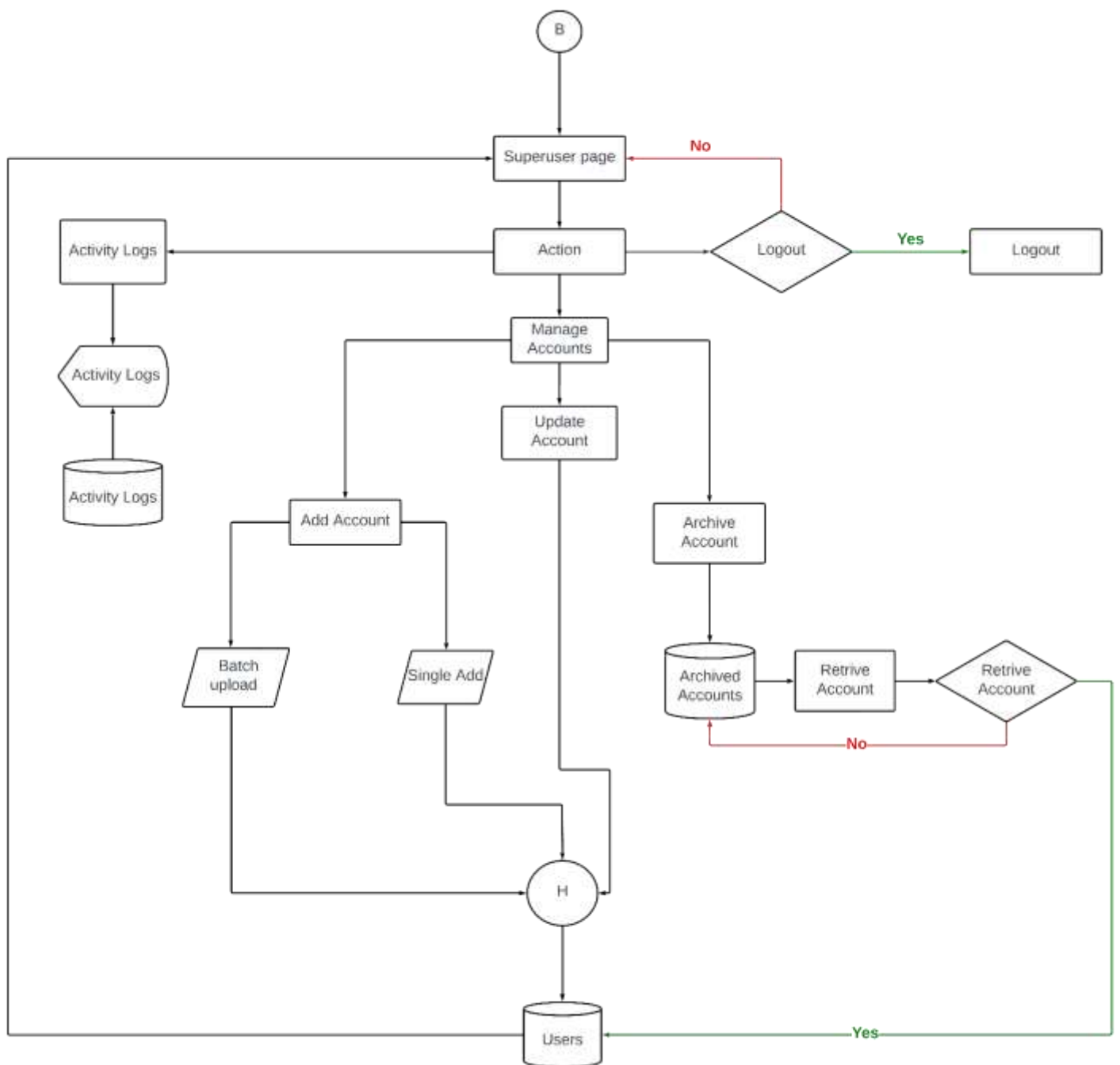


Figure 4 Superuser Flowchart

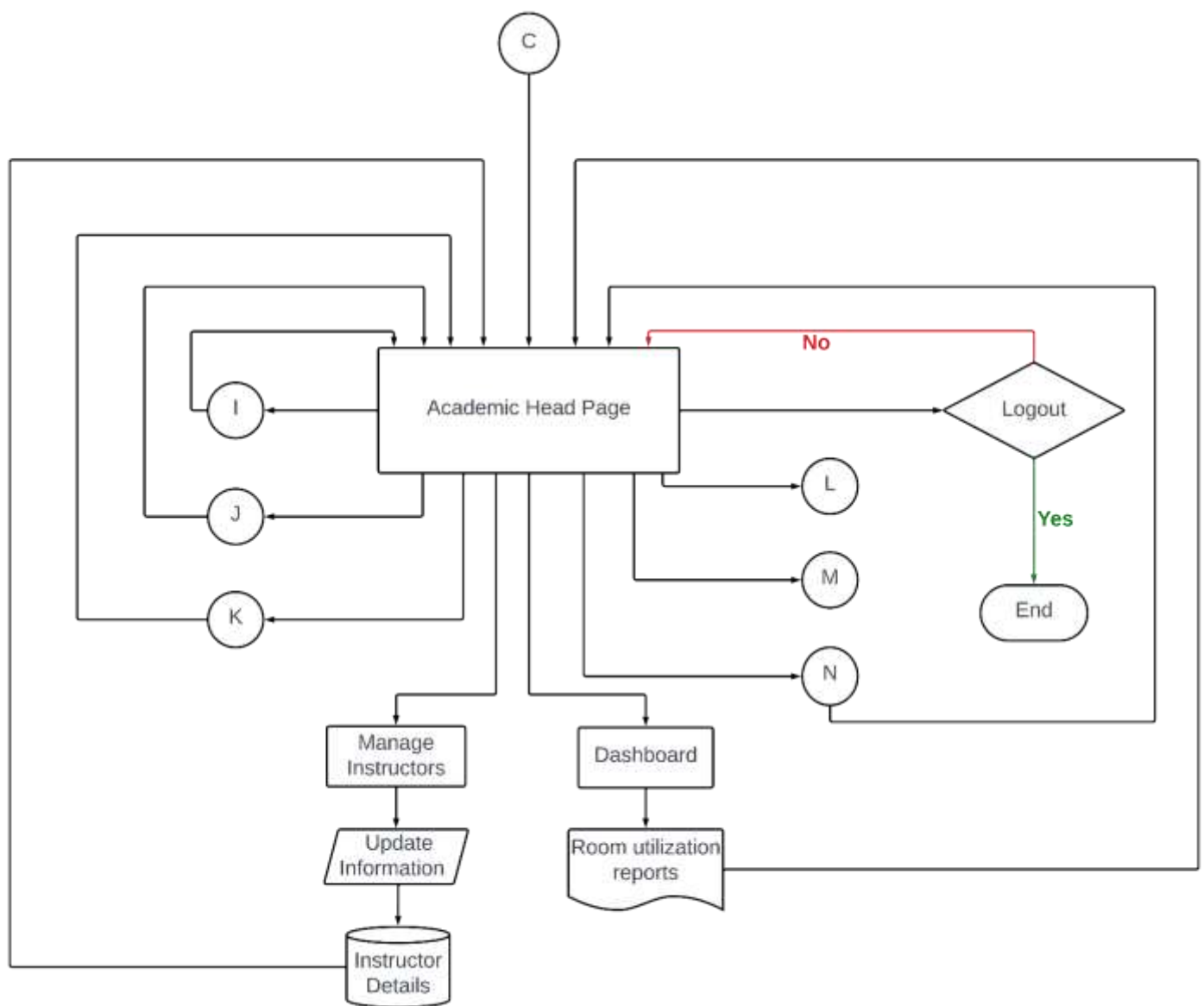


Figure 5 Academic head Flowchart

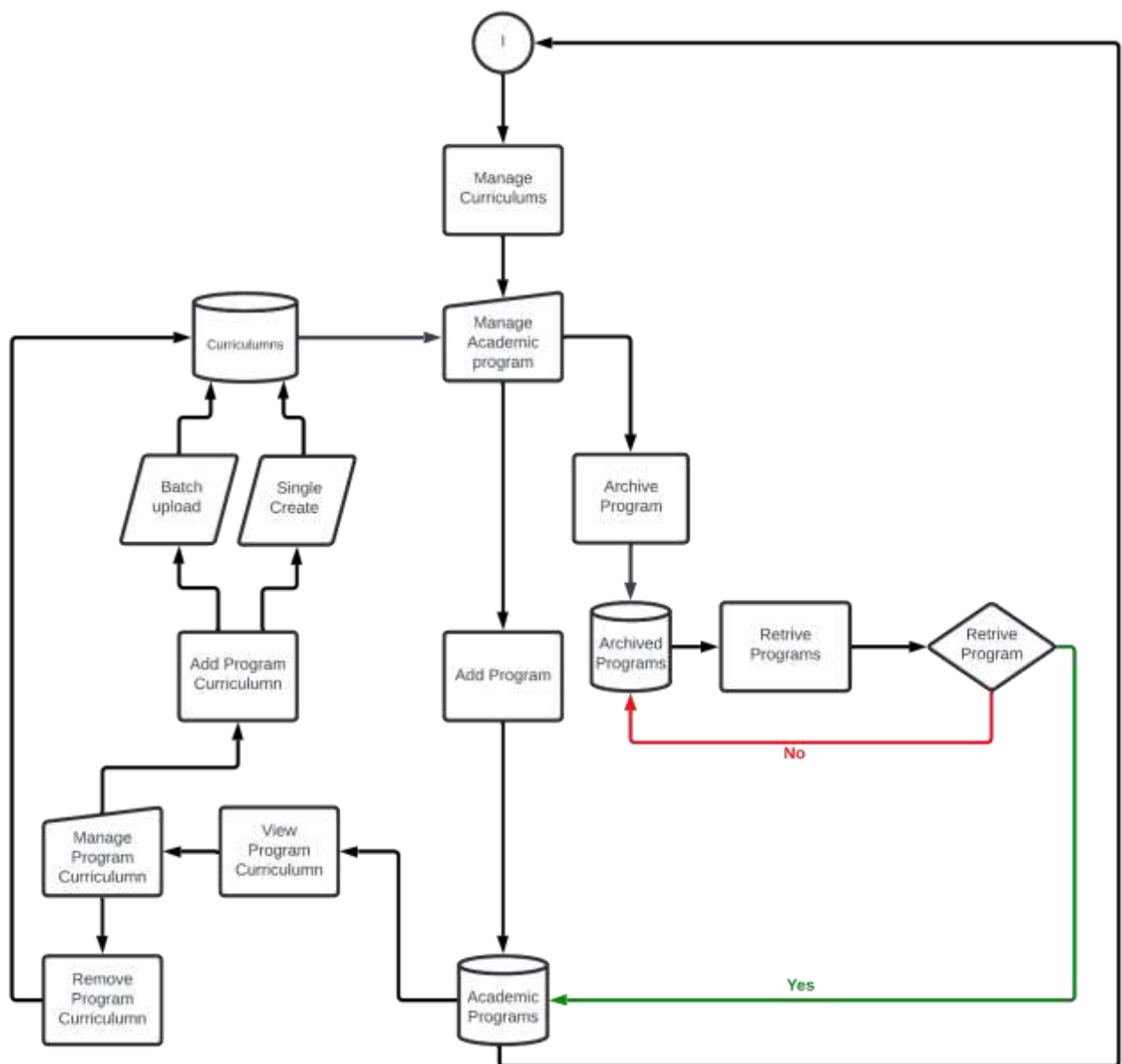


Figure 5.1 Academic Head Flowchart

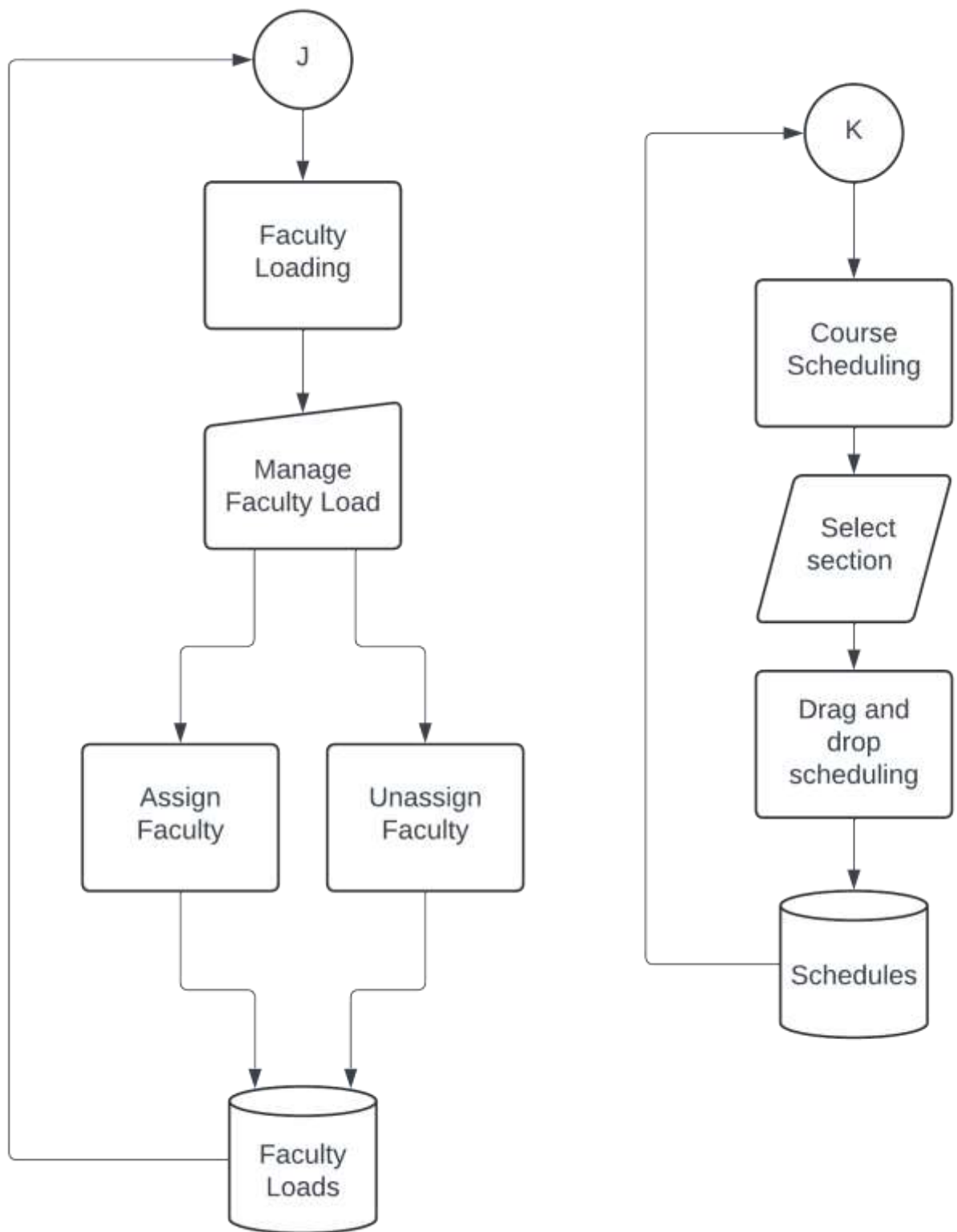


Figure 5.2 Academic Head Flowchart

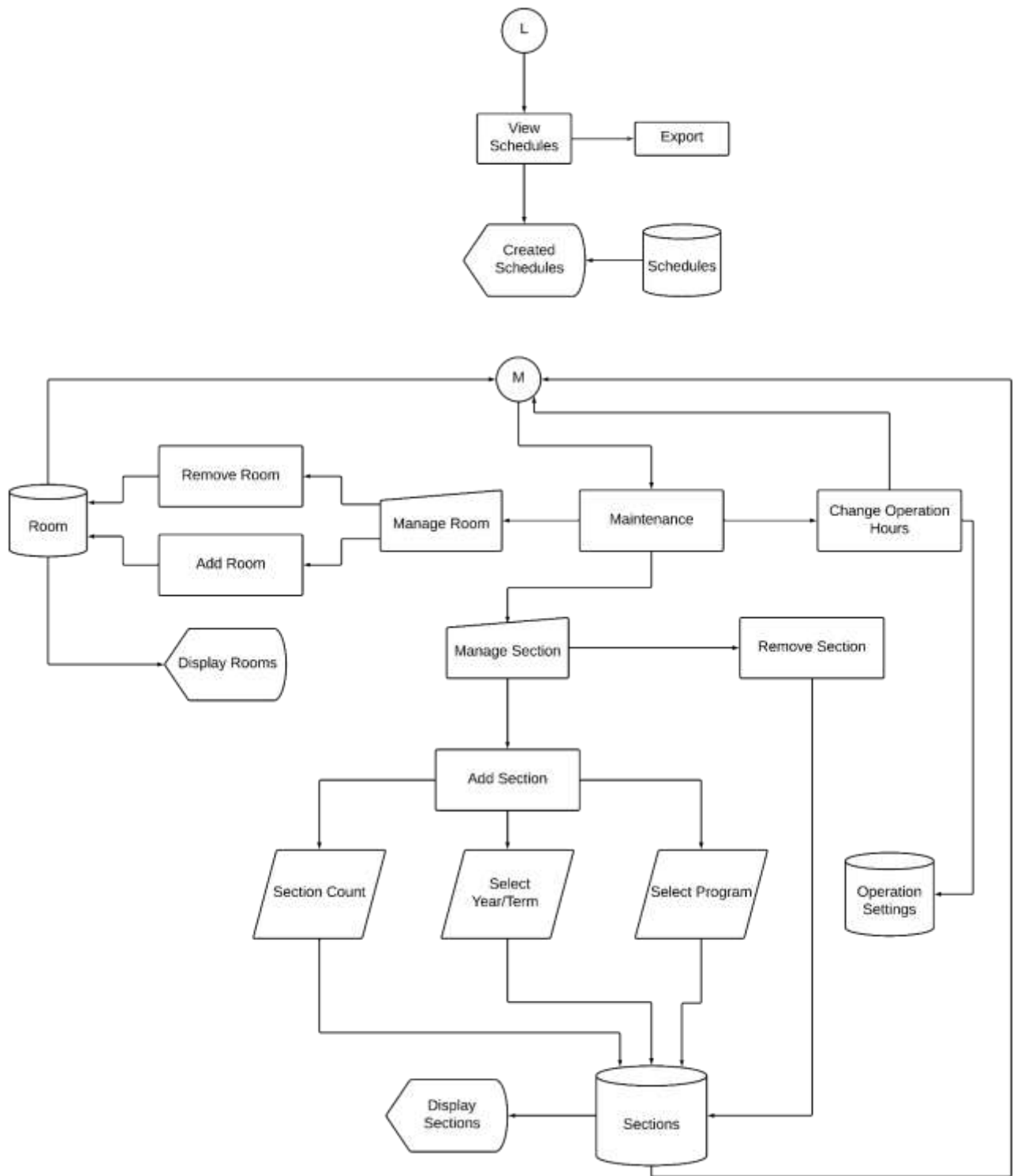


Figure 5.3 Academic Head Flowchart

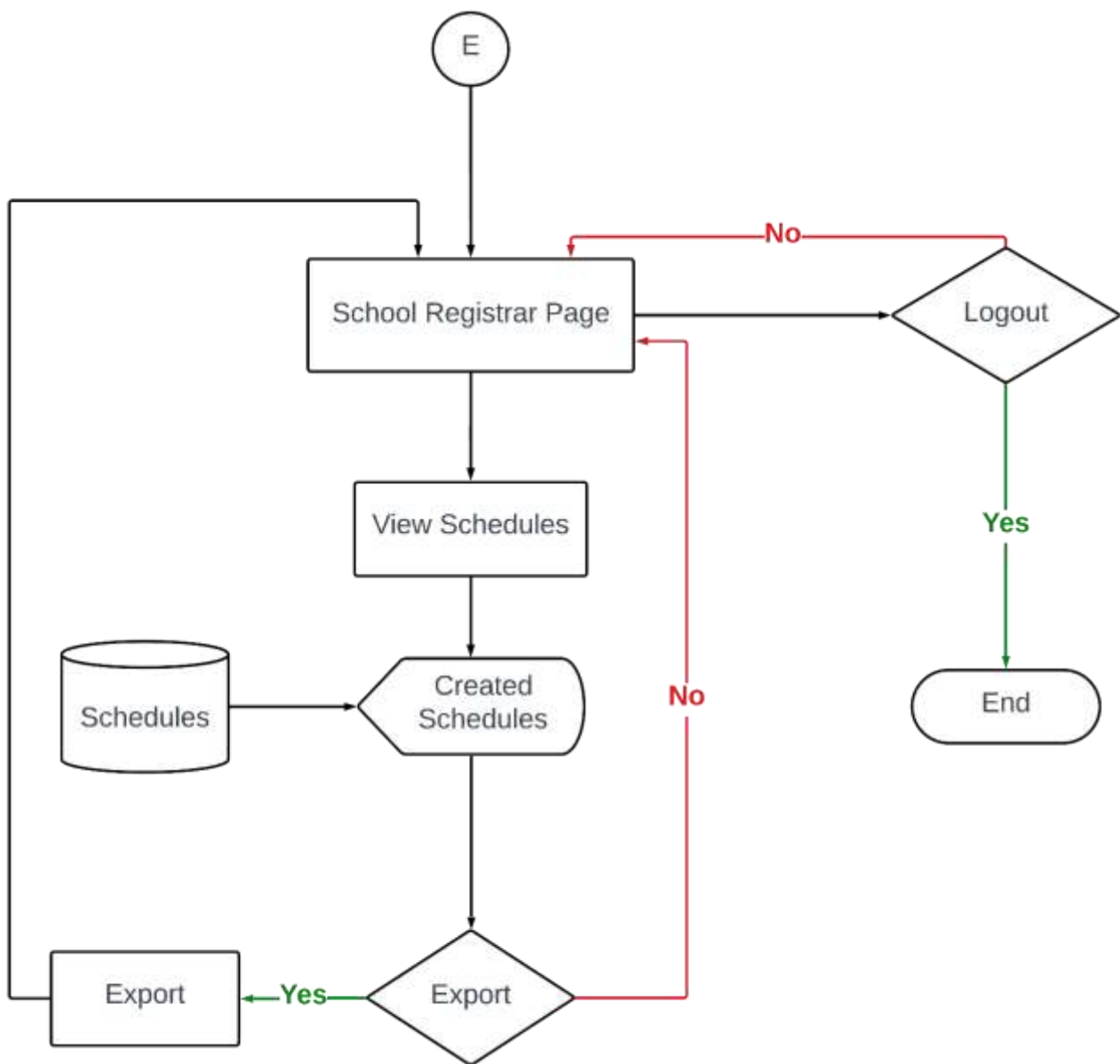


Figure 7 School Registrar Flowchart

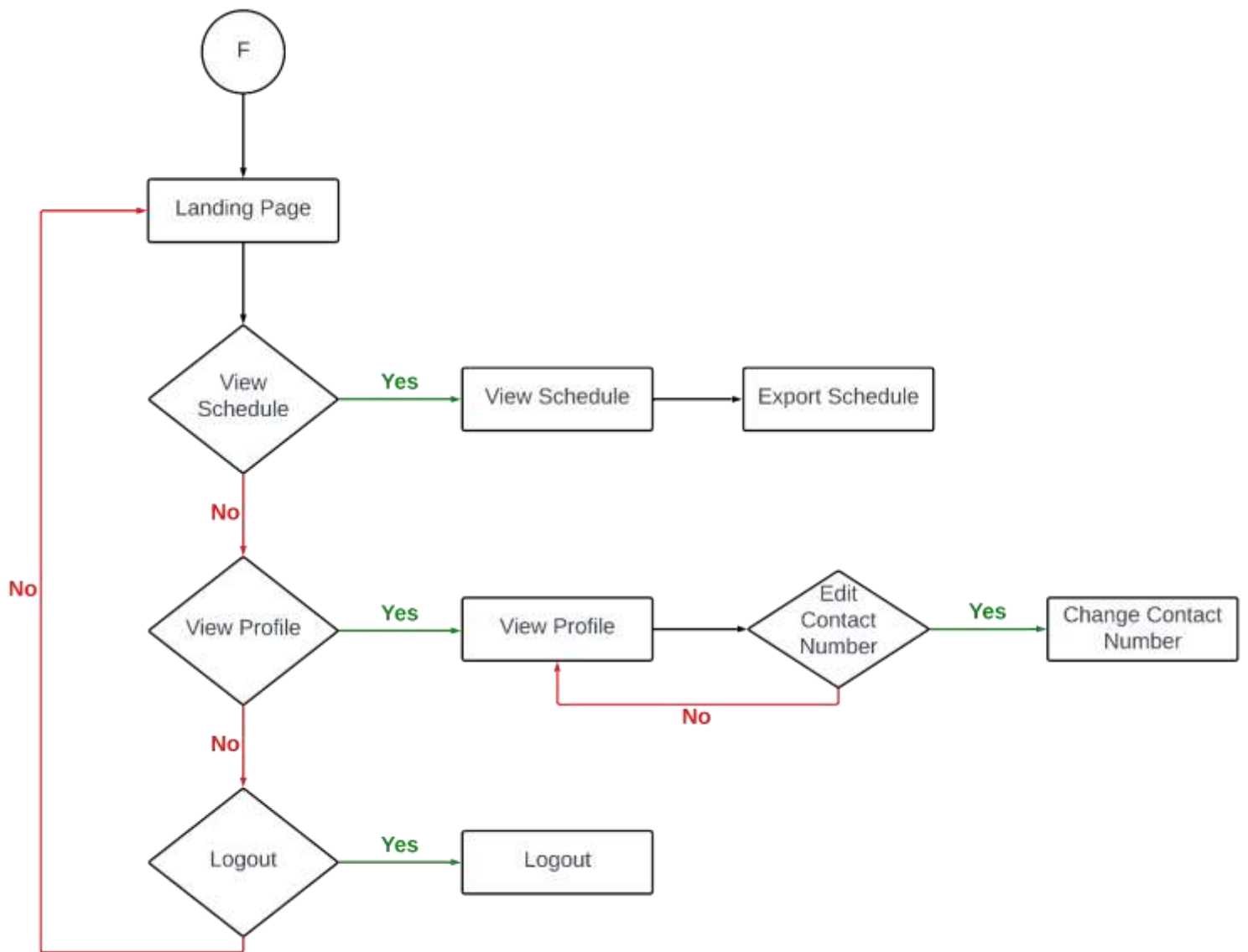


Figure 8 Faculty Flowchart

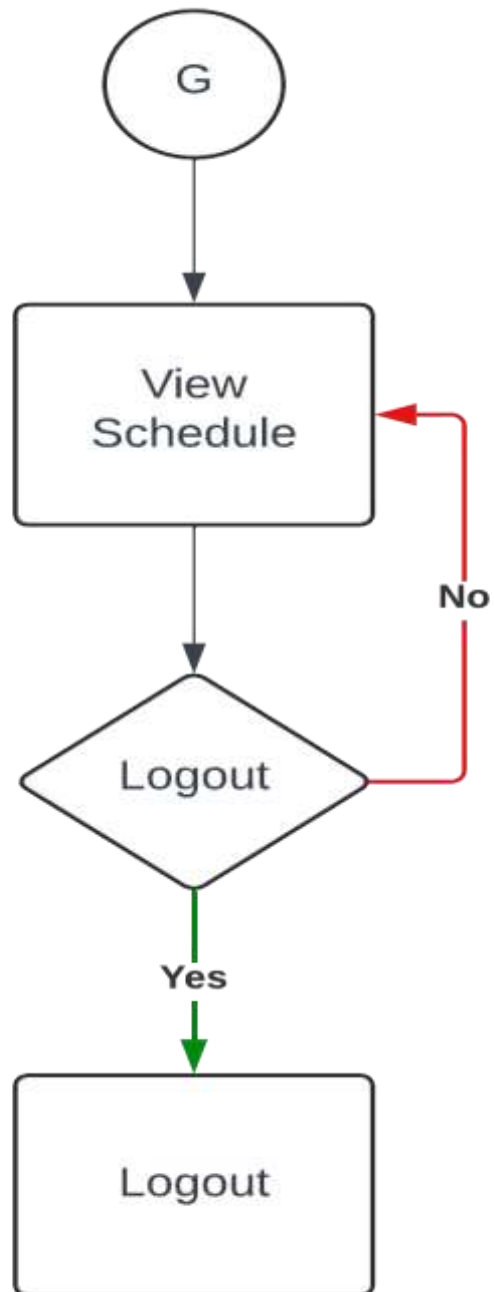


Figure 9 Student Flowchart

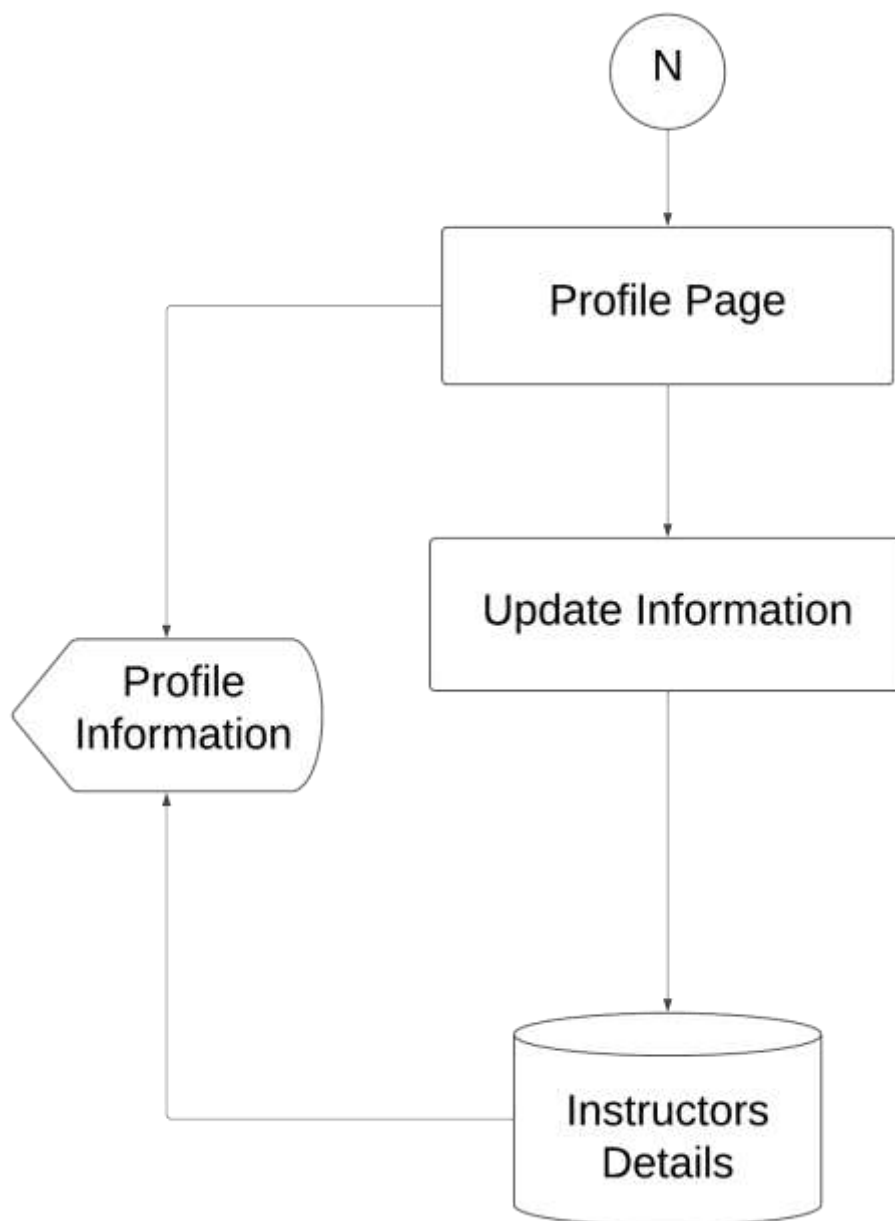


Figure 10 Profile Module Flowchart

Development and Testing

Development and Testing Process

The researchers develop the capstone project by using Visual Studio Code (VS Code) it is emerged as the preferred Integrated Development Environment (IDE) due to its features and flexibility; its customization options integrated tools make it an ideal choice for development system.

Web Development with VS Code

During the web development, VS Code facilitated an efficient workflow for coding and debugging. Its built – in extensions and vibrant plugin ecosystem, significantly contributed to the development of the system.

Capstone Adviser's Guidance

The involvement of the researchers' capstone adviser was instrumental in improving the development and testing process. Regular meetings and consultations allowed the researchers to receive clear feedback and identify areas that requires improvement and modification. This collaborative approached ensured that our project met the highest standards and aligned with the project objectives.

Preliminary Design

For web development, Visual Studio Code is used in the initial design. While the backend uses PHP with AJAX for dynamic updates, the frontend combines HTML5, CSS3, and JavaScript.

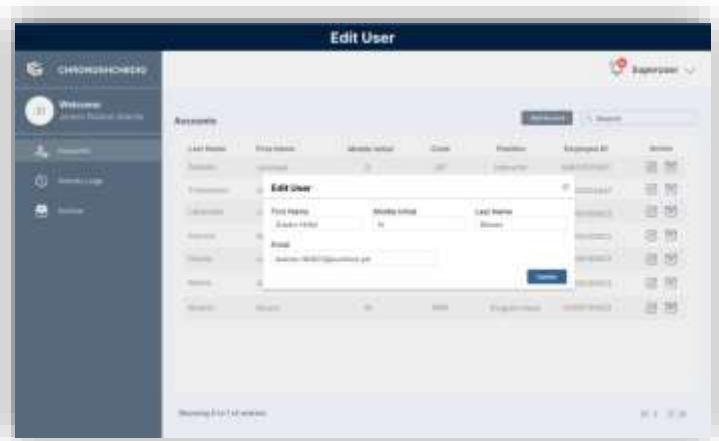
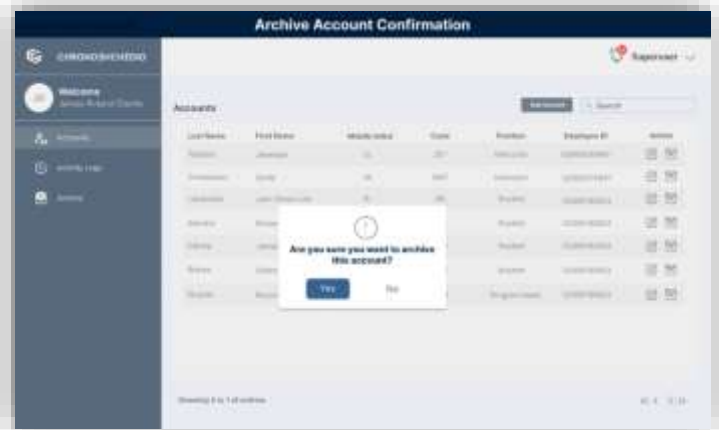
LOG IN (USER SIDE)

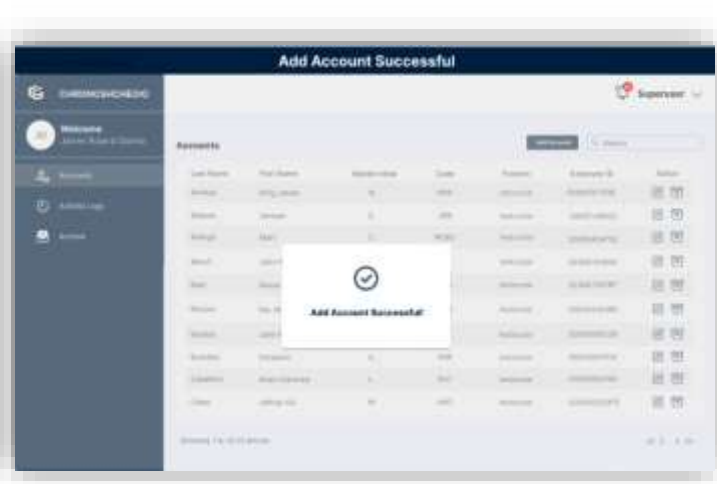
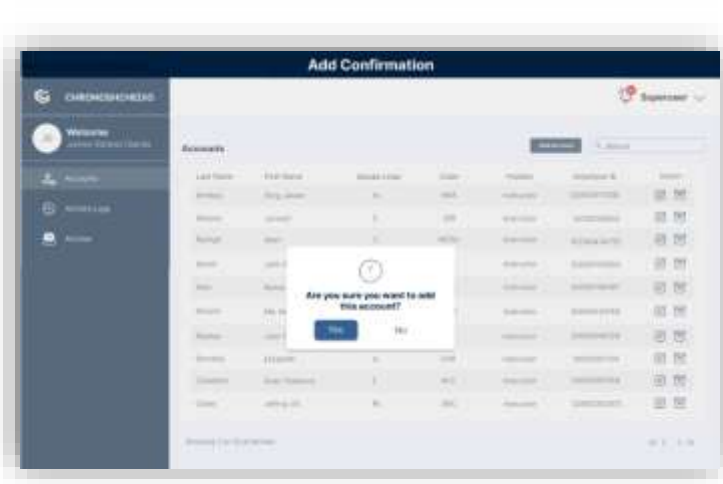
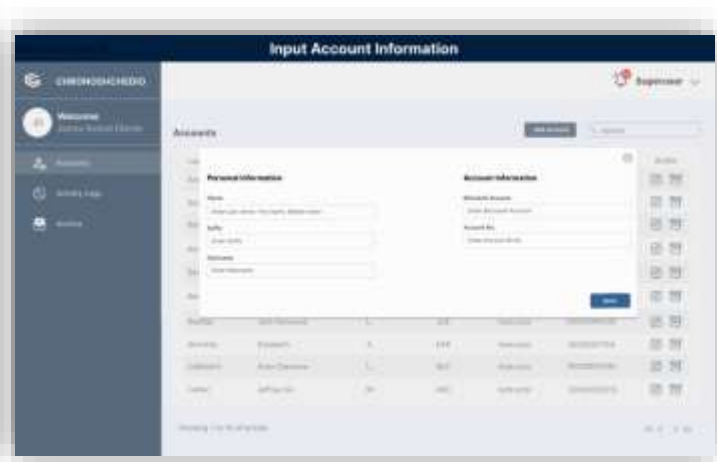
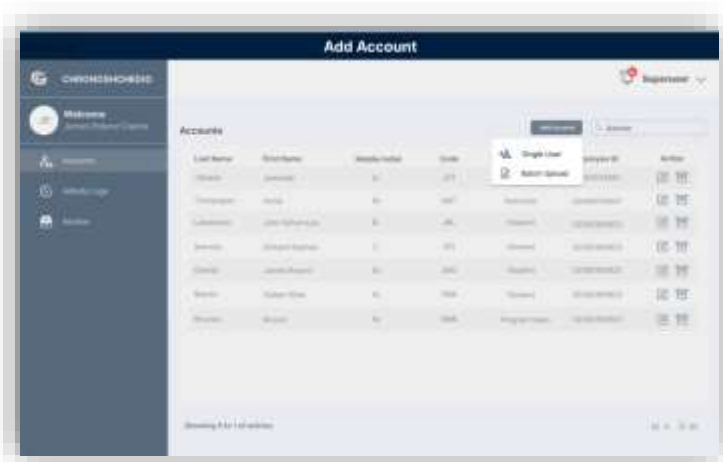
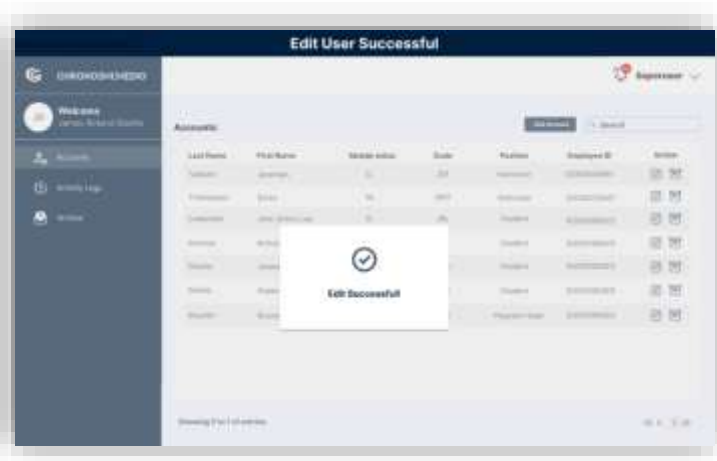
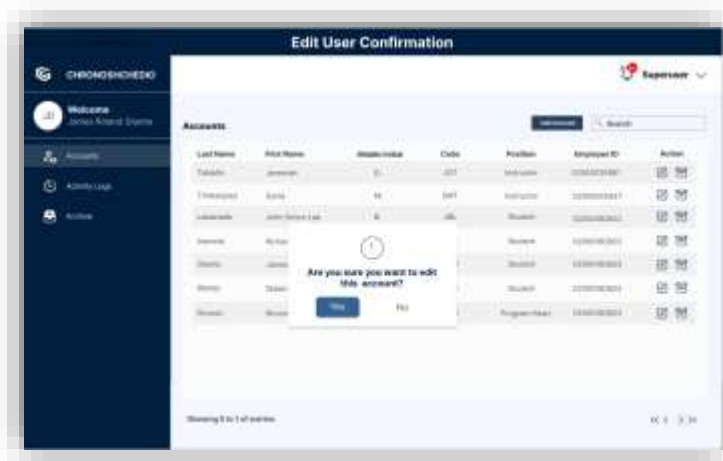


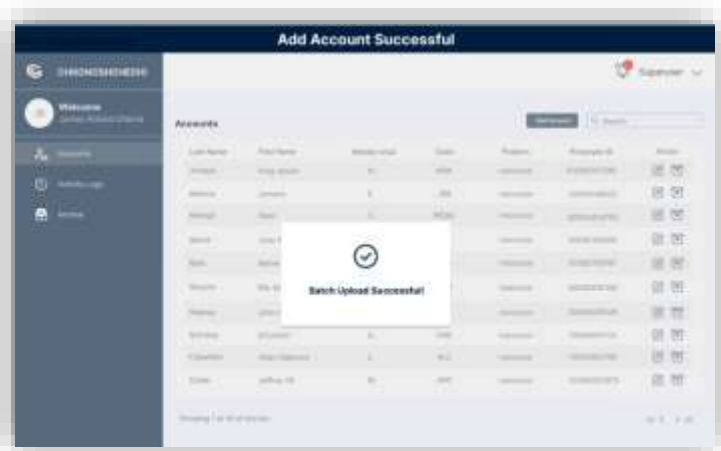
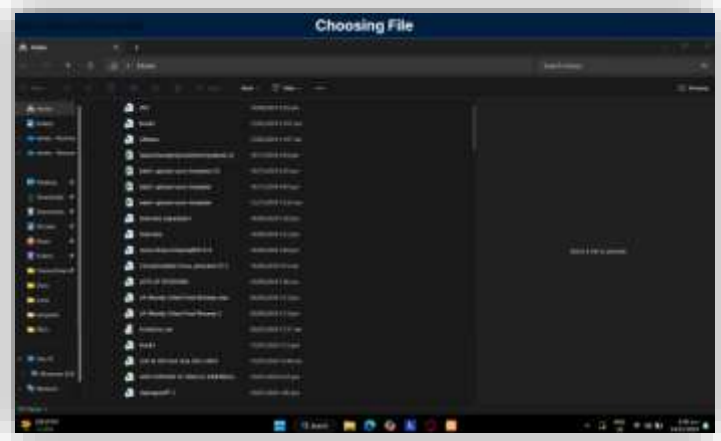
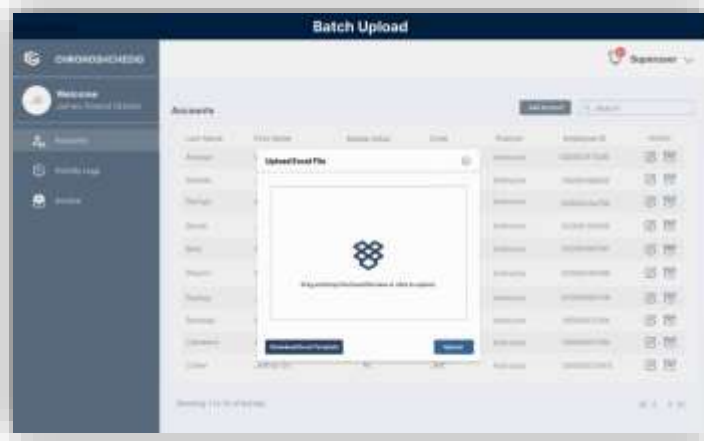
LOG IN (SUPERUSER SIDE)

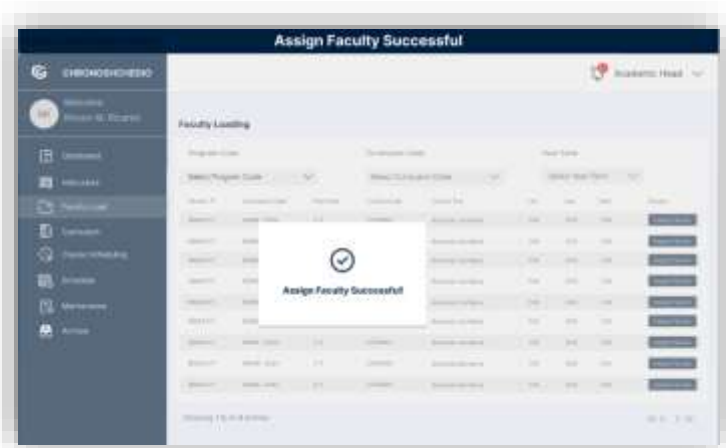
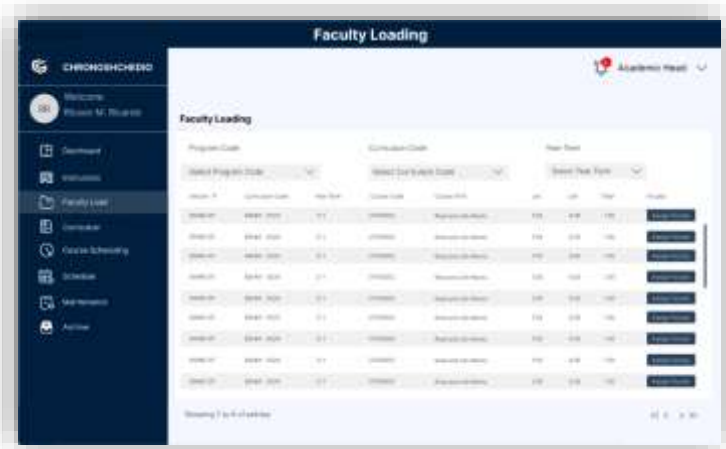
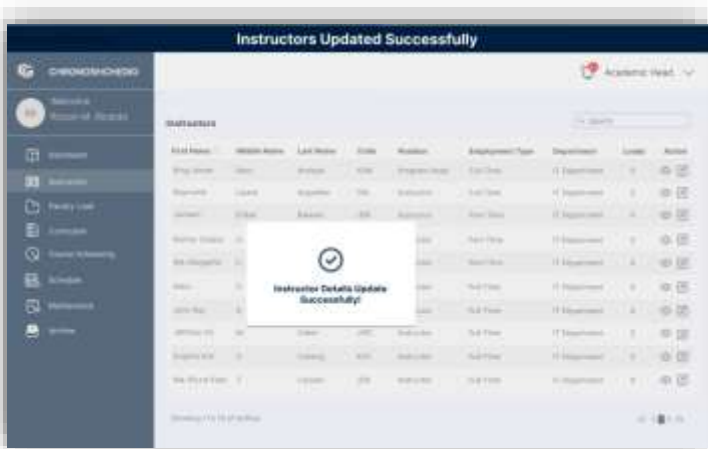


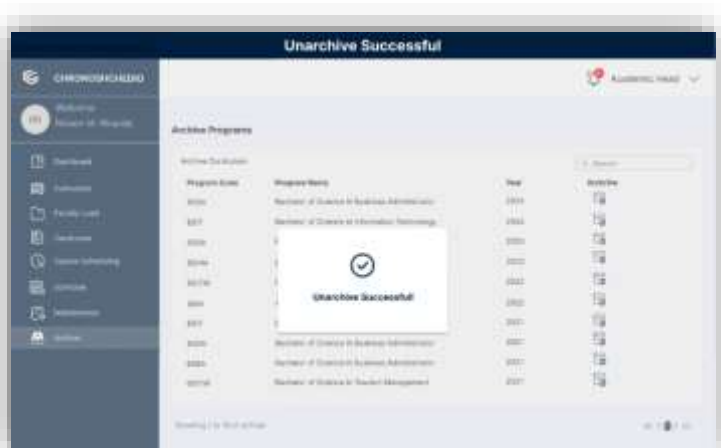
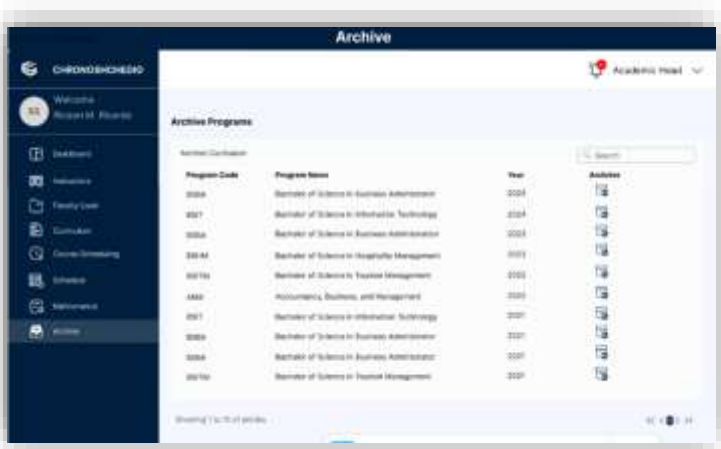
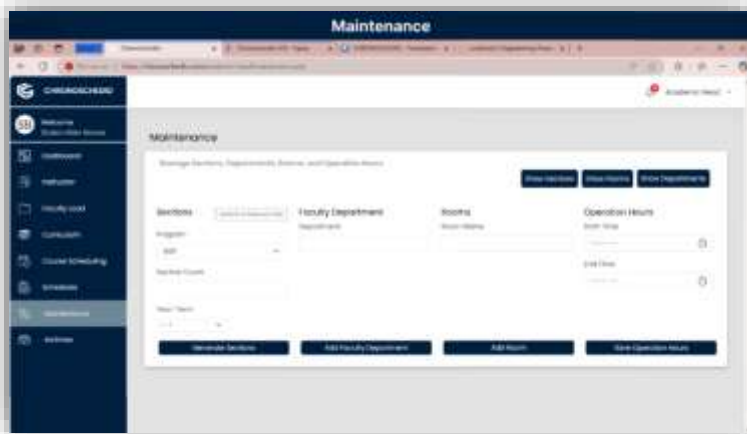
SUPERUSER SIDE











ACADEMIC HEAD (SIDE BAR)



Instructors

Table listing instructors with columns: Instructor Name, Program Code, Status, and Action. The table shows a list of instructors with their respective program codes and status.

Instructor Name	Program Code	Status	Action
Jeffrey M. La Rosa	001	Open	View
Jeffrey M. La Rosa	002	Open	View
Jeffrey M. La Rosa	003	Open	View
Jeffrey M. La Rosa	004	Open	View
Jeffrey M. La Rosa	005	Open	View
Jeffrey M. La Rosa	006	Open	View
Jeffrey M. La Rosa	007	Open	View
Jeffrey M. La Rosa	008	Open	View
Jeffrey M. La Rosa	009	Open	View
Jeffrey M. La Rosa	010	Open	View

Edit Instructors

Form for editing instructor details. The form includes fields for:

- First Name
- Last Name
- Program Code
- Status
- Room
- Classroom
- Room Code
- Room Name
- Room Type
- Room Size
- Room Color
- Room Image
- Room Description
- Room Notes

Buttons: Save, Cancel, Add New

Faculty Loading

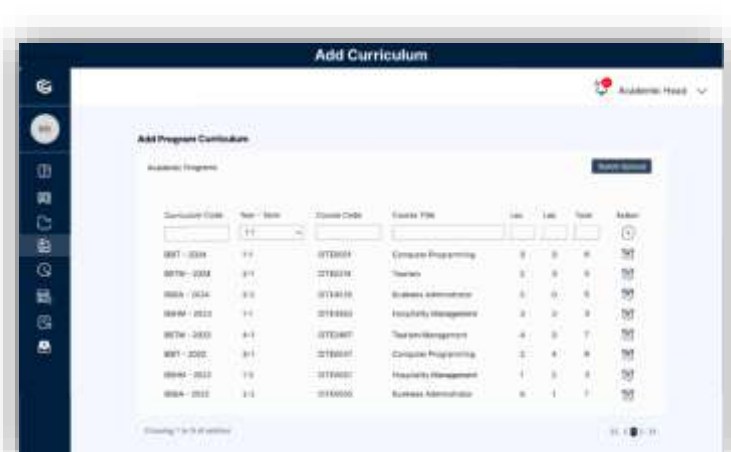
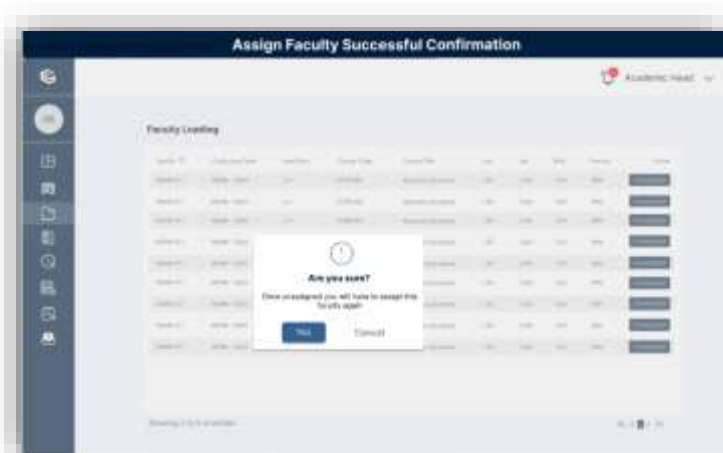
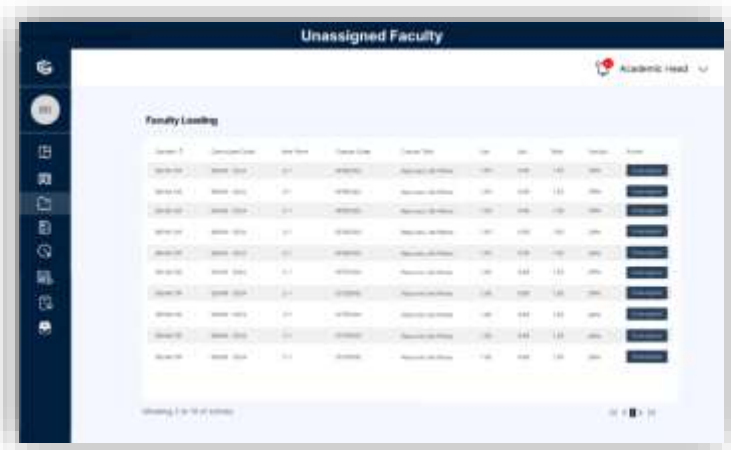
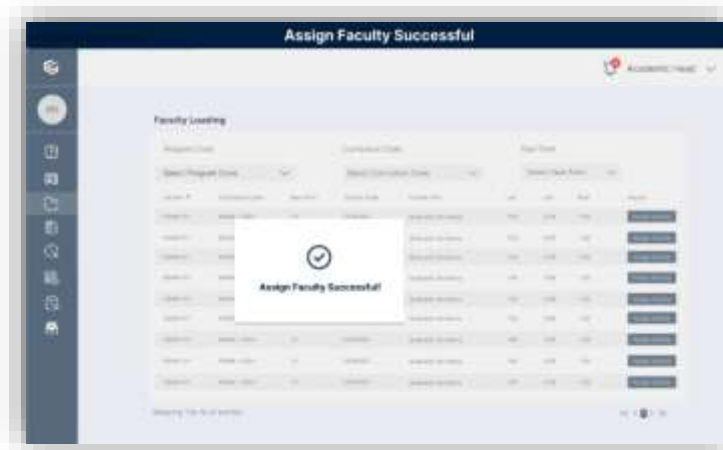
Table showing faculty loading details. The table has columns for Program Code, Course Code, Course Title, and Faculty. The table lists various programs and courses with their respective faculty members.

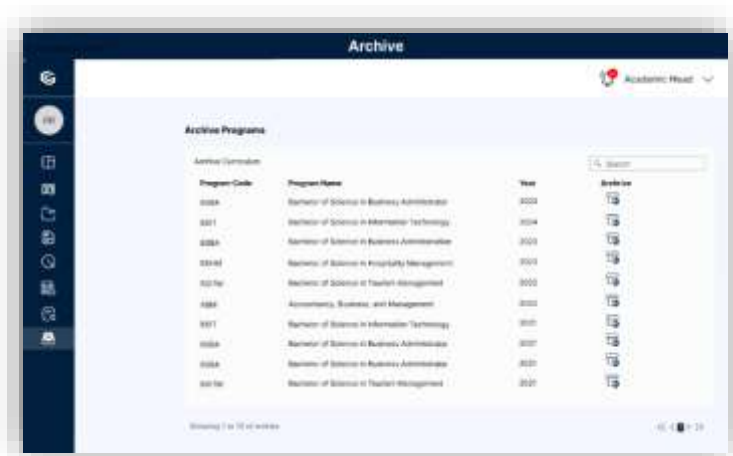
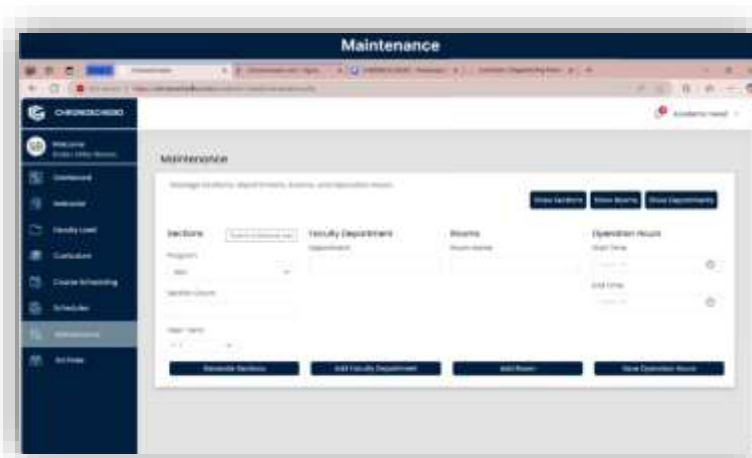
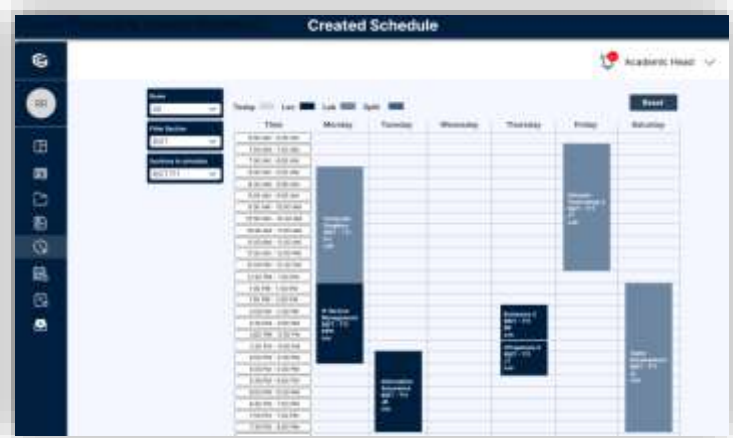
Program Code	Course Code	Course Title	Faculty
001	001	001	001
002	002	002	002
003	003	003	003
004	004	004	004
005	005	005	005
006	006	006	006
007	007	007	007
008	008	008	008
009	009	009	009
010	010	010	010

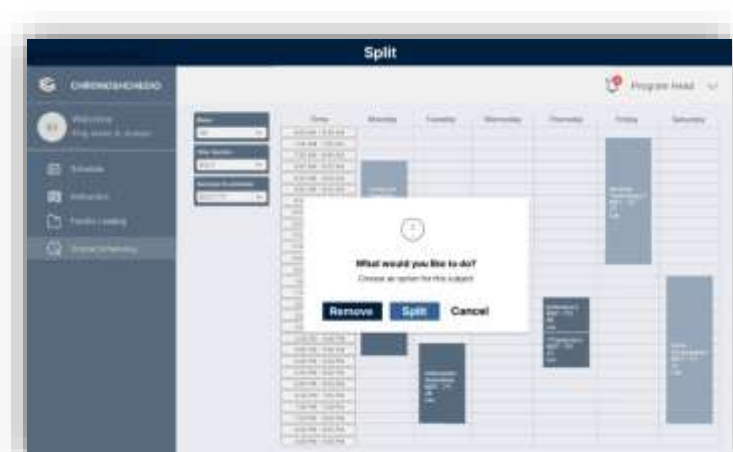
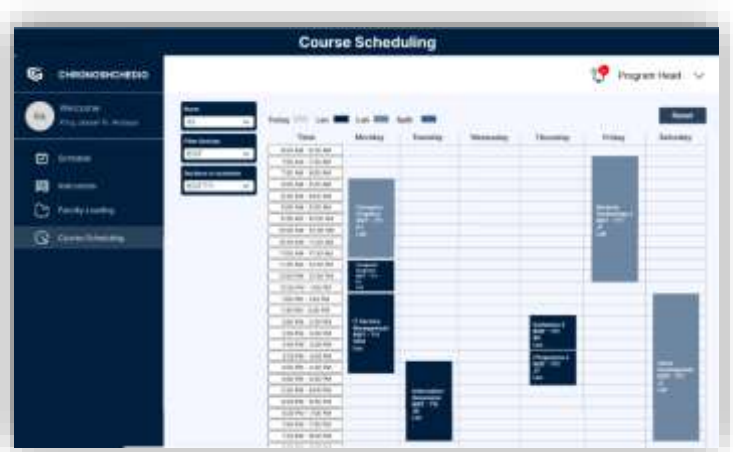
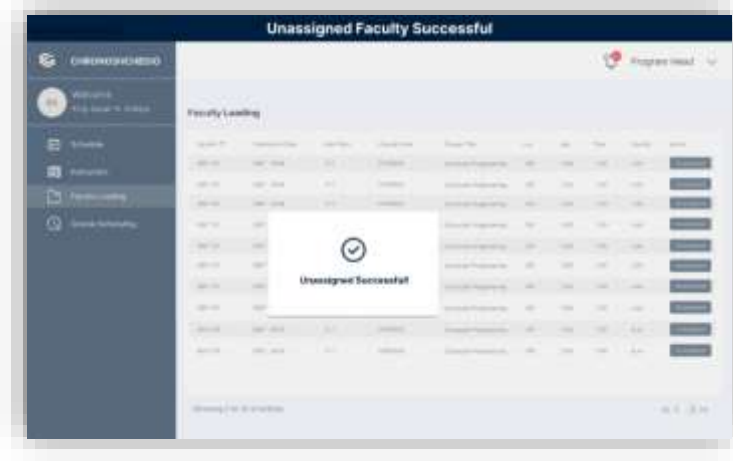
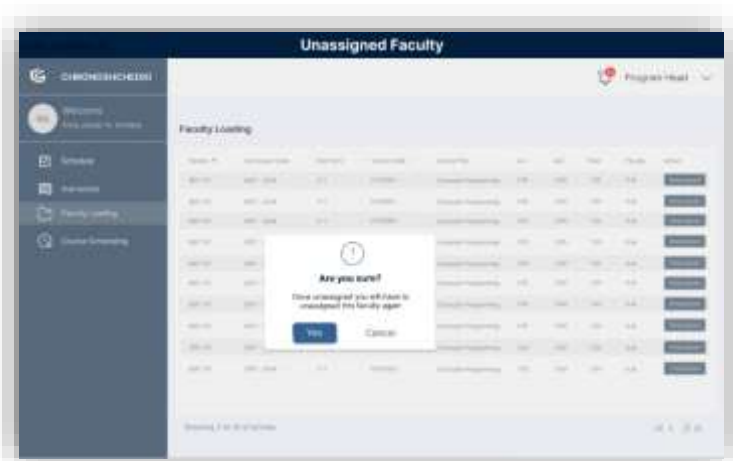
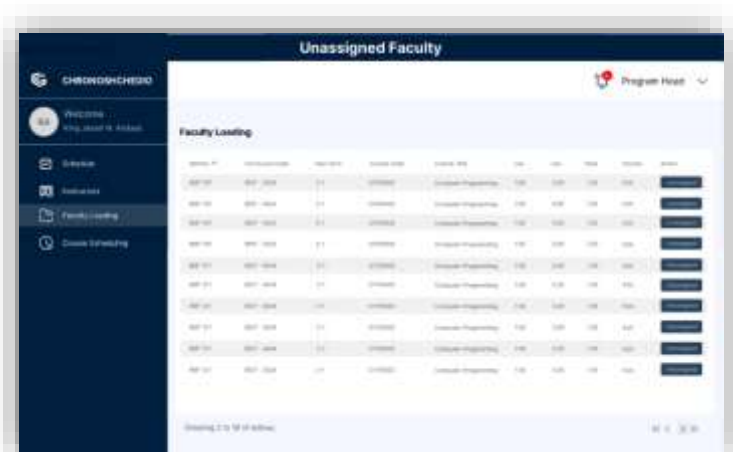
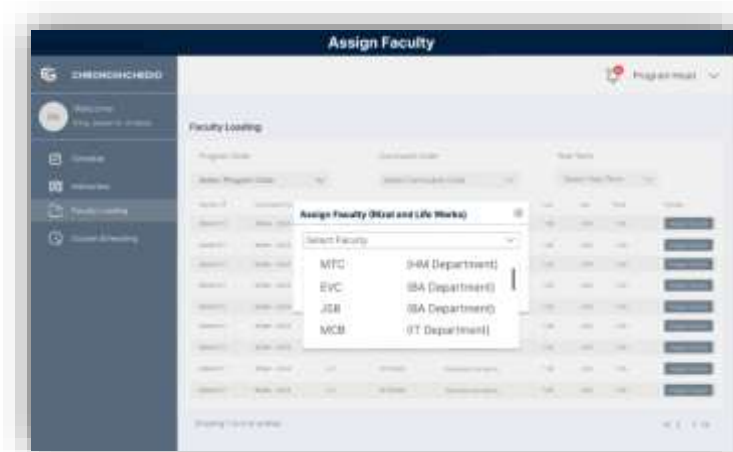
Assign Faculty

Form for assigning faculty to a course. The form includes a dropdown menu for selecting a faculty member. The form also includes a table for assigning faculty to a course.

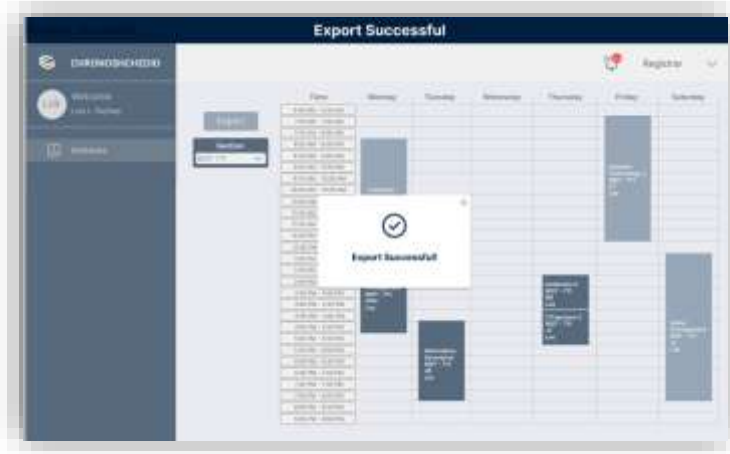
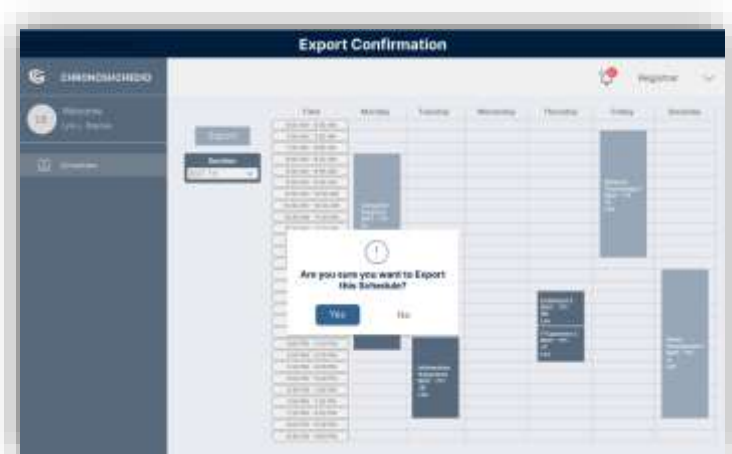
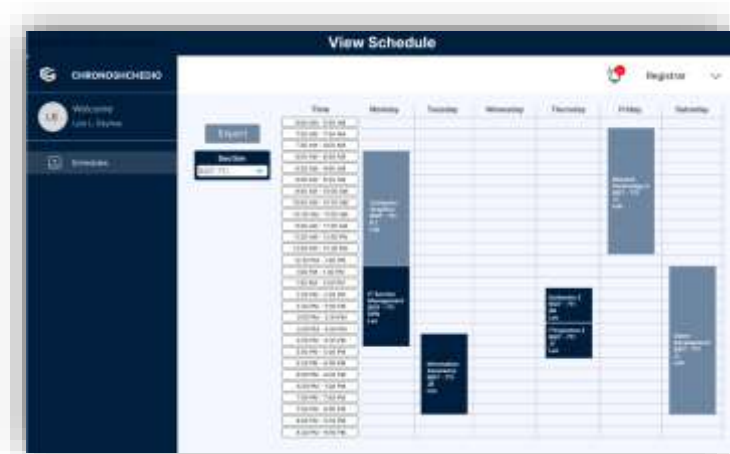
Program Code	Course Code	Course Title	Faculty
001	001	001	001
002	002	002	002
003	003	003	003
004	004	004	004
005	005	005	005
006	006	006	006
007	007	007	007
008	008	008	008
009	009	009	009
010	010	010	010





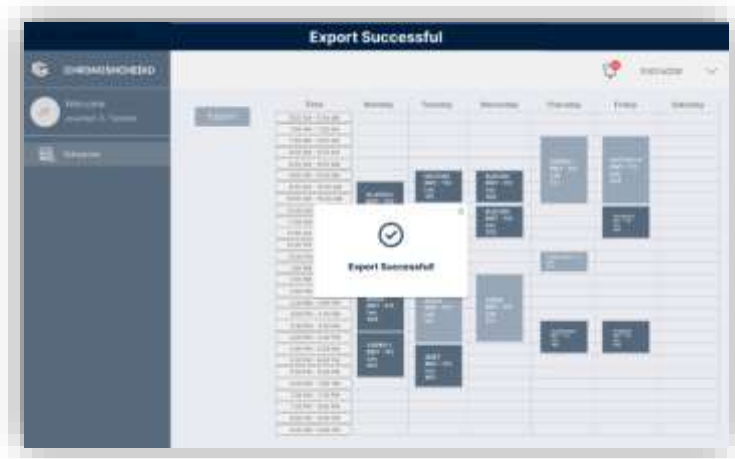


REGISTRAR MODULE

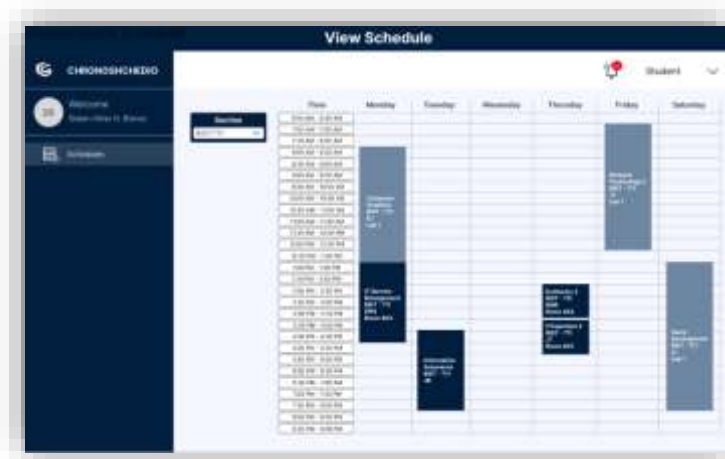


FACULTY MODULE

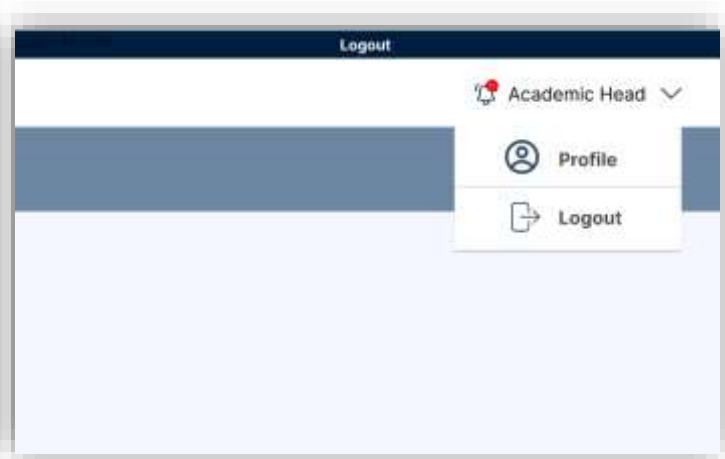




STUDENT MODULE



LOG OUT MODULE



PROFILE MODULE (ACADEMIC HEAD)

Profile Module (Academic Head)

CHRONOSHCHEDO Academic Head

RR Ricson M. Ricardo [Edit Profile](#)

Personal

Basic Information

First Name: Ricson	Middle Name: Miguel	Last Name: Ricardo
Suffix: Nil	Nicknames: N/A	

Contact Information

Contact Number: +63 2 123 0000	Email Address: ricson.m.ricardo@sticollegealabang.edu.ph
--------------------------------	--

Profile Module (Academic Head)

CHRONOSHCHEDO Academic Head

RR Ricson M. Ricardo [Edit Profile](#)

Contact Information

Contact Number: +63 2 123 0000	Email Address: ricson.m.ricardo@sticollegealabang.edu.ph
--------------------------------	--

Work

Job Information

Employee ID: 0000012345	Job Title: Assistant Head	Status: Full Time
-------------------------	---------------------------	-------------------

PROFILE MODULE (PROGRAM HEAD)

Profile Module (Program Head)

CHRONOSHCHEDO Program Head

KA King Jasser N. Andaya [Edit Profile](#)

Personal

Basic Information

First Name: King	Middle Name: James	Last Name: Andaya
Suffix: Nil	Nicknames: King	

Contact Information

Contact Number: +63 2 123 000 000	Email Address: kingjasser.andaya@sticollegealabang.edu.ph
-----------------------------------	---

Profile Module (Program Head)

CHRONOSHCHEDO Program Head

KA King Jasser N. Andaya [Edit Profile](#)

Contact Information

Contact Number: +63 2 123 000 000	Email Address: kingjasser.andaya@sticollegealabang.edu.ph
-----------------------------------	---

Work

Job Information

Employee ID: 0000012345	Job Title: Program Head	Status: Full Time
-------------------------	-------------------------	-------------------

PROFILE MODULE (INSTRUCTOR)

Profile Module (Instructor)

CHRONOSHCHEDO Instructor

DN Darryl Pauline S. Nietes [Edit Profile](#)

Personal

Basic Information

First Name: Darryl	Middle Name: Pauline	Last Name: Nietes
Suffix: S/N	Nicknames: N/A	

Contact Information

Contact Number: +63 91 766 00 00	Email Address: dn-nietes@sticollegealabang.edu.ph
----------------------------------	---

Profile Module (Instructor)

CHRONOSHCHEDO Instructor

DN Darryl Pauline S. Nietes [Edit Profile](#)

Work

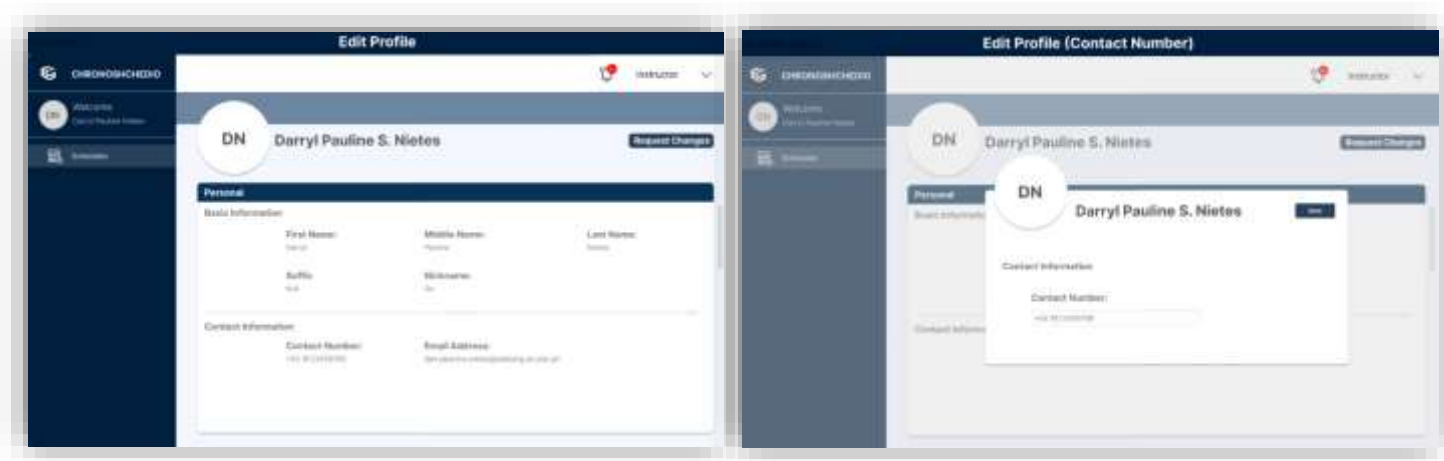
Job Information

Employee ID: 0000012345	Job Title: Instructor	Status: Part Time
-------------------------	-----------------------	-------------------

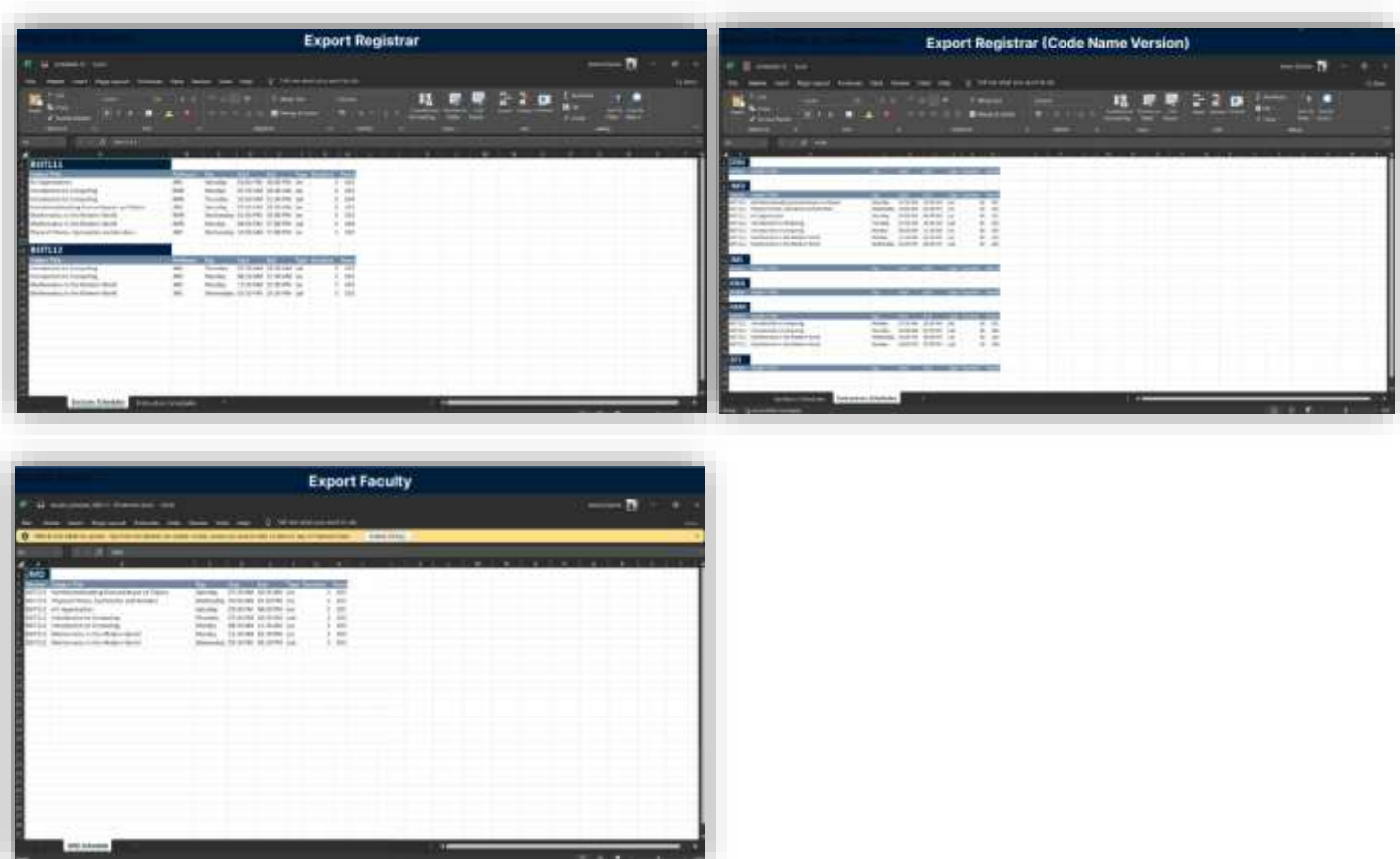
Company

Company Name: St. Columba Schools, Inc.	Position: Instructor
---	----------------------

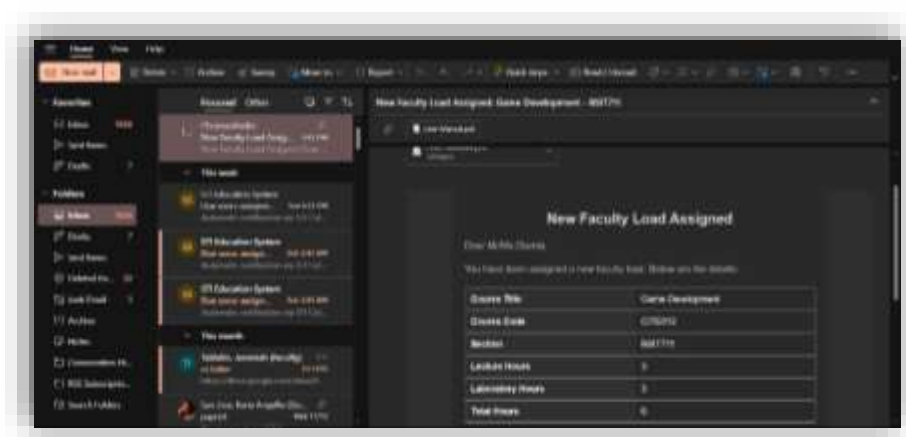
EDIT PROFILE MODULE



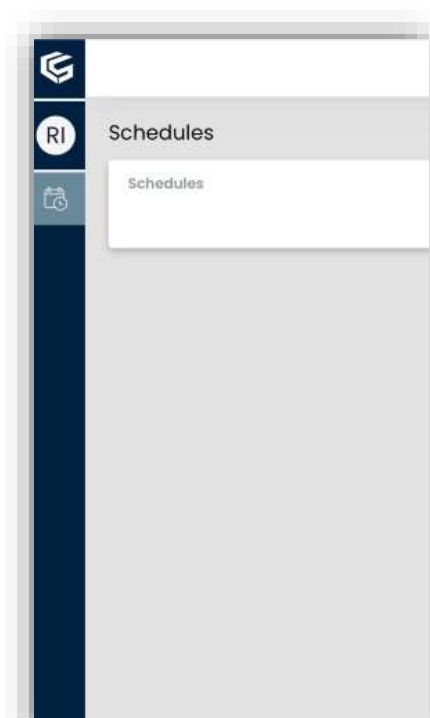
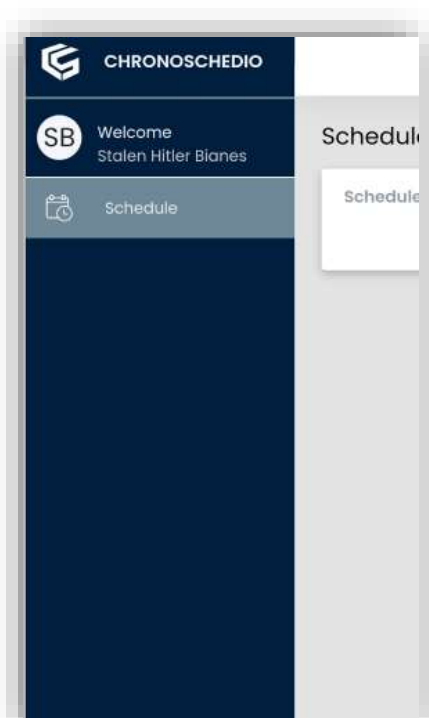
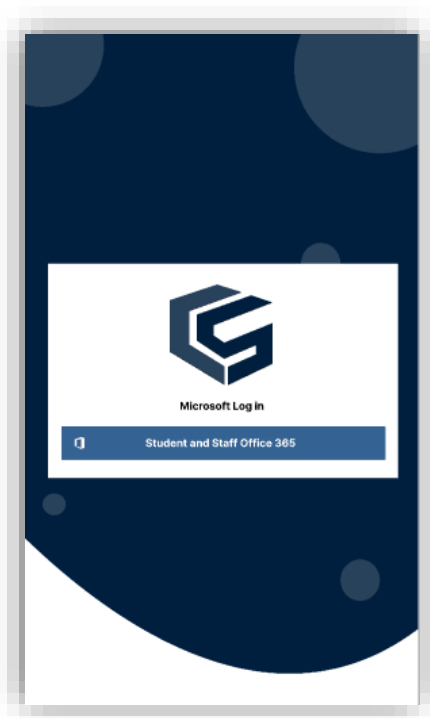
EXPORT EXCEL FILE



EMAIL NOTIFICATION



CHRONOSCHEDIO MOBILE VIEW



CONCLUSION AND RECOMMENDATIONS

Conclusions

In conclusion, the Chronoschedio Implementation of a Web-Based Class Scheduling and Faculty Loading Management for STI College Alabang serves as the solution, the proposed system effectively addresses the current problem faced by the client which is the Academic head of STI College Alabang. By automating the class schedule and utilizing the Office 365 within the institution, it offers counter measure to the problem in creating schedule with conflict checker, room utilization reports, and faculty loading eased the creation of schedule as well as distributing it within the institution.

Recommendations

The researchers' conducted a complete evaluation of the system and identified areas for improvement such as **AI Integration in generating class schedule, Offer Offline Access, Offer a Feedback Section for Academic Head, Program Head, Faculty, Registrar and Students.**

The proponents propose the following recommendation:

1. **AI Integration in generating class schedule**

The AI Integration in terms of generating class schedule will help to create an efficient class scheduling. It will suggest a non-overlapping schedule whenever the academic head and program heads is creating a schedule.

2. **Offer Offline Access for Distant Areas**

Give Academic Head and Program Heads who live in remote locations or have poor internet connectivity offline access so they can examine schedules and make revisions; once a link is established, the changes will be synchronized.

3. **Offer a Feedback Section for Academic Head, Program Head, Faculty, Registrar and Students.**

Add a feature where faculty and students can submit feedback about the class schedule and system usability, which can be used for ongoing improvements and adjustments.

REFERENCES

- AlSaeed, Duaa. (2021). Toward Achieving Quality in Faculty-Load Allocation : *A Developed Faculty-Load-Management System*. International Journal for Quality Research. 14. 10.24874/IJQR14.04-13.
- Campanilla, Bell & Etcuban, Jonathan & Jean, Mary & Campanilla, M & Niñal, Melvin & Dela, Romeo. (2019). *Development of a Web-Based Pre-Loading System for Faculty in a University*
- Caraig, Renelle. (2022). *A Proposed Faculty Loading Guide Framework for the Research Subjects in the Senior High School in the Philippines*. 15. 178-191.
- Grigorkevich, A., Savelyeva, E., Gaifullina, N. et al. *Rigid class scheduling and its value for online learning in higher education*. Educ Inf Technol 27, 12567–12584 (2022). Review from <https://doi.org/10.1007/s10639-022-11131-3>
- H. Algethami and W. Laesanklang, "A Mathematical Model for Course Timetabling Problem With Faculty-Course Assignment Constraints," in IEEE Access, vol. 9, pp. 111666-111682, 2021, doi: 10.1109/ACCESS.2021.3103495
- Jayrhom R. Almonteros, Mark Phil B. Pacot, and Vicente A. Pitogo. 2023. *Automation of Curriculum-based Student-Subject Encoding: A Web Application*. In Proceedings of the 2022 11th International Conference on Networks, Communication and Computing (ICNCC '22). Association for Computing Machinery, New York, NY, USA, 328–333. <https://doi.org/10.1145/3579895.3579944>
- Labuanan, Frederick & Tapaoan, Sheena & Ricardo, Camungao. (2021). *Class Scheduling System*. 10.35940/ijrte.B1026.078219.
- Legaspi, John & De Angel, Roman & Lagman, Ace & Ortega, John. (2019). *Web Based Course Scheduling System Using Greedy Algorithm*. International journal of simulation: systems, science & technology. 10.5013/IJSSST.a.20.S2.14.
- Ren, Xuyue & Li, Chongwei. (2022). *Computer Intelligent Course Scheduling System Based on Deep Learning*. 1-5. 10.1109/ICKECS56523.2022.10060177
- Y. Chen, "Design and implementation of intellective class scheduling system based on negotiation," 2014 9th International Conference on Computer Science & Education, Vancouver, BC, Canada, 2014, pp. 1078-1082, doi: 10.1109/ICCSE.2014.6926629.

APPENDIX A. RESOURCE PERSON

RESOURCE PERSON

The developers of this capstone project would like to express their gratitude to several individuals whose contributions were invaluable to its successful completion. We are particularly indebted to our esteemed capstone advisor, **Ms. Darry Pauline S. Nietes**. Her insightful guidance, unwavering support, and vast knowledge in the field were instrumental in shaping this project's direction. Ms. Nietes extensive knowledge and wealth of experience have left an indelible mark on the academic research and daily pursuit of the developers.

Furthermore, the authors express their sincere gratitude to the faculty at STI College Alabang. Their unwavering support, patience, and insightful feedback throughout the project journey significantly enhanced the outcome of Capstone Project.

APPENDIX B. RELEVANT SOURCECODE

RELEVAN SOURCECODE

user_session.php:

```
<?php

class UserSession

{

    public static function start()

    {

        if (session_status() === PHP_SESSION_NONE) {

            session_start();

        }

    }

    public static function getUserData()

    {

        $user = [];

        if (isset($_SESSION['email'], $_SESSION['firstName'], $_SESSION['lastName'],
$_SESSION['rank'])) {

            $user['email'] = $_SESSION['email'];

            $user['firstName'] = $_SESSION['firstName'];

            $user['lastName'] = $_SESSION['lastName'];

            $user['rank'] = $_SESSION['rank'];
```



```

    }

    return $user;

}

public static function isLoggedIn()

{

    return isset($_SESSION['valid']) && $_SESSION['valid'] === true;

}

public static function logout()

{

    session_unset();

    session_destroy();

}

}

```

UserSession::**start()**;

header.php:

```
<head>
```

```
<meta charset="utf-8">
```

```

<meta http-equiv="X-UA-Compatible" content="IE=edge">

<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

<link rel="stylesheet" href="../shared/css/style.css">

<link rel="stylesheet" href="../shared/loader/loader.css">

<link rel="preconnect" href="https://fonts.gstatic.com">

<link rel="shortcut icon" href="../shared/img/photos/logo.png" />

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

<title>ChronoSchedio</title>

<link href="../shared/css/app.css" rel="stylesheet">

<link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600&display=swap"
rel="stylesheet">

<link rel="stylesheet"
href="//cdn.jsdelivr.net/npm/alertifyjs@1.14.0/build/css/alertify.min.css"/>

<link rel="stylesheet"
href="//cdn.jsdelivr.net/npm/alertifyjs@1.14.0/build/css/themes/bootstrap.min.css"/>

<script src="https://code.jquery.com/jquery-3.7.1.js"></script>

<!-- DataTables CSS-->

<link rel="stylesheet" href="https://cdn.datatables.net/2.0.5/css/dataTables.bootstrap5.css">

```

```

<link rel="stylesheet"
href="https://cdn.datatables.net/responsive/3.0.2/css/responsive.bootstrap5.css">

<script src="https://unpkg.com/dropzone@5/dist/min/dropzone.min.js"></script>

<link rel="stylesheet" href="https://unpkg.com/dropzone@5/dist/min/dropzone.min.css"
type="text/css" />

<!-- DataTables JS -->

<script src="https://cdn.datatables.net/2.1.8/js/dataTables.js"></script>

<script src="https://cdn.datatables.net/2.0.5/js/dataTables.bootstrap5.js"></script>

<script src="https://cdn.datatables.net/responsive/3.0.2/js/dataTables.responsive.js"></script>

<script src="https://cdn.datatables.net/responsive/3.0.2/js/responsive.bootstrap5.js"></script>

<script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.11/index.global.min.js"></script>

<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>

<script src="https://cdn.jsdelivr.net/npm/alertifyjs@1.14.0/build/alertify.min.js"></script>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/html2pdf.js/0.9.2/html2pdf.bundle.min.js"></script>

</head>

```

top-nav.php:

```

<?php

error_reporting(E_ALL);

ini_set('display_errors', 1);

```

```

require_once '../shared/core/user_session.php';

if (!UserSession::isLoggedIn()) {

    echo '

    <style>

        /* Hide everything */

        body > .wrapper {

            display: none !important;

        }

    </style>

    <script type="text/javascript">

        document.addEventListener("DOMContentLoaded", function() {

            Swal.fire({

                icon: "error",

                title: "Oops...",

                text: "You are not yet logged in!",

                confirmButtonColor: "#6D86A1"

            }).then(() => {

                setTimeout(function() {

                    window.location.href = "../";

```

```

        }, 800);

    });

});

</script>

';

exit;

}

$UserData = UserSession::getUserData();

$initials = "";

if (isset($UserData['firstName']) && isset($UserData['lastName'])) {

    $initials = strtoupper(substr($UserData['firstName'], 0, 1) . substr($UserData['lastName'], 0, 1));

}

?>

<nav class="navbar navbar-expand navbar-light navbar-bg topnav">

<!-- <a class="toggle-sidebar">

    <i class="hamburger align-self-center"></i>

</a> -->

<div class="navbar-collapse collapse">

    <ul class="navbar-nav navbar-align">

```

```

<li class="nav-item dropdown">

    <?php if (isset($rank) && ($rank == 2 || $rank == 3)): ?>

        <a class="nav-icon dropdown-toggle" href="#" id="alertsDropdown" data-bs-
toggle="dropdown">

            <div class="position-relative">

                <i class="align-middle" data-feather="bell"></i>

                <span class="indicator bg-danger" id="notification-count" hidden>0</span>

            </div>

        </a>

        <div class="dropdown-menu dropdown-menu-lg dropdown-menu-end py-0" aria-
labelledby="alertsDropdown">

            <div class="dropdown-menu-header" id="notifications-header">

                No New Notifications

            </div>

            <div class="list-group" id="notification-list">

                <!-- Notifications will be dynamically inserted here -->

            </div>

        </div>

    <?php endif; ?>

</li>

<li class="nav-item">

```

```
<a class="nav-link dropdown-toggle d-none d-sm-inline-block" href="#" data-bs-  
toggle="dropdown">
```

```
<span class="text-dark me-2">
```

```
<?php
```

```
switch ($rank) {
```

```
    case 1:
```

```
        echo 'Faculty';
```

```
        break;
```

```
    case 2:
```

```
        echo 'Academic Head';
```

```
        break;
```

```
    case 3:
```

```
        echo 'Admin';
```

```
        break;
```

```
    case 4:
```

```
        echo 'Student';
```

```
        break;
```

```
    case 5:
```

```
        echo 'Program Head';
```

```
        break;
```

```
    case 6:
```

```

        echo 'Registrar';

        break;

    default:

        echo 'Unknown Rank';

        break;

    }

    ?>

</span>

</a>

<div class="dropdown-menu dropdown-menu-end">

    <?php if (in_array($rank, [1, 2, 5])): ?>

        <a class="dropdown-item" href="profile.php">

            <i class="align-middle me-1" data-feather="user"></i> Profile

        </a>

        <div class="dropdown-divider"></div>

    <?php endif; ?>

    <form class="dropdown-item" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]); ?>" method="post">

        <i class="align-middle me-1" data-feather="log-out"></i>

        <input type="submit" name="logout" value="Log out" style="background: none;
border: none; padding: 0; font: inherit; color: inherit; cursor: pointer;">

```



```

        </form>

    </div>

</li>

</ul>

</div>

</nav>

<script>

function loadNotifications() {

    $.ajax({

        url: '../shared/scripts/get-activity-logs.php',

        type: 'GET',

        success: function(response) {

            const notificationCount = $('#notification-count');

            const notificationList = $('#notification-list');

            const notificationHeader = $('#notifications-header');

            const count = response.length;

            const lastNotificationCount = sessionStorage.getItem('lastNotificationCount');

            // Show or hide notification count based on sessionStorage

```

```

if (count > 0) {

    const newNotifications = lastNotificationCount ? count - lastNotificationCount :
count;

    if (newNotifications > 0) {

        notificationCount.text(newNotifications).removeAttr('hidden');

    }

    notificationHeader.text(`${count} New Notifications`);

} else {

    notificationCount.attr('hidden', true);

    notificationHeader.text('No New Notifications');

}

// Update notification list

notificationList.empty();

response.forEach(function(log) {

    const timeAgo = getTimeAgo(log.timestamp);

    const instructorCode = log.instructor_code ?

        '<div class="text-muted small mt-1">Code: ${log.instructor_code}</div>' :

        '';

```

```

const notificationItem = `

    <a href="#" class="list-group-item">

        <div class="row g-0 align-items-center">

            <div class="col-2">

                <i class="text-${getNotificationIconClass(log.action_type)}" data-
feather="${getNotificationIcon(log.action_type)}"></i>

            </div>

            <div class="col-10">

                <div class="text-dark">${log.action_type}</div>

                <div class="text-muted small mt-1">${log.action_description}</div>

                ${instructorCode}

                <div class="text-muted small mt-1">${timeAgo}</div>

            </div>

        </div>

    </a>`;

notificationList.append(notificationItem);

});

feather.replace();

},

error: function(xhr, status, error) {

```

```

        console.error('Error fetching notifications:', error);

    }

});

}

```

```

$('#alertsDropdown').on('click', function() {

    // Hide notification count when clicked

    $('#notification-count').attr('hidden', true);

    // Save the current count to sessionStorage as viewed count

    const notificationCount = $('#notification-count').text();

    sessionStorage.setItem('lastNotificationCount', notificationCount || 0);

});

```

```

function getTimeAgo(timestamp) {

    const now = Date.now();

    const adjustedTimestamp = new Date(timestamp).getTime() + 8 * 60 * 60 * 1000; // Add 8
hours in milliseconds

    const diff = now - adjustedTimestamp;

    const seconds = Math.floor(diff / 1000);

    const minutes = Math.floor(seconds / 60);

    const hours = Math.floor(minutes / 60);

```

```

const days = Math.floor(hours / 24);

if (days > 0) return `${days}d ago`;

if (hours > 0) return `${hours}h ago`;

if (minutes > 0) return `${minutes}m ago`;

return `${seconds}s ago`;

}

```

```

function getNotificationIcon(actionType) {

  const icons = {

    'Profile Update': 'alert-circle',

    'Schedule Export': 'file-text'

  };

  return icons[actionType] || 'bell';

}

```

```

function getNotificationIconClass(actionType) {

  const classes = {

    'Profile Update': 'warning',

    'Schedule Export': 'success'

  };

}

```

```

        return classes[actionType] || 'secondary';

    }

    loadNotifications();

</script>

side-nav.php:

<?php

error_reporting(E_ALL);

ini_set('display_errors', 1);

require '../shared/core/user_session.php';

require '../shared/scripts/data.php';

$current_page = basename($_SERVER['PHP_SELF']);

$rank = $_SESSION['rank'];

if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["logout"])) {

    UserSession::logout();

    header("Location: ../");

    exit;

}

$userData = UserSession::getUserData();

```

```

$initials = "";

if (isset($userData['firstName']) && isset($userData['lastName'])) {

    $initials = strtoupper(substr($userData['firstName'], 0, 1) . substr($userData['lastName'], 0, 1));

}

function checkAccess($current_page, $rank)

{

    $access = [

        1 => ['schedules-faculty.php', 'profile.php'], // Rank 1 (Faculty)

        2 => ['index.php', 'academic-programs.php', 'course-scheduling.php', 'curriculum.php',
'faculty-load.php', 'instructor.php', 'maintenance.php', 'profile.php', 'schedules.php',
'archives.php'], // Rank 2 (Academic Head)

        3 => ['./admin/archived-users.php', './admin/user-management.php'], // Rank 3 (Admin)

        4 => ['schedule.php'], // Rank 4 (Student)

        5 => ['course-scheduling-ph.php', 'profile.php', 'faculty-load-ph.php', 'schedules-ph.php'], //
Rank 5 (Program Head)

        6 => ['schedules-registrar.php'], // Rank 6 (Registrar)

    ];

    $defaultRedirects = [

        1 => './faculty/schedules-faculty',

        2 => './academic-head/',

```

```

3 => '../admin/user-management',

4 => '../student/schedule',

5 => '../program-head/faculty-load-ph',

6 => '../registrar/schedules-registrar',

];

if (isset($access[$rank]) && !in_array($current_page, $access[$rank])) {

    if (isset($defaultRedirects[$rank])) {

        header("Location: " . $defaultRedirects[$rank]);

    } else {

        header("Location: ../");

    }

    exit();

}

}

checkAccess($current_page, $rank);

?>

<nav id="sidebar" class="sidebar js-sidebar" onmouseover="toggleSidebar(true)"

onmouseout="toggleSidebar(false)">
STI College Alabang

```



```

<div class=" sidebar-content js-simplebar">

    <a class="sidebar-brand" href="#">

        <span

            class="align-left logo-text ms-2">CHRONOSCHEDIO</span>

        </a>

    <div class="sidebar-user">

        <div class="d-flex justify-content-center">

            <div class="profile-image rounded-circle p-1 bg-light text-dark d-flex justify-content-
center align-items-center"

                style="width: 40px; height: 40px; font-size: 24px;">

                <?= htmlspecialchars($initials, ENT_QUOTES); ?>

            </div>

            <div class="flex-grow-1 ps-1 profile-details">

                <p class="m-0 fs-5 text-light">

                    Welcome

                </p>

                <div class="fs-6 sidebar-user-subtitle sidebar-name">

                    <?= htmlspecialchars($userData['firstName']) . " " .
htmlspecialchars($userData['lastName']) ?>

                </div>

```

```

        </div>

    </div>

</div>

<ul class="sidebar-nav">

    <?php if ($rank == 1): ?>

        <li class="sidebar-item <?php if ($current_page == 'schedules-faculty.php') {echo
'active';} ?>">

            <a class="sidebar-link" href="schedules-faculty">

                <span class="align-middle sidenav-text">Class Schedules</span>

            </a>

        </li>

    <?php endif; ?>

    <?php if ($rank == 2): ?>

        <li class="sidebar-item <?php if ($current_page == 'index.php') {echo 'active';} ?>">

            <a class="sidebar-link" href="/">

                <span

                    class="align-middle sidenav-text">Dashboard</span>

```

```

        </a>

    </li>

    <li class="sidebar-item <?php if ($current_page == 'instructor.php') {echo 'active';}
?>">

        <a class="sidebar-link" href="instructor">

            <span

                class="align-middle sidenav-text">Instructor</span>

            </a>

        </li>

    <li class="sidebar-item <?php if ($current_page == 'faculty-load.php') {echo 'active';}
?>">

        <a class="sidebar-link" href="faculty-load">

            <span

                class="align-middle sidenav-text">Faculty Load</span>

            </a>

        </li>

    <li class="sidebar-item <?php echo ($current_page == 'academic-programs.php' ||
$current_page == 'curriculum.php') ? 'active' : ''; ?>">

        <a class="sidebar-link" href="academic-programs">

```

```
<span class="align-middle sidenav-text">Curriculum</span>
```

```
</a>
```

```
</li>
```

```
<li class="sidebar-item <?php echo ($current_page == 'course-scheduling.php') ?  
'active' : ''; ?>">
```

```
<a class="sidebar-link" href="course-scheduling">
```

```
<span class="align-middle sidenav-text">Course Scheduling</span>
```

```
</a>
```

```
</li>
```

```
<li class="sidebar-item <?php echo ($current_page == 'schedules.php') ? 'active' : '';  
?>">
```

```
<a class="sidebar-link" href="schedules">
```

```
<span class="align-middle sidenav-text">Schedules</span>
```

```
</a>
```

```
</li>
```

```
<li class="sidebar-item <?php if ($current_page == 'maintenance.php') {
```

```
    echo 'active';
```

```
} ?>">
```

```
<a class="sidebar-link" href="maintenance">
```

```

```

```
<span class="align-middle sidenav-text">Maintenance</span>
```

```
</a>
```

```
</li>
```

```
<li class="sidebar-item <?php if ($current_page == 'archives.php') {
```

```
    echo 'active';
```

```
    } ?>">
```

```
<a class="sidebar-link" href="archives">
```

```

```

```
<span class="align-middle sidenav-text">Archives</span>
```

```
</a>
```

```
</li>
```

```
<?php endif; ?>
```

```
<?php if ($rank == 4): ?>
```

```
<li class="sidebar-item <?php if ($current_page == 'schedule.php') {echo 'active';} ?>">
```

```
<a class="sidebar-link" href="schedule">
```

```

```

```
<span class="align-middle sidenav-text">Schedule</span>
```


<?php endif; ?>

<?php if (\$rank == 5): ?>

<li class="sidebar-item <?php if (\$current_page == 'faculty-load-ph.php'){echo
'active';} ?>">

<span

class="align-middle sidenav-text">Faculty Load

<li class="sidebar-item <?php echo (\$current_page == 'course-scheduling-ph.php') ?
'active' : "; ?>">

Class Scheduling


```

<li class="sidebar-item <?php echo ($current_page == 'schedules-ph.php') ? 'active' : ";
?>">

    <a class="sidebar-link" href="schedules-ph">

        <span class="align-middle sidenav-text">Schedules</span>

    </a>

</li>

<?php endif; ?>

<?php if ($rank == 6): ?>

    <li class="sidebar-item <?php if ($current_page == 'schedules-registrar.php') {echo
'active';} ?>">

        <a class="sidebar-link" href="schedules-registrar">

            <span class="align-middle sidenav-text">Schedule</span>

        </a>

    </li>

<?php endif; ?>

</ul>

</div>

</nav>

```

course-scheduling.php:

```
$(document).ready(function () {  
  
    $.ajax({  
  
        url: '../shared/scripts/get-operation-hours.php',  
  
        type: 'GET',  
  
        dataType: 'json',  
  
        success: function (response) {  
  
            console.log("AJAX Success:", response);  
  
            if (response.status === 'success') {  
  
                // Initialize FullCalendar after successful AJAX call  
  
                initializeFullCalendar(response.operationHours);  
  
            } else {  
  
                console.error('Failed to fetch operation hours:', response.message);  
  
            }  
  
        },  
  
        error: function (xhr, status, error) {  
  
            console.error('An error occurred while fetching operation hours:', error);  
  
        }  
  
    });  
  
    // Initialize FullCalendar function
```



```

function initializeFullCalendar(operationHours) {

    var calendarEl = document.querySelector('calendar');

    var contentEl = document.querySelector('content');

    var externalEventsContainer = $('#external-events');

    externalEventsContainer.hide();

    $('#section').prop('disabled', true);

    $('#program').prop('disabled', true);

    $('#section').append(<option value="" disabled selected>Select a section</option>');


    function getContentWidth() {

        return parseInt(getComputedStyle(contentEl).width, 10);

    }


    // Initialize Draggable only once

    new FullCalendar.Draggable(externalEventsContainer[0], {

        itemSelector: '.fc-event',

        eventData: function (eventEl) {

            return JSON.parse(eventEl.dataset.event);

        }

    });

```

```

function whichView(width) {

    return width < 996 ? 'timeGridDay' : 'timeGridWeek';

}

var calendar = new FullCalendar.Calendar(calendarEl, {

    themeSystem: 'bootstrap5',

    initialView: whichView(contentEl.offsetWidth),

    hiddenDays: [0],

    headerToolbar: whichView(contentEl.offsetWidth) === 'timeGridDay' ? {

        start: 'prev,next',

        center: "",

        end: ""

    } : false,

    contentHeight: "auto",

    allDaySlot: false,

    slotMinTime: operationHours.start_time,

    slotMaxTime: operationHours.end_time,

    slotDuration: '00:30:00',

    slotLabelInterval: '00:30:00',

    slotLabelFormat: {

        hour: 'numeric',

```

```

    minute: '2-digit',

    omitZeroMinute: false,

    meridiem: false

  },

  editable: true,

  eventDurationEditable: false, // Disable resizing for all events

  droppable: true,

  events: [],

  viewDidMount: function () {

    // Add the "Time" label to the time grid axis header

    const axisHeader = document.querySelector('fc-timegrid-axis');

    if (axisHeader && !axisHeader.querySelector('time-label')) {

      const timeLabel = document.createElement('div');

      timeLabel.className = 'time-label'; // Custom styling

      timeLabel.innerText = 'Time';

      axisHeader.prepend(timeLabel); // Add the "Time" label to the header

    }

  },

  slotLabelContent: function (arg) {

    var startTime = arg.date.toLocaleTimeString([], {

```

```

        hour: 'numeric',

        minute: '2-digit',

        hour12: true

    });

    var endTime = new Date(arg.date.getTime() + 30 * 60 *
1000).toLocaleTimeString([], {

        hour: 'numeric',

        minute: '2-digit',

        hour12: true

    });

    return startTime + ' - ' + endTime;

},

dayHeaderContent: function (arg) {

    return arg.date.toLocaleDateString(undefined, {

        weekday: 'long'

    }).toUpperCase();

},

eventReceive: function (info) {

    if (!info || !info.event) {

        console.error('Invalid event data received:', info);

```

```

        return;

    }

    var room_id = $('#room').val(); // Room selected by the user

    if (room_id == 0) {

        alertify.error("Select room first");

    }

    var professor = info.event.extendedProps.prof;

    var startTime = info.event.start instanceof Date ?

moment(info.event.start).format('HH:mm:ss') : '';

    var duration = info.event.extendedProps.duration; // Should already be in minutes

    var endTime = startTime ? moment(info.event.start).add(duration,

'minutes').format('HH:mm:ss') : '';

    var daysOfWeek = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',

'Saturday'];

    var startDayOfWeek = info.event.start instanceof Date ?

daysOfWeek[info.event.start.getDay()] : '';

    var section = info.event.extendedProps.section;

    var eventData = {

        action: 'insert',

```

```

        title: info.event.title,

        startTime: startTime,

        endTime: endTime,

        room: room_id,

        section: section,

        professor: professor,

        daysOfWeek: startDayOfWeek,

        type: info.event.extendedProps.type,

        duration: duration // Keep this as total minutes

    };

    if (info.event.extendedProps.is_split) {

        info.event.setProp('classNames', ['event-split']);

        eventData.is_split = info.event.extendedProps.is_split;

        eventData.split_reference = info.event.extendedProps.split_reference;

    }

    // Fetch all existing schedules to check for conflicts

    $.ajax({

        url: '../shared/scripts/get-schedules-room.php', // Endpoint to fetch all schedules

        type: 'GET',

```

```

success: function (response) {

    if (response.status === 'success') {

        var schedules = response.data;

        var conflictFound = false;

        for (var i = 0; i < schedules.length; i++) {

            var existingSchedule = schedules[i];

            if (existingSchedule.professor === professor && existingSchedule.section
=== eventData.section &&

                existingSchedule.days_of_week === eventData.daysOfWeek) {

                var existingStartTime = moment(existingSchedule.start_time,
'HH:mm:ss');

                var existingEndTime = moment(existingSchedule.end_time, 'HH:mm:ss');

                console.log(existingSchedule.room_number);

                var newStartTime = moment(startTime, 'HH:mm:ss');

                var newEndTime = moment(endTime, 'HH:mm:ss');

                if (newStartTime.isBefore(existingEndTime) &&
newEndTime.isAfter(existingStartTime)) {

                    conflictFound = true;

                    alertify.set('notifier', 'delay', 10);

                    alertify.error("Conflict detected! Professor " +

existingSchedule.professor +
STI College Alabang

```

```

        " and section " + existingSchedule.section +

        " already have a schedule for subject " +
existingSchedule.subject_title +

        " in room " + existingSchedule.room_number +

        " on " + existingSchedule.days_of_week +

        " from " + existingSchedule.start_time + " to " +
existingSchedule.end_time + ".".");

        alertify.set('notifier', 'delay', 5);

    }

    } else if (existingSchedule.professor === professor &&
existingSchedule.days_of_week === eventData.daysOfWeek) {

        // Conflict for professor only

        console.log("Conflict detected for professor " + professor);

        var existingStartTime = moment(existingSchedule.start_time,
'HH:mm:ss');

        var existingEndTime = moment(existingSchedule.end_time, 'HH:mm:ss');

        var newStartTime = moment(startTime, 'HH:mm:ss');

        var newEndTime = moment(endTime, 'HH:mm:ss');

```



```

        if (newStartTime.isBefore(existingEndTime) &&
newEndTime.isAfter(existingStartTime)) {

            conflictFound = true;

            alertify.set('notifier', 'delay', 10);

            alertify.error("Conflict detected with an existing schedule in room " +
existingSchedule.room_number +

                ". Professor " + existingSchedule.professor +

                " already has a schedule for subject " + existingSchedule.subject_title
+

                " on " + existingSchedule.days_of_week +

                " from " + existingSchedule.start_time + " to " +
existingSchedule.end_time + ".");

            alertify.set('notifier', 'delay', 5);

        }

    } else if (existingSchedule.section === eventData.section &&
existingSchedule.days_of_week === eventData.daysOfWeek) {

        var existingStartTime = moment(existingSchedule.start_time,
'HH:mm:ss');

        var existingEndTime = moment(existingSchedule.end_time, 'HH:mm:ss');

```

```

var newStartTime = moment(startTime, 'HH:mm:ss');

var newEndTime = moment(endTime, 'HH:mm:ss');


if (newStartTime.isBefore(existingEndTime) &&
newEndTime.isAfter(existingStartTime)) {

    conflictFound = true;

    alertify.set('notifier', 'delay', 10);

    alertify.error("Conflict detected with an existing schedule in room " +
existingSchedule.room_number +

    ". Section " + existingSchedule.section +

    " already has a schedule for subject " + existingSchedule.subject_title
+

    " on " + existingSchedule.days_of_week +

    " from " + existingSchedule.start_time + " to " +
existingSchedule.end_time + ".");

    alertify.set('notifier', 'delay', 5);

    }

    }

    }

    if (conflictFound) {

        info.event.remove();

    } else {

```

```

        saveEventToDatabase(eventData, info);

    }

    } else {

        console.error('Error fetching schedules:', response.message);

        alertify.error("Error fetching schedules.");

    }

    },

    error: function (xhr, status, error) {

        console.error("AJAX error:", error);

    }

    });

    },

    eventDrop: function (info) {

        const movedEvent = info.event;

        const eventId = movedEvent.id; // Assuming the event ID is now correctly set

        console.log("Moved Event ID from schedules table:", eventId);

        // Get new start time and calculate end time based on original duration

        const startTime = movedEvent.start instanceof Date ?

moment(movedEvent.start).format('HH:mm:ss') : ";

```

```

const duration = movedEvent.extendedProps.duration || 180; // Use the same duration
logic

const endTime = startTime ? moment(movedEvent.start).add(duration,
'minutes').format('HH:mm:ss') : ";

// Get the day of the week based on the new start time

const daysOfWeek = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
'Saturday'];

const startDayOfWeek = movedEvent.start instanceof Date ?
daysOfWeek[movedEvent.start.getDay()] : ";

// Prepare data to send to the server

const updatedData = {

    id: eventId,

    start: startTime,

    end: endTime,

    days_of_week: startDayOfWeek,

};

// Check for conflicts with existing events

$.ajax({

    url: '../shared/scripts/get-schedules-room.php', // Fetch all schedules

```

```

type: 'GET',

success: function (response) {

    if (response.status === 'success') {

        var schedules = response.data;

        var conflictFound = false;

        for (var i = 0; i < schedules.length; i++) {

            var existingSchedule = schedules[i];

            if (String(existingSchedule.id) === String(eventId)) {

                console.log("Skipping self-conflict check for ID:", eventId);

                continue;

            }

            if (existingSchedule.professor === movedEvent.extendedProps.prof &&
existingSchedule.section === movedEvent.extendedProps.section &&

                existingSchedule.days_of_week === updatedData.days_of_week) {

                var existingStartTime = moment(existingSchedule.start_time,
'HH:mm:ss');

                var existingEndTime = moment(existingSchedule.end_time, 'HH:mm:ss');

                var newStartTime = moment(startTime, 'HH:mm:ss');

                var newEndTime = moment(endTime, 'HH:mm:ss');

```

```

        if (newStartTime.isBefore(existingEndTime) &&
newEndTime.isAfter(existingStartTime)) {

            conflictFound = true;

            alertify.set('notifier', 'delay', 10);

            alertify.error("Conflict detected! Professor " +
existingSchedule.professor +

                " and section " + existingSchedule.section +

                " already have a schedule for subject " +
existingSchedule.subject_title +

                " in room " + existingSchedule.room_number +

                " on " + existingSchedule.days_of_week +

                " from " + existingSchedule.start_time + " to " +
existingSchedule.end_time + ".");

            alertify.set('notifier', 'delay', 5);

        }

    } else if (existingSchedule.professor === movedEvent.extendedProps.prof
&& existingSchedule.days_of_week === updatedData.days_of_week) {

        // Conflict for professor only

        console.log("Conflict detected for professor " + professor);

```

```

var existingStartTime = moment(existingSchedule.start_time,
'HH:mm:ss');

var existingEndTime = moment(existingSchedule.end_time, 'HH:mm:ss');

var newStartTime = moment(startTime, 'HH:mm:ss');

var newEndTime = moment(endTime, 'HH:mm:ss');

if (newStartTime.isBefore(existingEndTime) &&
newEndTime.isAfter(existingStartTime)) {

    conflictFound = true;

    alertify.set('notifier', 'delay', 10);

    alertify.error("Conflict detected with an existing schedule in room " +
existingSchedule.room_number +

    ". Professor " + existingSchedule.professor +

    " already has a schedule for subject " + existingSchedule.subject_title
+

    " on " + existingSchedule.days_of_week +

    " from " + existingSchedule.start_time + " to " +
existingSchedule.end_time + ".");

    alertify.set('notifier', 'delay', 5);

}

```

```

    } else if (existingSchedule.section === movedEvent.extendedProps.section
    && existingSchedule.days_of_week === updatedData.days_of_week) {

        var existingStartTime = moment(existingSchedule.start_time,
        'HH:mm:ss');

        var existingEndTime = moment(existingSchedule.end_time, 'HH:mm:ss');

        var newStartTime = moment(startTime, 'HH:mm:ss');

        var newEndTime = moment(endTime, 'HH:mm:ss');

        if (newStartTime.isBefore(existingEndTime) &&
        newEndTime.isAfter(existingStartTime)) {

            conflictFound = true;

            alertify.set('notifier', 'delay', 10);

            alertify.error("Conflict detected with an existing schedule in room " +
            existingSchedule.room_number +

            ". Section " + existingSchedule.section +

            " already has a schedule for subject " + existingSchedule.subject_title
            +

            " on " + existingSchedule.days_of_week +

            " from " + existingSchedule.start_time + " to " +
            existingSchedule.end_time + ".");

```



```

        alertify.set('notifier', 'delay', 5);

    }

}

}

if (conflictFound) {

    info.revert();

} else {

    $.ajax({

        url: '../shared/scripts/update-schedule.php',

        type: 'POST',

        contentType: 'application/json',

        data: JSON.stringify(updatedData),

        success: function (response) {

            const result = JSON.parse(response);

            if (result.success) {

                alertify.success("Schedule Updated");

            } else {

                console.error("Error updating schedule:", result.error);

            }

        },

```

```

        error: function (xhr, status, error) {

            console.error("AJAX error:", error);

        }

    });

}

} else {

    console.error('Error fetching schedules:', response.message);

    alertify.error("Error fetching schedules.");

}

},

error: function (xhr, status, error) {

    console.error("AJAX error:", error);

}

});

},

```

```

eventClick: function (info) {

```

```

    const originalDuration = info.event.end.getTime() - info.event.start.getTime(); //

```

Duration in milliseconds

```

    const eventDurationInHours = originalDuration / (1000 * 60 * 60); // Convert to hours

```

```

    Swal.fire({

```

```

        title: 'What would you like to do?',

        text: "Choose an option for this subject.",

        icon: 'question',

        showCancelButton: true,

        denyButtonColor: '#346292',

        showDenyButton: eventDurationInHours === 3, // Show "Split" only for 3-hour
events

        confirmButtonColor: '#6D86A1',

        confirmButtonText: 'Remove',

        denyButtonText: 'Split'

    }).then((result) => {

        if (result.isConfirmed) {

            // Handle the deletion of the event

            const eventId = info.event.id; // Get the ID of the event

            $.ajax({

                url: '../shared/scripts/delete-schedule.php',

                type: 'POST',

                contentType: 'application/json',

                data: JSON.stringify({

                    id: eventId

                }),

```

```

success: function (response) {

    const result = JSON.parse(response);

    if (result.status === "success") {

        info.event.remove(); // Remove the event from the calendar

        appendSubjectBackToContainer(info.event); // Re-add the event to the
container

        alertify.success("Subject Removed"); // Show success message

    } else {

        alertify.error("Error deleting event: " + result.message); // Handle error
response

    }

    },

    error: function (xhr, status, error) {

        alertify.error("Failed to delete event.");

    }

});

} else if (result.isDenied && eventDurationInHours === 3) {

    // Handle splitting the event

    const eventId = info.event.id;

    const halfDuration = originalDuration / 2;

    const firstHalfEnd = new Date(info.event.start.getTime() + halfDuration);

```

```

const secondHalfStart = firstHalfEnd;

const secondHalfEnd = new Date(secondHalfStart.getTime() + halfDuration);

const room_id = $('#room').val();

const duration = info.event.extendedProps.duration / 2;


// Generate a unique split reference for these parts

const splitReference = `split-${eventId}-${Date.now()}`;


// Prepare event data for the split events

const splitEventData1 = {

  action: 'insert',

  title: info.event.title,

  startTime: moment(info.event.start).format('HH:mm:ss'),

  endTime: moment(firstHalfEnd).format('HH:mm:ss'),

  room: room_id,

  section: info.event.extendedProps.section,

  professor: info.event.extendedProps.prof,

  daysOfWeek: moment(info.event.start).format('dddd'),

  type: info.event.extendedProps.type,

  duration: duration,

  is_split: 1,

```

```

        split_reference: splitReference

    };

    const splitEventData2 = {

        action: 'insert',

        title: info.event.title,

        startTime: moment(secondHalfStart).format('HH:mm:ss'),

        endTime: moment(secondHalfEnd).format('HH:mm:ss'),

        room: room_id,

        section: info.event.extendedProps.section,

        professor: info.event.extendedProps.prof,

        daysOfWeek: moment(info.event.start).format('dddd'),

        type: info.event.extendedProps.type,

        duration: duration,

        is_split: 1,

        split_reference: splitReference

    };

    // Delete the original event

    $.ajax({

        url: '../shared/scripts/delete-schedule.php',

```

```

type: 'POST',

contentType: 'application/json',

data: JSON.stringify({

    id: eventId

}),

success: function (response) {

    const result = JSON.parse(response);

    if (result.status === "success") {

        info.event.remove();

        // Save each split event

        [splitEventData1, splitEventData2].forEach((splitData, index) => {

            $.ajax({

                url: '../shared/scripts/save-schedule.php',

                type: 'POST',

                contentType: 'application/json',

                data: JSON.stringify(splitData),

                success: function (response) {

                    const result = JSON.parse(response);

                    if (result.status === 'success') {

                        console.log(splitData);

```

```

calendar.addEvent({

    id: result.id,

    title: splitData.title,

    start: index === 0 ? info.event.start : secondHalfStart,

    end: index === 0 ? firstHalfEnd : secondHalfEnd,

    extendedProps: {

        section: splitData.section,

        prof: splitData.professor,

        type: splitData.type,

        duration: splitData.duration,

        is_split: 1,

        split_reference: splitReference

    },

    classNames: ['event-split']

});

```

```

const eventElement = document.createElement('div');

eventElement.dataset.event = JSON.stringify({

    title: splitData.title,

    section: splitData.section,

    prof: splitData.professor,

```



```

        type: splitData.type,

        duration: splitData.duration,

        is_split: 1,

        split_reference: splitReference,

        extendedProps: {

            section: splitData.section,

            prof: splitData.professor,

            type: splitData.type,

            duration: splitData.duration

        }

    });

}

},

    error: function (xhr, status, error) {

        console.error("Failed to save split event:", error);

    }

});

});

    alertify.success("Subject Split Successfully");

}

```

```

    },

    error: function (xhr, status, error) {

        console.error("Failed to delete original event:", error);

    }

});

}

});

},

```

```

eventContent: function (arg) {

    return {

        html: '<div class="fc-event-content ">' +

            '<div class="fc-event-title">' + arg.event.title + '</div>' +

            '<div class="fc-event-details">' +

            '<div class="fc-event-section">' + arg.event.extendedProps.section + '</div>' +

            '<div class="fc-event-prof">' + arg.event.extendedProps.prof + '</div>' +

            '<div class="fc-event-type">' + arg.event.extendedProps.type + '</div>' +

            '</div>' +

            '</div>'

    };

}

```

```

    },

    eventDidMount: function (info) {

        let backgroundColor;

        if (info.event.extendedProps.is_split) {

            // Set color for split events

            backgroundColor = '#346292';

        } else {

            // Set color based on type (lec/lab) for non-split events

            let eventType = info.event.extendedProps.type.toLowerCase();

            backgroundColor = eventType === 'lec' ? '#6D86A1' : '#001F3F';

        }

        // Apply the color to the event element

        info.el.style.backgroundColor = backgroundColor;

    },

    eventOverlap: false

});

calendar.render();

```

```

function updateHeaderToolbar() {

    const view = calendar.view.type;

    if (view === 'timeGridDay') {

        calendar.setOption('headerToolbar', {

            start: 'prev,next',

            center: "",

            end: ""

        });

    } else {

        calendar.setOption('headerToolbar', false);

    }

}

window.addEventListener('resize', function () {

    calendar.changeView(whichView(getContentWidth()));

    updateHeaderToolbar();

});

calendar.on('viewDidMount', function () {

    updateHeaderToolbar();

});

```

```

function appendSubjectBackToContainer(event) {

    var selectedSection = $('#section').val();

    console.log('Selected Section:', selectedSection);

    if (event.extendedProps.section !== selectedSection) {

        console.log("Event section does not match the selected section, skipping append.");

        return;

    }

    externalEventsContainer.find('no-faculty-loads').remove();

    // Only perform duplicate check for non-split events

    if (!event.extendedProps.is_split) {

        let existingEvent = externalEventsContainer.find(

            `[data-event*="title":"${event.title}"][data-`
event*="type":"${event.extendedProps.type}"]`

        );

        if (existingEvent.length) {

            console.log("Non-split event already exists in external events container, skipping re-
add.");

            return;

```

```

    }

}

// Check if this is a split event and if its pair is already in the container

if (event.extendedProps.is_split) {

    let relatedSplitEvents = externalEventsContainer.find(

        `[data-event*="title": "${event.title}"] [data-
event*="type": "${event.extendedProps.type}"] [data-event*="is_split":1]`

    );

    if (relatedSplitEvents.length === 1) {

        // Found the matching split event, now merge both

        let firstHalfDuration = parseInt(event.extendedProps.duration);

        let secondHalfDuration =

parseInt($(relatedSplitEvents[0]).data("event").extendedProps.duration);

        let mergedDuration = firstHalfDuration + secondHalfDuration;

        // Remove the existing split events from the container

        relatedSplitEvents.remove();

        // Create merged event element

        var mergedEventElement = document.createElement('div');

```

```

mergedEventElement.className = 'fc-event'; // Base class

// Add conditional classes based on the type

if (event.extendedProps.type === 'Lec') {

    mergedEventElement.classList.add('event-lec');

} else if (event.extendedProps.type === 'Lab') {

    mergedEventElement.classList.add('event-lab');

}

let mergedHours = Math.floor(mergedDuration / 60);

let mergedMinutes = mergedDuration % 60;

let formattedMergedDuration = `${mergedHours}:${mergedMinutes < 10 ? '0' :
`${mergedMinutes}`;

// Set inner HTML for the merged event

mergedEventElement.innerHTML = `

<div class="event-title">${event.title}</div>

<div class="event-details">

    <div class="event-section">Section: ${event.extendedProps.section}</div>

    <div class="event-prof">Professor: ${event.extendedProps.prof}</div>

    <div class="event-type">Type: ${event.extendedProps.type}</div>

```

```

        <div class="event-duration">Duration: ${formattedMergedDuration}</div>

    </div>

`;

// Include all relevant properties in the data-event attribute

mergedEventElement.dataset.event = JSON.stringify({

    title: event.title,

    section: event.extendedProps.section,

    prof: event.extendedProps.prof,

    type: event.extendedProps.type,

    duration: formattedMergedDuration,

    is_split: false, // Mark as merged

    extendedProps: {

        section: event.extendedProps.section,

        prof: event.extendedProps.prof,

        type: event.extendedProps.type,

        duration: mergedDuration // Store the merged duration

    }

});

// Append the merged event element to the external events container

```



```

        externalEventsContainer.append(mergedEventElement);

        console.log("Merged split events successfully.");

        return;
    }
}

// If not a split event or not merging, create a regular event element

let type = event.extendedProps.type === 'Lec' ? 'Lec' : event.extendedProps.type ===
'Lab' ? 'Lab' : 'Unknown';

var eventElement = document.createElement('div');

eventElement.className = type === 'Lec' ? 'fc-event event-lec' : 'fc-event event-lab';

if (event.extendedProps.is_split) {

    eventElement.classList.add('event-split');

    eventElement.style.backgroundColor = '#346292'; // Set color for split events
}

let duration = event.extendedProps.duration / 60;

let hours = Math.floor(duration);

let minutes = Math.round((duration - hours) * 60);

let formattedDuration = `${hours}:${minutes < 10 ? '0' : ''}${minutes}`;

```

```

// Set inner HTML with duration

eventElement.innerHTML = `

    <div class="event-title">${event.title}</div>

    <div class="event-details">

        <div class="event-section">Section: ${event.extendedProps.section}</div>

        <div class="event-prof">Professor: ${event.extendedProps.prof}</div>

        <div class="event-type">Type: ${type === 'Lec' ? 'Lecture' : 'Laboratory'}</div>

        <div class="event-duration">Duration: ${formattedDuration}</div>

    </div>

`;

// Include all relevant properties in the data-event attribute

eventElement.dataset.event = JSON.stringify({

    title: event.title,

    section: event.extendedProps.section,

    prof: event.extendedProps.prof,

    type: type,

    duration: formattedDuration,

    is_split: event.extendedProps.is_split,

    split_reference: event.extendedProps.split_reference,

```

```

    extendedProps: {

        section: event.extendedProps.section,

        prof: event.extendedProps.prof,

        type: type,

        duration: event.extendedProps.duration // Original duration in minutes or seconds

    }

});

// Append the new event element to the external events container if not merged

externalEventsContainer.append(eventElement);

}

$('#program').on('change', function () {

    let selectedProgram = $(this).val(); // Get the selected program

    console.log("")

    // Clear the sections dropdown and disable it initially

    let courseSelect = $('#section');

    courseSelect.empty();

    courseSelect.append('<option value="" disabled selected>Select a section</option>');

    courseSelect.prop('disabled', true); // Disable section until program is selected

```

```

// If a program is selected

if (selectedProgram) {

    $.ajax({

        url: '../shared/scripts/get-section-by-program-code.php', // Use your existing get-
sections.php

        data: {

            program: selectedProgram

        }, // Send the selected program to the backend

        dataType: 'json',

        success: function (response) {

            if (response.data && response.data.length > 0) {

                // Enable the section select dropdown and populate it

                courseSelect.prop('disabled', false); // Enable section select

                response.data.sort((a, b) => a.section.localeCompare(b.section)); // Sort
sections

                // Populate the sections dropdown

                response.data.forEach(function (section) {

                    courseSelect.append('<option
value="$ {section.section}">$ {section.section}</option>');

                });

```

```

    } else {

        // If no sections found, alert the user

        alertify.error('No sections available for the selected program.');
```

}

```

    },

    error: function (xhr, status, error) {

        console.error(error);

        alertify.error('Error fetching sections.');
```

}

```

    });

} else {

    // If no program is selected, keep the section dropdown disabled

    courseSelect.prop('disabled', true); // Keep the section disabled if no program is
selected

}

});

$('#room').on('change', function () {

    var roomId = $(this).val();

    $('#program').prop('disabled', false);
```

```

$.ajax({

    url: '../shared/scripts/fetch-schedules.php',

    type: 'GET',

    data: {

        room_id: roomId

    },

    success: function (response) {

        var data = JSON.parse(response);

        console.log("Schedules: ", data);

        if (data.events) {

            calendar.getEvents().forEach(event => event.remove());

            data.events.forEach(eventData => {

                const daysOfWeek = ["Sunday", "Monday", "Tuesday", "Wednesday",
"Thursday", "Friday", "Saturday"];

                const dayIndex = daysOfWeek.indexOf(eventData.days_of_week);

                if (dayIndex !== -1) {

                    const isSplit = eventData.is_split || false;

                    calendar.addEvent({

```

```

        id: eventData.id,

        title: eventData.title,

        daysOfWeek: [dayIndex],

        startTime: eventData.start_time,

        endTime: eventData.end_time,

        extendedProps: {

            section: eventData.section,

            prof: eventData.prof,

            type: eventData.type,

            duration: eventData.duration,

            is_split: isSplit,

            split_reference: eventData.split_reference

        },

        classNames: isSplit ? ['event-split'] : [],

        backgroundColor: isSplit ? '#346292' : (eventData.type === 'lec' ?
'#6D86A1' : '#001F3F')

    });

} else {

    console.warn(`Invalid day of week: ${eventData.days_of_week}`);

}

});

```

```

    }

    },

    error: function (xhr, status, error) {

        console.error("Failed to fetch events:", error);

    }

});

});

// Populate rooms select

$.ajax({

    url: '../shared/scripts/get-rooms.php',

    dataType: 'json',

    success: function (response) {

        if (response.data) {

            let roomSelect = $('#room');

            roomSelect.empty();

            roomSelect.append(<option value="" disabled selected>Select a room</option>');

            response.data.sort((a, b) => a.room_number.localeCompare(b.room_number));

            response.data.forEach(function (room) {

```



```

        roomSelect.append('<option
value="{room.room_id}">{room.room_number}</option>');

    });

}

},

error: function (xhr, status, error) {

    console.error(error);

    alertify.error('Error fetching rooms.');
```

```

// Create a Set to store unique program codes

let uniquePrograms = new Set();

// Append only unique programs to the select dropdown

response.data.forEach(function (program) {

    if (!uniquePrograms.has(program.program_code)) {

        uniquePrograms.add(program.program_code); // Add the program code to the
set

        programSelect.append('<option
value="{program.program_code}">{program.program_code}</option>');

    }

});

},

error: function (xhr, status, error) {

    console.error(error);

    alertify.error('Error fetching programs.');
```

```

// Fetch faculty loads based on selected section
STI College Alabang
```

```

$('#section').on('change', function () {

    const section = $(this).val();

    var resetButton = $('#resetButton');

    externalEventsContainer.show();

    externalEventsContainer.empty(); // Clear previous events

    if (section) {

        console.log("Selected section:", section);

        // Step 1: Fetch all scheduled events from the database

        $.ajax({

            url: '../shared/scripts/get-schedules.php',

            type: 'GET',

            dataType: 'json',

            success: function (scheduleResponse) {

                const scheduledSubjects = new Set();

                const splitEvents = {};

                resetButton.prop('disabled', false);

                console.log("Database schedules response:", scheduleResponse);

                // Populate scheduledSubjects with database events

```

```

if (scheduleResponse.data && Array.isArray(scheduleResponse.data)) {

  scheduleResponse.data.forEach(eventData => {

    const title = eventData.subject_title || 'No Title';

    const identifier = `${title}-${eventData.section}-${eventData.type}`;

    scheduledSubjects.add(identifier);

    // Track split events based on duration

    if (eventData.is_split) {

      if (!splitEvents[identifier]) splitEvents[identifier] = {

        count: 0,

        reference: eventData.split_reference

      };

      splitEvents[identifier].count++;

    }

  });

} else {

  console.warn("No events found in scheduleResponse or it's not an array:",
scheduleResponse);

}

// Step 2: Fetch the faculty loads

```

```

$.ajax({

    url: '../shared/scripts/fetch-faculty-loads.php',

    type: 'GET',

    dataType: 'json',

    data: {

        section: section

    },

    success: function (response) {

        console.log("Faculty loads response:", response);

        if (response && response.data) {

            let hasLoads = false;

            // Display lecture events

            response.data.lec.forEach(function (load) {

                const title = load.course_title || 'No Title';

                const identifier = `${title}-${load.section}-Lec`;

                if (!scheduledSubjects.has(identifier)) {

                    hasLoads = true;

                    scheduledSubjects.add(identifier);

```

```

        let eventHtml = createEventHtml(load, 'Lec');

        externalEventsContainer.append(eventHtml);

    }

    // Handle split events not fully scheduled

    if (splitEvents[identifier] && splitEvents[identifier].count === 1) {

        hasLoads = true;

        let partHtml = createSplitEventHtml(load, 'Lec', 90,
splitEvents[identifier].reference);

        externalEventsContainer.append(partHtml);

    }

});

// Display laboratory events

response.data.lab.forEach(function (load) {

    const title = load.course_title || 'No Title';

    const identifier = `${title}-${load.section}-Lab`;

    if (!scheduledSubjects.has(identifier)) {

        hasLoads = true;

        scheduledSubjects.add(identifier);
    }
});

```

```

        let eventHtml = createEventHtml(load, 'Lab');

        externalEventsContainer.append(eventHtml);

    }

    // Handle split events not fully scheduled

    if (splitEvents[identifier] && splitEvents[identifier].count === 1) {

        hasLoads = true;

        let partHtml = createSplitEventHtml(load, 'Lab', 90,
splitEvents[identifier].reference);

        externalEventsContainer.append(partHtml);

    }

});

if (!hasLoads) {

    externalEventsContainer.append(<div class="no-faculty-loads">No
loads available for this section.</div>');

}

}

},

error: function (xhr, status, error) {

    console.error("AJAX error fetching faculty loads:", error);

```

```

        alertify.error('Error fetching faculty loads.');
```

}

```

    });

  },

  error: function (xhr, status, error) {

    console.error("Error fetching existing schedules:", error);

    alertify.error('Error fetching existing schedules.');
```

}

```

  });

}

});
```

```

function saveEventToDatabase(eventData, info) {

  $.ajax({

    url: '../shared/scripts/save-schedule.php',

    type: 'POST',

    contentType: 'application/json',

    data: JSON.stringify(eventData),

    success: function (response) {

      var result = JSON.parse(response);

      if (result.status === 'success') {
```



```

    if (result.id) {

        info.event.setProp('id', result.id);

        console.log("Assigned new event ID:", result.id);

    }

    info.draggedEl.parentNode.removeChild(info.draggedEl);

    const sectionToCheck = $('#section').val();

    const remainingEvents = externalEventsContainer.find('.fc-event').filter(function
() {

        const eventData = JSON.parse($(this).attr('data-event').replace(/&quot;/g,
        ""));

        return eventData.section === sectionToCheck;

    });

    if (remainingEvents.length === 0) {

        console.log("No loads available for this section.");

        externalEventsContainer.append('<div class="no-faculty-loads">No loads
available for this section.</div>');

    }

    alertify.success("Schedule Saved");

} else {

```

```

        console.error("Error saving event:", result.message);

        alertify.error("Error saving subject");

    }

},

error: function (xhr, status, error) {

    console.error("AJAX error:", error);

}

});

}

function createEventHtml(load, type) {

    const hours = Math.floor(load.duration);

    const minutes = Math.round((load.duration - hours) * 60);

    const formattedDuration = `${hours}:${minutes < 10 ? '0' : ''}${minutes}`;

    return `

<div class='fc-event event-${type.toLowerCase()}' data-event='{

    "title": "${load.course_title}",

    "section": "${load.section}",

    "prof": "${load.prof}",

    "type": "${type}",

    "duration": "${formattedDuration}",

```

```

    "extendedProps": {

        "section": "${load.section}",

        "prof": "${load.prof}",

        "type": "${type}",

        "duration": ${load.duration * 60}

    }

}'>

<div class="event-title">${load.course_title}</div>

<div class="event-details">

    <div class="event-section">Section: ${load.section}</div>

    <div class="event-prof">Professor: ${load.prof}</div>

    <div class="event-type">Type: ${type === 'Lec' ? 'Lecture' : 'Laboratory'}</div>

    <div class="event-duration">Duration: ${formattedDuration}</div>

</div>

</div>`;

}

```

```

function createSplitEventHtml(load, type, durationMinutes, splitReference) {

    const hours = Math.floor(durationMinutes / 60);

    const minutes = durationMinutes % 60;

    const formattedDuration = `${hours}:${minutes < 10 ? '0' : ''}${minutes}`;

```

```
return `
```

```
<div class='fc-event event-split event-${type.toLowerCase()}'
```

```
  style='background-color: #346292;' data-event='{
```

```
    "title": "${load.course_title}",
```

```
    "section": "${load.section}",
```

```
    "prof": "${load.prof}",
```

```
    "type": "${type}",
```

```
    "duration": "${formattedDuration}",
```

```
    "is_split": 1,
```

```
    "split_reference": "${splitReference}",
```

```
    "extendedProps": {
```

```
      "section": "${load.section}",
```

```
      "prof": "${load.prof}",
```

```
      "type": "${type}",
```

```
      "duration": ${durationMinutes}
```

```
    }
```

```
  }>
```

```
<div class="event-title">${load.course_title}</div>
```

```
<div class="event-details">
```

```
  <div class="event-section">Section: ${load.section}</div>
```

```

        <div class="event-prof">Professor: ${load.prof}</div>

        <div class="event-type">Type: ${type === 'Lec' ? 'Lecture' : 'Laboratory'}</div>

        <div class="event-duration">Duration: ${formattedDuration}</div>

    </div>

</div>`;

}

```

```

$("#resetButton").on("click", function () {

    Swal.fire({

        title: "Are you sure?",

        text: "This will reset all schedules.",

        icon: "warning",

        showCancelButton: true,

        confirmButtonColor: "#6D86A1",

        confirmButtonText: "Yes",

        cancelButtonText: "Cancel",

    }).then((result) => {

        if (result.isConfirmed) {

            $.ajax({

                url: "../shared/scripts/reset-schedules.php",

                type: "POST",

```

```

success: function (response) {

    const res = JSON.parse(response);

    if (res.status === "success") {

        calendar.removeAllEvents();

        // Trigger the section change event

        $('#section').trigger('change');

        Swal.fire({

            title: "Reset!",

            text: res.message,

            icon: "success",

            confirmButtonColor: "#6D86A1",

        });

    } else {

        Swal.fire({

            title: "Error!",

            text: res.message,

            icon: "error",

            confirmButtonColor: "#6D86A1",

        });

    }
}

```

```

    },

    error: function () {

        Swal.fire({

            title: "Error!",

            text: "An error occurred while resetting schedules.",

            icon: "error",

            confirmButtonColor: "#6D86A1",

        });

    },

    });

}

});

});

}

});

data.php:

<?php

error_reporting(E_ALL);

ini_set('display_errors', 1);

require_once $_SERVER['DOCUMENT_ROOT'] . "/shared/database/connection.php";

```

```

class Data

{

    private $db;


    public function __construct()

    {

        $this->db = new Database();

    }


    public function gethaha()

    {

        $conn = $this->db->getConnection();

        $query = "SELECT u.id, u.first_name, u.middle_name, u.last_name, u.email, u.rankLevel

            FROM users u

            LEFT JOIN instructor_details d ON u.id = d.user_id";

        $result = $conn->query($query);

        $instructors = array();

        if ($result->num_rows > 0) {

            while ($row = $result->fetch_assoc()) {

                if (empty($row['code'])) {

```



```

        $row['code'] = strtoupper(substr($row['first_name'], 0, 1)

        . substr($row['middle_name'], 0, 1)

        . substr($row['last_name'], 0, 1));

    }

    $instructors[] = $row;

}

}

return $instructors;

}

```

```

public function fetchDataFromTable($tableName)

```

```

{

    // Establish database connection

    $conn = $this->db->getConnection();

    if (!$conn) {

        die(json_encode(['status' => 'error', 'message' => 'Failed to connect to the database']));

    }

    // Prepare the query to fetch all rows from the provided table

    $query = "SELECT * FROM " . $tableName;

```

```

// Prepare statement

$stmt = $conn->prepare($query);

if ($stmt === false) {

    die(json_encode(['status' => 'error', 'message' => 'Error preparing statement: ' . $conn-
>error]));

}


// Execute query

if (!$stmt->execute()) {

    die(json_encode(['status' => 'error', 'message' => 'Error executing query: ' . $stmt-
>error]));

}


// Get results

$result = $stmt->get_result();

if (!$result) {

    die(json_encode(['status' => 'error', 'message' => 'Error fetching results: ' . $stmt->error]));

}


// Fetch all rows from the result set

$data = $result->fetch_all(MYSQLI_ASSOC);

```

```

        // Close statement

        $stmt->close();

        return $data;
    }

    public function getUserById($userId)
    {

        $conn = $this->db->getConnection();

        $query = "SELECT * FROM users WHERE id=?";

        $stmt = $conn->prepare($query);

        $stmt->bind_param("i", $userId);

        $stmt->execute();

        $result = $stmt->get_result();

        $user = $result->fetch_assoc();

        return $user;
    }

    public function addUser($userData)
    {

        $conn = $this->db->getConnection();

```

```

$checkQuery = "SELECT id FROM users WHERE email = ?";

$checkStmt = $conn->prepare($checkQuery);

$checkStmt->bind_param("s", $userData['email']);

$checkStmt->execute();

$checkStmt->store_result();

if ($checkStmt->num_rows > 0) {

    return null;

}

$query = "INSERT INTO users (first_name, middle_name, last_name, email, rankLevel)
VALUES (?, ?, ?, ?, ?)";

$stmt = $conn->prepare($query);

$stmt->bind_param("ssssi", $userData['first_name'], $userData['middle_name'],
$userData['last_name'], $userData['email'], $userData['rankLevel']);

$stmt->execute();

return $stmt->insert_id;

}

public function editUser($userId, $newUserData)

{

```

```

$conn = $this->db->getConnection();

$query = "UPDATE users SET first_name=?, middle_name=?, last_name=?, email=?,
rankLevel=? WHERE id=?";

$stmt = $conn->prepare($query);

$stmt->bind_param("ssssii", $newUserData['first_name'], $newUserData['middle_name'],
$newUserData['last_name'], $newUserData['email'], $newUserData['rankLevel'], $userId);

$stmt->execute();

}

```

```

public function archiveUser($userId, $role)

```

```

{

```

```

    $conn = $this->db->getConnection();

```

```

    $query = "SELECT * FROM users WHERE id = ?";

```

```

    $stmt = $conn->prepare($query);

```

```

    $stmt->bind_param("i", $userId);

```

```

    $stmt->execute();

```

```

    $result = $stmt->get_result();

```

```

    $user = $result->fetch_assoc();

```

```

    $archiveQuery = "INSERT INTO archive_users (first_name, middle_name, last_name,
email, rank_level)

```

```

VALUES (?, ?, ?, ?, ?)";

$archiveStmt = $conn->prepare($archiveQuery);

$archiveStmt->bind_param(

    "ssssi",

    $user['first_name'],

    $user['middle_name'],

    $user['last_name'],

    $user['email'],

    $role

);

$archiveStmt->execute();


$deleteQuery = "DELETE FROM users WHERE id=?";

$deleteStmt = $conn->prepare($deleteQuery);

$deleteStmt->bind_param("i", $userId);

$deleteStmt->execute();

}

public function getArchivedUsers()

{

    $conn = $this->db->getConnection();

```

```
$query = "SELECT id, first_name, middle_name, last_name, email, rank_level, archived_at  
FROM archive_users WHERE archived_at IS NOT NULL";
```

```
$result = $conn->query($query);
```

```
$archivedUsers = [];
```

```
while ($row = $result->fetch_assoc()) {
```

```
    $archivedUsers[] = $row;
```

```
}
```

```
return $archivedUsers;
```

```
}
```

```
public function getArchivedUserById($id)
```

```
{
```

```
    $conn = $this->db->getConnection();
```

```
    $query = "SELECT * FROM archive_users WHERE id = ?";
```

```
    $stmt = $conn->prepare($query);
```

```
    $stmt->bind_param("i", $id);
```

```
    $stmt->execute();
```

```
    $result = $stmt->get_result();
```

```
    if ($result->num_rows > 0) {
```

```

        return $result->fetch_assoc();

    } else {

        return null;

    }

}

public function restoreUserToUsersTable($archivedUser)

{

    $conn = $this->db->getConnection();

    $query = "INSERT INTO users (first_name, middle_name, last_name, email, rankLevel)

        VALUES (?, ?, ?, ?, ?)";

    $stmt = $conn->prepare($query);

    $stmt->bind_param(

        "ssssi",

        $archivedUser['first_name'],

        $archivedUser['middle_name'],

        $archivedUser['last_name'],

        $archivedUser['email'],

        $archivedUser['rank_level']

```



```
);
```

```
return $stmt->execute();
```

```
}
```

```
public function deleteArchivedUser($id)
```

```
{
```

```
    $conn = $this->db->getConnection();
```

```
    $query = "DELETE FROM archive_users WHERE id = ?";
```

```
    $stmt = $conn->prepare($query);
```

```
    $stmt->bind_param("i", $id);
```

```
    return $stmt->execute();
```

```
}
```

```
public function getArchivedPrograms()
```

```
{
```

```
    $conn = $this->db->getConnection();
```

```
    $query = "SELECT * FROM academic_programs WHERE is_archived = 1";
```

```
    $result = $conn->query($query);
```

```
    $archivedProgs = [];
```

```

while ($row = $result->fetch_assoc()) {

    $archivedProgs[] = $row;

}

return $archivedProgs;

}

public function restoreProgram($program_id)

{

    $conn = $this->db->getConnection();

    // Update the `is_archived` column to restore the program

    $query = "UPDATE academic_programs SET is_archived = 0 WHERE program_id = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("i", $program_id);

    return $stmt->execute();

}

public function deleteArchivedProgram($id)

{

    $conn = $this->db->getConnection();

```

```

$query = "DELETE FROM archives_program_table WHERE archive_id = ?";

$stmt = $conn->prepare($query);

$stmt->bind_param("i", $id);

return $stmt->execute();

}

public function getAcademicHead()

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM users WHERE rankLevel = 2 LIMIT 1"; // Only need to check
if one exists

    $stmt = $conn->prepare($query);

    $stmt->execute();

    $result = $stmt->get_result();

    return $result->fetch_assoc(); // Return the first match or null if not found

}

public function getProgramHeadCount()

{

    $conn = $this->db->getConnection();

```

```
$query = "SELECT COUNT(*) AS count FROM instructor_details WHERE position =  
'Program Head'";
```

```
$result = $conn->query($query);
```

```
$row = $result->fetch_assoc();
```

```
return $row['count'];
```

```
}
```

```
//Faculty
```

```
public function getInstructors()
```

```
{
```

```
$conn = $this->db->getConnection();
```

```
$query = "SELECT u.id, u.first_name, u.middle_name, u.last_name, u.email,
```

```
        d.code, d.contact_number, d.employment_type, d.department, u.rankLevel,
```

```
d.total_loads
```

```
        FROM users u
```

```
        LEFT JOIN instructor_details d ON u.id = d.user_id
```

```
        WHERE u.rankLevel IN (1,2,5)";
```

```
$result = $conn->query($query);
```

```
$instructors = array();
```

```
if ($result->num_rows > 0) {
```

```
    while ($row = $result->fetch_assoc()) {
```

```

        if (empty($row['code'])) {

            $row['code'] = strtoupper(substr($row['first_name'], 0, 1)

                . substr($row['middle_name'], 0, 1)

                . substr($row['last_name'], 0, 1));

        }

        $instructors[] = $row;

    }

}

return $instructors;

}

public function getInstructorIdByCode($code)

{

    $conn = $this->db->getConnection();

    $query = "SELECT user_id FROM instructor_details WHERE code = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $code);

    $stmt->execute(); // Execute the query

    $result = $stmt->get_result(); // Get the result set

    $row = $result->fetch_assoc(); // Fetch the row as an associative array

```

```

        return $row['user_id']; // Return the user_id
    }

    public function getInstructorDetailsById($id){

        $conn = $this->db->getConnection();

        $query = "SELECT email, last_name, rankLevel FROM users WHERE id = ?";

        $stmt = $conn->prepare($query);

        $stmt->bind_param("i", $id);

        $stmt->execute();

        $result = $stmt->get_result();

        if ($result->num_rows > 0) {

            return $result->fetch_assoc(); // Fetch email and last_name as an associative array

        } else {

            return null; // Return null if no results found

        }

    }
}

```

```

public function getInstructorsByDepartments($departments)

{

```

```

$conn = $this->db->getConnection();

$placeholders = implode(',', array_fill(0, count($departments), '?'));

$query = "SELECT * FROM instructor_details WHERE department IN ($placeholders)";

$stmt = $conn->prepare($query);

$stmt->bind_param(str_repeat('s', count($departments)), ...$departments);

$stmt->execute();

$result = $stmt->get_result();

$instructors = [];

while ($row = $result->fetch_assoc()) {

    $instructors[] = $row;

}

return $instructors;

}

public function getInstructorById($userId)

{

    $conn = $this->db->getConnection();

```

```

$query = "SELECT u.id, u.first_name, u.middle_name, u.last_name, u.email,

           d.code, d.contact_number, d.employment_type, d.department, u.rankLevel,

d.total_loads

FROM users u

LEFT JOIN instructor_details d ON u.id = d.user_id

WHERE u.id = ?";

$stmt = $conn->prepare($query);

$stmt->bind_param("i", $userId);

$stmt->execute();

$result = $stmt->get_result();

if ($result->num_rows > 0) {

    $row = $result->fetch_assoc();

    // Generate default code if not present

    if (empty($row['code'])) {

        $row['code'] = strtoupper(substr($row['first_name'], 0, 1)

        . substr($row['middle_name'], 0, 1)

        . substr($row['last_name'], 0, 1));

    }
}

```



```

        return $row;

    } else {

        throw new Exception("Instructor not found");

    }

}

public function updateInstructorDetail($userId, $column, $value)

{

    $conn = $this->db->getConnection();

    $checkQuery = "SELECT * FROM instructor_details WHERE user_id = ?";

    $stmt = $conn->prepare($checkQuery);

    $stmt->bind_param("i", $userId);

    $stmt->execute();

    $result = $stmt->get_result();

    if ($result->num_rows > 0) {

        $updateQuery = "UPDATE instructor_details SET {$column} = ? WHERE user_id = ?";

        $updateStmt = $conn->prepare($updateQuery);

        if (!$updateStmt) {

            die("Update prepare failed: " . $conn->error);

        }
    }
}

```

```

$updateStmt->bind_param("si", $value, $userId);

if (!$updateStmt->execute()) {

    die("Update execute failed: " . $updateStmt->error);

}

$updateStmt->close();

} else {

    $insertQuery = "INSERT INTO instructor_details (user_id, {$column}) VALUES (?, ?)";

    $insertStmt = $conn->prepare($insertQuery);

    if (!$insertStmt) {

        die("Insert prepare failed: " . $conn->error);

    }

    $insertStmt->bind_param("is", $userId, $value);

    if (!$insertStmt->execute()) {

        die("Insert execute failed: " . $insertStmt->error);

    }

    $insertStmt->close();

}

$stmt->close();

}

```

```

public function getInstructorPosition($userId)

{

    $conn = $this->db->getConnection();

    $query = "SELECT position FROM instructor_details WHERE user_id = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("i", $userId);

    $stmt->execute();

    $result = $stmt->get_result();

    if ($result->num_rows > 0) {

        $row = $result->fetch_assoc();

        return $row['position'];

    } else {

        return 2;

    }

}

public function getProfessorCodeById($id)

{

    $conn = $this->db->getConnection();

```

```

// SQL query to get professor code based on email

$query = "SELECT code FROM instructor_details WHERE user_id = ?";

// Prepare the statement

$stmt = $conn->prepare($query);

$stmt->bind_param("i", $id); // Bind email parameter


// Execute the statement

$stmt->execute();


// Get the result

$result = $stmt->get_result();


if ($result->num_rows > 0) {

    // Return the professor code if found

    $row = $result->fetch_assoc();

    return $row['code'];

} else {

    // Return false if no result found

    return false;

}

```

```
}
```

```
public function updateRankLevel($userId, $position)
```

```
{
```

```
    $conn = $this->db->getConnection();
```

```
    $query = "UPDATE users SET rankLevel = ? WHERE id = ?";
```

```
    $stmt = $conn->prepare($query);
```

```
    $stmt->bind_param("ii", $position, $userId);
```

```
    if (!$stmt->execute()) {
```

```
        throw new Exception('Failed to update rank level: ' . $stmt->error);
```

```
    }
```

```
}
```

```
public function addFacultyLoad($section, $curriculumCode, $yearTerm, $courseCode,  
$courseTitle, $lec, $lab, $total, $faculty)
```

```
{
```

```
    $conn = $this->db->getConnection();
```

```
    // Query to fetch faculty_id based on faculty code
```

```
    $facultyQuery = "SELECT id, total_loads FROM instructor_details WHERE code = ?";
```

```
    $facultyStmt = $conn->prepare($facultyQuery);
```

```

$facultyStmt->bind_param("s", $faculty);

$facultyStmt->execute();

$facultyResult = $facultyStmt->get_result();

if ($facultyRow = $facultyResult->fetch_assoc()) {

    $facultyId = $facultyRow['id'];

    $currentTotalLoads = $facultyRow['total_loads'];

} else {

    // Faculty not found, handle error or return false

    return false;

}

// Query to fetch section_id based on section

$sectionQuery = "SELECT section_id FROM sections WHERE section = ?";

$sectionStmt = $conn->prepare($sectionQuery);

$sectionStmt->bind_param("s", $section);

$sectionStmt->execute();

$sectionResult = $sectionStmt->get_result();

if ($sectionRow = $sectionResult->fetch_assoc()) {

    $sectionId = $sectionRow['section_id'];

} else {

    // Section not found, handle error or return false

```

```

        return false;

    }

    // Insert faculty load with section_id and faculty_id

    $insertQuery = "INSERT INTO faculty_loads (section, curriculum_code, year_term,
course_code, course_title, lec, lab, total, section_id, faculty_id, faculty) VALUES (?, ?, ?, ?, ?, ?,
?, ?, ?, ?, ?)";

    $insertStmt = $conn->prepare($insertQuery);

    $insertStmt->bind_param("ssssdddis", $section, $curriculumCode, $yearTerm,
$courseCode, $courseTitle, $lec, $lab, $total, $sectionId, $facultyId, $faculty);

    $insertStmt->execute();

    if ($insertStmt->error) {

        error_log("Insert Error: " . $insertStmt->error);

        return false;

    }

    // Update the total_loads for the faculty after assignment

    $newTotalLoads = $currentTotalLoads + $total;

    $updateQuery = "UPDATE instructor_details SET total_loads = ? WHERE id = ?";

    $updateStmt = $conn->prepare($updateQuery);

    $updateStmt->bind_param("di", $newTotalLoads, $facultyId);

```

```
$updateStmt->execute();
```

```
if ($updateStmt->error) {
```

```
    error_log("Update Error: " . $updateStmt->error);
```

```
    return false;
```

```
}
```

```
return true;
```

```
}
```

```
public function removeFacultyLoad($id, $section)
```

```
{
```

```
    $conn = $this->db->getConnection();
```

```
    $query = "SELECT total, faculty_id FROM faculty_loads WHERE faculty_loads_id = ?  
AND section = ?";
```

```
    $stmt = $conn->prepare($query);
```

```
    $stmt->bind_param("ss", $id, $section);
```

```
    $stmt->execute();
```

```
    $result = $stmt->get_result();
```

```
    if ($row = $result->fetch_assoc()) {
```



```

$totalLoad = $row['total'];

$facultyId = $row['faculty_id'];

$deleteLoadQuery = "DELETE FROM faculty_loads WHERE faculty_loads_id = ? AND
section = ?";

$deleteLoadStmt = $conn->prepare($deleteLoadQuery);

$deleteLoadStmt->bind_param("ss", $id, $section);

$deleteLoadStmt->execute();

if ($deleteLoadStmt->affected_rows > 0) {

    $updateInstructorQuery = "UPDATE instructor_details SET total_loads = total_loads -
? WHERE id = ?";

    $updateInstructorStmt = $conn->prepare($updateInstructorQuery);

    $updateInstructorStmt->bind_param("di", $totalLoad, $facultyId);

    $updateInstructorStmt->execute();

    $deleteSchedulesQuery = "DELETE FROM schedules WHERE professor = ? AND
section = ?";

    $deleteSchedulesStmt = $conn->prepare($deleteSchedulesQuery);

    $deleteSchedulesStmt->bind_param("ss", $facultyId, $section);

    $deleteSchedulesStmt->execute();

```

```

        if ($deleteSchedulesStmt->error) {

            return false;

        }

        return true;

    }

}

return false;

}

public function removeFacultyLoadByCurriculumCode($programCode, $year)

{

    $conn = $this->db->getConnection();

    // Step 1: Concatenate the program code and year to form the curriculum_code

    $curriculumCode = $programCode . '-' . $year;

    // Step 2: Find faculty loads that are associated with this curriculum_code

    $query = "SELECT total, faculty_id FROM faculty_loads WHERE curriculum_code = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $curriculumCode);

```

```

$stmt->execute();

$result = $stmt->get_result();

// Step 3: Loop through each faculty load and remove it

while ($row = $result->fetch_assoc()) {

    $totalLoad = $row['total'];

    $facultyId = $row['faculty_id'];

    // Fetch the faculty code from the instructor_details table

    $facultyQuery = "SELECT code FROM instructor_details WHERE id = ?";

    $facultyStmt = $conn->prepare($facultyQuery);

    $facultyStmt->bind_param("i", $facultyId);

    $facultyStmt->execute();

    $facultyResult = $facultyStmt->get_result();

    // Check if a valid faculty code was returned

    if ($facultyResult->num_rows > 0) {

        $facultyRow = $facultyResult->fetch_assoc();

        $faculty = $facultyRow['code'];

    } else {

        // Handle the case where the faculty code is not found (e.g., error or skip)
    }
}

```

```

        continue; // or return false if needed

    }

    // Step 4: Delete the faculty load from faculty_loads

    $deleteLoadQuery = "DELETE FROM faculty_loads WHERE curriculum_code = ? AND
faculty_id = ?";

    $deleteLoadStmt = $conn->prepare($deleteLoadQuery);

    $deleteLoadStmt->bind_param("si", $curriculumCode, $facultyId);

    $deleteLoadStmt->execute();

    // Step 5: If the delete was successful, update the instructor's total loads

    if ($deleteLoadStmt->affected_rows > 0) {

        $updateInstructorQuery = "UPDATE instructor_details SET total_loads = total_loads -
? WHERE id = ?";

        $updateInstructorStmt = $conn->prepare($updateInstructorQuery);

        $updateInstructorStmt->bind_param("di", $totalLoad, $facultyId);

        $updateInstructorStmt->execute();

        // Step 6: Optionally, delete related schedules for the professor and section

        $deleteSchedulesQuery = "DELETE FROM schedules WHERE professor = ?";

        $deleteSchedulesStmt = $conn->prepare($deleteSchedulesQuery);

        $deleteSchedulesStmt->bind_param("s", $faculty);
    }

```

```

        $deleteSchedulesStmt->execute();

        if ($deleteSchedulesStmt->error) {

            return false;

        }

    }

}

// If no faculty loads were found for this curriculum_code, return false

return true;

}

```

```

public function getFacultyLoadDetails($curriculumCode, $section)

{

    $conn = $this->db->getConnection();

    $query = "

        SELECT

            c.curriculum_code,

            c.course_code,

            c.course_title,

            c.lect AS curriculum_lect,

            c.lab AS curriculum_lab,

```

```

        c.total AS curriculum_total,

        fl.faculty_loads_id,

        fl.section,

        c.year_term,

        fl.faculty

FROM curriculums c

LEFT JOIN faculty_loads fl

    ON c.curriculum_code = fl.curriculum_code

    AND c.course_code = fl.course_code

    AND fl.section = ? -- Match the section explicitly

WHERE c.curriculum_code = ?;

";

$stmt = $conn->prepare($query);

$stmt->bind_param("ss", $section, $curriculumCode); // Bind both section and
curriculum_code

$stmt->execute();

$result = $stmt->get_result();

$facultyLoadDetails = array();

if ($result->num_rows > 0) {

    while ($row = $result->fetch_assoc()) {

```

```

        $facultyLoadDetails[] = $row;

    }

}

return $facultyLoadDetails;

}

public function getFacultyLoadsBySection($section)

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM faculty_loads

        WHERE section = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $section);

    $stmt->execute();

    $result = $stmt->get_result();

    $facultyLoads = array();

    if ($result->num_rows > 0) {

        while ($row = $result->fetch_assoc()) {

            $facultyLoads[] = $row;

```

```

    }

}

return $facultyLoads;

}

//Curriculum

public function getProgramByCode($programCode)

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM academic_programs WHERE program_code = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $programCode);

    $stmt->execute();

    $result = $stmt->get_result();

    return $result->fetch_assoc();

}

public function addProgram($programData)

{

```



```

$conn = $this->db->getConnection();

$query = "INSERT INTO academic_programs (program_code, program_name, year)
VALUES (?, ?, ?)";

$stmt = $conn->prepare($query);

$stmt->bind_param("ssi", $programData['program_code'],
$programData['program_name'], $programData['year']);

$stmt->execute();

return $stmt->insert_id;

}

```

```

public function getPrograms()
{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM academic_programs WHERE is_archived = 0";

    $result = $conn->query($query);

    $programs = array();

    if ($result->num_rows > 0) {

        while ($row = $result->fetch_assoc()) {

            $programs[] = $row;

        }

    }

}

```

```

        return $programs;

    }

    public function archiveProgram($programId)

    {

        $conn = $this->db->getConnection();

        // Fetch the program details to archive

        $query = "SELECT program_id, program_name, program_code, year FROM
academic_programs WHERE program_id = ?";

        $stmt = $conn->prepare($query);

        $stmt->bind_param("i", $programId);

        $stmt->execute();

        $result = $stmt->get_result();

        $program = $result->fetch_assoc();

        if ($program) {

            // Archive the program by setting is_archived to 1

            $updateQuery = "UPDATE academic_programs SET is_archived = 1 WHERE
program_id = ?";

            $updateStmt = $conn->prepare($updateQuery);

            $updateStmt->bind_param("i", $programId);

```

```

$updateStmt->execute();

$programCode = $program['program_code'];

$year = $program['year'];


// Call removeFacultyLoadByCurriculumCode method

$facultyLoadRemoved = $this-
>removeFacultyLoadByCurriculumCode($programCode, $year);


// If faculty loads were successfully removed, proceed with archiving

if ($facultyLoadRemoved) {

    return true;

} else {

    // If faculty loads were not successfully removed, return false

    return false;

}

}

return false;

}


public function addCurriculum($curriculumData)

{

```

```

$conn = $this->db->getConnection();

$query = "INSERT INTO curriculums (program_id, curriculum_code, year_term,
course_code, course_title, lec, lab, total) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";

$stmt = $conn->prepare($query);

$stmt->bind_param(

    "isssddd",

    $curriculumData['program_id'],

    $curriculumData['curriculum_code'],

    $curriculumData['year_term'],

    $curriculumData['course_code'],

    $curriculumData['course_title'],

    $curriculumData['lec'],

    $curriculumData['lab'],

    $curriculumData['total']

);

$stmt->execute();

return $stmt->insert_id;

}

public function deleteCurriculum($curriculumId)

{

```

```

$conn = $this->db->getConnection();

$query = "DELETE FROM curriculums WHERE curriculum_id=?";

$stmt = $conn->prepare($query);

$stmt->bind_param("s", $curriculumId);

return $stmt->execute();

}

```

```

public function getCurriculumCodes()

{

    $conn = $this->db->getConnection();

    $query = "SELECT curriculum_code FROM curriculums";

    $stmt = $conn->prepare($query);

    $stmt->execute();

    $result = $stmt->get_result();

    $curriculumCodes = array();

    if ($result->num_rows > 0) {

        while ($row = $result->fetch_assoc()) {

            $curriculumCodes[] = $row;

        }

    }

}

```

```

        return $curriculumCodes;

    }

    public function getCurriculumsByCode($curriculumCode)

    {

        $conn = $this->db->getConnection();

        $query = "SELECT * FROM curriculums WHERE curriculum_code = ?";

        $stmt = $conn->prepare($query);

        $stmt->bind_param("s", $curriculumCode);

        $stmt->execute();

        $result = $stmt->get_result();

        $curriculums = array();

        if ($result->num_rows > 0) {

            while ($row = $result->fetch_assoc()) {

                $curriculums[] = $row;

            }

        }

        return $curriculums;

    }

```

```
//Maintenance
```

```
public function addSection($sectionName, $yearTerm, $programCode)
```

```
{
```

```
    $conn = $this->db->getConnection();
```

```
    $query = "INSERT INTO sections (section, yearTerm, program) VALUES (?, ?, ?)";
```

```
    $stmt = $conn->prepare($query);
```

```
    $stmt->bind_param("sss", $sectionName, $yearTerm, $programCode);
```

```
    $stmt->execute();
```

```
    return $stmt->insert_id;
```

```
}
```

```
public function getSections()
```

```
{
```

```
    $conn = $this->db->getConnection();
```

```
    $query = "SELECT * FROM sections";
```

```
    $result = $conn->query($query);
```

```
    $sections = array();
```

```
    if ($result->num_rows > 0) {
```

```
        while ($row = $result->fetch_assoc()) {
```

```
            $sections[] = $row;
```

```

    }

}

return $sections;

}

public function getSectionsByProgram($programCode)

{

    $conn = $this->db->getConnection();

    $query = "SELECT section, yearTerm FROM sections WHERE program = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $programCode);

    $stmt->execute();

    $result = $stmt->get_result();

    $sections = array();

    if ($result->num_rows > 0) {

        while ($row = $result->fetch_assoc()) {

            $sections[] = $row;

        }

    }

    $stmt->close();

```



```

        return $sections;

    }

    public function getSectionsByProgramCode($program)

    {

        $conn = $this->db->getConnection();

        $query = "SELECT * FROM sections WHERE program = ?";

        $stmt = $conn->prepare($query);

        $stmt->bind_param("s", $program); // Bind the program parameter

        $stmt->execute();

        $result = $stmt->get_result();

        $sections = array();

        while ($row = $result->fetch_assoc()) {

            $sections[] = $row;

        }

        return $sections;

    }

```

```

public function deleteSection($sectionId)

{

    $conn = $this->db->getConnection();


    $conn->begin_transaction();


    try {

        // Fetch the section name using sectionId

        $query = "SELECT section FROM sections WHERE section_id = ?";

        $stmt = $conn->prepare($query);

        $stmt->bind_param("i", $sectionId);

        $stmt->execute();

        $stmt->store_result();


        // Check if section exists

        if ($stmt->num_rows === 0) {

            $stmt->close();

            throw new Exception("Section not found.");

        }


        $stmt->bind_result($sectionName);
    }

```

```

$stmt->fetch();

// Debugging - Output section name

error_log("Deleting Section: " . $sectionName);

// Fetch faculty loads before deleting section

$queryGetFacultyLoad = "SELECT faculty_id, total FROM faculty_loads WHERE
section = ?";

$stmt = $conn->prepare($queryGetFacultyLoad);

$stmt->bind_param("s", $sectionName);

$stmt->execute();

$stmt->store_result();

if ($stmt->num_rows > 0) {

    $stmt->bind_result($facultyId, $facultyLoad);

    while ($stmt->fetch()) {

        // First, delete faculty loads related to the section

        $queryDeleteFaculty = "DELETE FROM faculty_loads WHERE section = ?";

        $deleteStmt = $conn->prepare($queryDeleteFaculty);

        $deleteStmt->bind_param("s", $sectionName);

        if (!$deleteStmt->execute()) {

```

```

        error_log("Error deleting faculty loads for section: " . $sectionName);

        throw new Exception("Error deleting faculty loads.");

    }

    // Then subtract the faculty load from the instructor's details

    $queryUpdateInstructor = "UPDATE instructor_details SET total_loads =
total_loads - ? WHERE id = ?";

    $updateStmt = $conn->prepare($queryUpdateInstructor);

    $updateStmt->bind_param("di", $facultyLoad, $facultyId);

    if (!$updateStmt->execute()) {

        error_log("Error updating instructor load for faculty_id: " . $facultyId);

        throw new Exception("Error updating instructor load.");

    }

}

} else {

    error_log("No faculty loads found for section: " . $sectionName);

}

// Delete schedules related to the section

$queryDeleteSchedules = "DELETE FROM schedules WHERE section = ?";

$stmt = $conn->prepare($queryDeleteSchedules);

```

```

$stmt->bind_param("s", $sectionName);

if (!$stmt->execute()) {

    error_log("Error deleting schedules for section: " . $sectionName);

    throw new Exception("Error deleting schedules.");

}

// Delete section itself

$queryDeleteSection = "DELETE FROM sections WHERE section_id = ?";

$stmt = $conn->prepare($queryDeleteSection);

$stmt->bind_param("i", $sectionId);

if (!$stmt->execute()) {

    error_log("Error deleting section with section_id: " . $sectionId);

    throw new Exception("Section deletion failed.");

}

// Commit transaction if no errors

$conn->commit();

return true;

} catch (Exception $e) {

    // Rollback and log error

    $conn->rollback();

```

```

        error_log("Error deleting section: " . $e->getMessage());

        return false;

    }

}

```

```

public function checkSection($room)

```

```

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM sections WHERE section=?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $room);

    $stmt->execute();

    $result = $stmt->get_result();

    $sectionName = $result->fetch_assoc();

    return $sectionName;

}

```

```

public function getDepartmentByName($departmentName)

```

```

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM faculty_departments WHERE department_name = ?";

```

```

$stmt = $conn->prepare($query);

$stmt->bind_param("s", $departmentName);

$stmt->execute();

$result = $stmt->get_result();

return $result->fetch_assoc();

}

public function addDepartment($departmentName)

{

    $conn = $this->db->getConnection();

    $query = "INSERT INTO faculty_departments (department_name) VALUES (?)";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $departmentName);

    $stmt->execute();

    return $stmt->insert_id;

}

public function getDepartments()

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM faculty_departments";

```

```

$result = $conn->query($query);

$departments = array();

if ($result->num_rows > 0) {

    while ($row = $result->fetch_assoc()) {

        $departments[] = $row;

    }

}

return $departments;

}

public function addRoom($roomName)

{

    $conn = $this->db->getConnection();

    $query = "INSERT INTO rooms (room_number) VALUES (?)";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $roomName);

    $stmt->execute();

    return $stmt->insert_id;

}

```



```

public function getRooms()

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM rooms";

    $result = $conn->query($query);

    $rooms = array();

    if ($result->num_rows > 0) {

        while ($row = $result->fetch_assoc()) {

            $rooms[] = $row;

        }

    }

    return $rooms;

}

```

```

public function deleteRoom($roomId)

{

    $conn = $this->db->getConnection();

    // Start a transaction to ensure atomicity

    $conn->begin_transaction();

    try {

```

```

// Delete schedules linked to the room

$deleteSchedulesQuery = "DELETE FROM schedules WHERE room_id=?";

$stmtSchedules = $conn->prepare($deleteSchedulesQuery);

$stmtSchedules->bind_param("i", $roomId);

$stmtSchedules->execute();


// Delete the room

$deleteRoomQuery = "DELETE FROM rooms WHERE room_id=?";

$stmtRoom = $conn->prepare($deleteRoomQuery);

$stmtRoom->bind_param("i", $roomId);

$stmtRoom->execute();


// Commit the transaction

$conn->commit();

return true;

} catch (Exception $e) {

// Roll back the transaction in case of an error

$conn->rollback();

return false;

}

}

```

```

public function getRoomByNumber($room)

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM rooms WHERE room_number=?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $room);

    $stmt->execute();

    $result = $stmt->get_result();

    $roomNum = $result->fetch_assoc();

    return $roomNum;

}

```

```

public function saveOperationHours($startTime, $endTime)

{

    $conn = $this->db->getConnection();

    $query = "UPDATE operation_hours SET start_time = ?, end_time = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("ss", $startTime, $endTime);

    $stmt->execute();

    $stmt->close();

```

```

        return true;

    }

    public function getOperationHours()

    {

        $conn = $this->db->getConnection();

        $query = "SELECT start_time, end_time FROM operation_hours LIMIT 1";

        $result = $conn->query($query);

        if ($result) {

            $operationHours = $result->fetch_assoc();

            $conn->close();

            return $operationHours;

        } else {

            return false;

        }

    }

```

```

    public function getAllSchedules($room)

```

```

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM schedules WHERE room = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $room);

    $stmt->execute();

    $result = $stmt->get_result();

    $schedules = $result->fetch_all(MYSQLI_ASSOC);

    $stmt->close();

    return $schedules;

}

```

public function **getAllScheduledSubjects**()

```

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM schedules"; // Adjust column name if needed

    $stmt = $conn->prepare($query);

    $stmt->execute();

    $result = $stmt->get_result();

    $schedules = $result->fetch_all(MYSQLI_ASSOC);

    $stmt->close();

```

```
        return $schedules; // Returns an array of scheduled subjects
    }
}
```

```
public function getSchedRooms()
```

```
{
```

```
    $conn = $this->db->getConnection();
```

```
    $query = "SELECT
```

```
        schedules.id,
```

```
        schedules.subject_title,
```

```
        schedules.start_time,
```

```
        schedules.end_time,
```

```
        schedules.room_id,
```

```
        rooms.room_number, -- Include room number
```

```
        schedules.section,
```

```
        schedules.professor,
```

```
        schedules.days_of_week,
```

```
        schedules.type,
```

```
        schedules.duration,
```

```
        schedules.is_split,
```

```
        schedules.split_reference
```

```

        FROM

        schedules

    INNER JOIN

        rooms ON schedules.room_id = rooms.room_id;

    "; // Adjust column name if needed

    $stmt = $conn->prepare($query);

    $stmt->execute();

    $result = $stmt->get_result();

    $schedules = $result->fetch_all(MYSQLI_ASSOC);

    $stmt->close();

    return $schedules; // Returns an array of scheduled subjects
}

public function getSchedulesByRoom($room_id)

{

    $conn = $this->db->getConnection();

    $query = "

        SELECT

        id,

        subject_title AS title,

```

```

start_time,

end_time,

section,

professor AS prof,

days_of_week,

CASE

    WHEN type = 'lec' THEN 'Lec'

    WHEN type = 'lab' THEN 'Lab'

    ELSE NULL

END AS type,

duration, -- Fetch duration directly from the table

is_split, -- Fetch split status

split_reference -- Fetch split reference if applicable

FROM

    schedules

WHERE

    room_id = ?";

$stmt = $conn->prepare($query);

$stmt->bind_param("i", $room_id);

$stmt->execute();

```



```

$result = $stmt->get_result();

$events = [];

while ($row = $result->fetch_assoc()) {

    $events[] = [

        'id' => $row['id'],

        'title' => $row['title'],

        'start_time' => $row['start_time'],

        'end_time' => $row['end_time'],

        'section' => $row['section'],

        'prof' => $row['prof'],

        'days_of_week' => $row['days_of_week'],

        'type' => $row['type'],

        'duration' => $row['duration'],

        'is_split' => $row['is_split'], // Include split status

        'split_reference' => $row['split_reference'] // Include split reference if applicable

    ];

}

return $events;

}

```

```

public function getSchedulesByProfessor($professorCode)

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM schedules WHERE professor = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $professorCode);

    $stmt->execute();

    $result = $stmt->get_result();

    $schedules = [];

    while ($row = $result->fetch_assoc()) {

        $schedules[] = $row;

    }

    if (count($schedules) > 0) {

        return $schedules;
    }
}

```

```

    } else {

        return false;

    }

}

public function getSchedulesBySection($section)

{

    $conn = $this->db->getConnection();

    $query = "SELECT * FROM schedules WHERE section = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $section);

    $stmt->execute();

    $result = $stmt->get_result();

    $schedules = [];

    while ($row = $result->fetch_assoc()) {

        $schedules[] = $row;
    }
}

```

```

    }

    if (count($schedules) > 0) {

        return $schedules;

    } else {

        return false;

    }

}

```

```

public function insertSchedule($subject_title, $start_time, $end_time, $room, $section,
$professor, $days_of_week, $type, $duration, $is_split = 0, $split_reference = null)
{

    $conn = $this->db->getConnection();

    $query = "INSERT INTO schedules (subject_title, start_time, end_time, room_id, section,
professor, days_of_week, type, duration, is_split, split_reference) VALUES (?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?)";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("sssssssis", $subject_title, $start_time, $end_time, $room, $section,
$professor, $days_of_week, $type, $duration, $is_split, $split_reference);

    $result = $stmt->execute();

    $insertedId = $stmt->insert_id;

    $stmt->close();

```

```

        return $result ? $insertedId : false;

    }

    public function updateSchedule($id, $start, $end, $daysOfWeek)

    {

        $conn = $this->db->getConnection();

        $query = "UPDATE schedules SET start_time = ?, end_time = ?, days_of_week = ?
WHERE id = ?";

        $stmt = $conn->prepare($query);

        $stmt->bind_param("sssi", $start, $end, $daysOfWeek, $id);

        $result = $stmt->execute();

        $stmt->close();

        return $result;

    }

    public function deleteSchedule($id)

    {

        $conn = $this->db->getConnection();

        $query = "DELETE FROM schedules WHERE id = ?";

        $stmt = $conn->prepare($query);

```

```

$stmt->bind_param("i", $id);

$result = $stmt->execute();

$stmt->close();

return $result;

}

public function getAllSections()

{

    $conn = $this->db->getConnection(); // Assuming a DB connection method exists

    $query = "SELECT DISTINCT section FROM schedules";

    $stmt = $conn->prepare($query);

    $stmt->execute();

    $result = $stmt->get_result();

    $sections = [];

    while ($row = $result->fetch_assoc()) {

        $sections[] = $row['section']; // Add each section to the array

    }

    return $sections;

}

```

```

// Fetch schedules for a specific section, including professor and room details

public function getSchedBySection($section)

{

    $conn = $this->db->getConnection();

    // Query to join schedules with rooms to get room details and professor info

    $query = "SELECT s.subject_title, s.professor, s.days_of_week, s.start_time, s.end_time,
s.type, s.duration, r.room_number FROM schedules s LEFT JOIN rooms r ON s.room_id =
r.room_id WHERE s.section = ? ORDER BY s.subject_title ASC, s.start_time;

";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("s", $section); // Bind section parameter

    $stmt->execute();

    $result = $stmt->get_result();

    $schedules = [];

    while ($row = $result->fetch_assoc()) {

        $schedules[] = $row; // Add each schedule data to the array

    }

```

```

        return $schedules;

    }

    public function getAllInstructorCodes()

    {

        $query = "

            SELECT DISTINCT code

            FROM instructor_details

            ORDER BY code ASC

        ";

        $stmt = $this->db->getConnection()->prepare($query);

        if (!$stmt) {

            die('Prepare failed: ' . $this->db->getConnection()->error);

        }

        $stmt->execute();

        $result = $stmt->get_result();

        $instructors = [];

        while ($row = $result->fetch_assoc()) {

```



```

        $instructors[] = $row;

    }

    $stmt->close();

    return $instructors;

}

public function getSchedulesByInstructor($instructorCode)

{

    $query = "

        SELECT

            s.section,

            s.subject_title,

            s.days_of_week,

            s.start_time,

            s.end_time,

            s.type,

            s.duration,

            r.room_number

        FROM schedules s

        LEFT JOIN rooms r ON s.room_id = r.room_id

```

WHERE s.professor = ?

ORDER BY s.section, s.start_time

";

\$stmt = \$this->db->getConnection()->prepare(\$query);

if (!\$stmt) {

die('Prepare failed: ' . \$this->db->getConnection()->error);

}

\$stmt->bind_param('s', \$instructorCode);

\$stmt->execute();

\$result = \$stmt->get_result();

\$schedules = [];

while (\$row = \$result->fetch_assoc()) {

\$schedules[] = \$row;

}

\$stmt->close();

return \$schedules;

}

```

public function getAllAdmins()

{

    $conn = $this->db->getConnection();

    $query = "SELECT id, username FROM admin_accounts";

    $stmt = $conn->prepare($query);

    $stmt->execute();

    $result = $stmt->get_result();

    $admins = [];

    while ($row = $result->fetch_assoc()) {

        $admins[] = [

            'id' => $row['id'],

            'username' => $row['username']

        ];

    }

    return $admins;

}

public function editAdmin($adminId, $adminData)

```

```

{

$conn = $this->db->getConnection(); // Get the database connection


// Prepare the SQL query for updating admin information

$query = "UPDATE admin_accounts SET username = ?";


// Add password to query if provided

if (isset($adminData['password'])) {

    $query .= ", password = ?";

}


$query .= " WHERE id = ?";


$stmt = $conn->prepare($query);


// Bind parameters to the query

if (isset($adminData['password'])) {

    // Bind username, password, and admin ID

    $stmt->bind_param("ssi", $adminData['username'], $adminData['password'], $adminId);

} else {

    // Bind only username and admin ID

    $stmt->bind_param("si", $adminData['username'], $adminId);

```

```

    }

    // Execute the query

    if ($stmt->execute()) {

        return true;

    } else {

        return false;

    }

}

}

public function updateUserContactNumber($userId, $contactNumber)

{

    $conn = $this->db->getConnection(); // Get the database connection

    $query = "UPDATE instructor_details SET contact_number = ? WHERE user_id = ?";

    $stmt = $conn->prepare($query);

    $stmt->bind_param("si", $contactNumber, $userId);

    return $stmt->execute(); // Returns true if the update was successful

}

```

```

public function logActivity($userId, $actionType, $actionDescription, $timestamp = null)

```

STI College Alabang

```

{

$conn = $this->db->getConnection();

if ($timestamp === null) {

    // If no timestamp is provided, use the current time

    $timestamp = date('Y-m-d H:i:s');

}

$query = "INSERT INTO activity_logs (user_id, action_type, action_description,
timestamp) VALUES (?, ?, ?, ?)";

$stmt = $conn->prepare($query);

$stmt->bind_param("iss", $userId, $actionType, $actionDescription, $timestamp);

return $stmt->execute(); // Returns true if the log was inserted successfully

}

// In the Data class

public function getRecentActivityLogs()

{

    $conn = $this->db->getConnection();

    // Adjust the query to fetch the instructor's `code` from `instructor_details`

```

```

$sql = "

SELECT

    al.action_type,

    al.action_description,

    al.timestamp,

    id.code AS instructor_code

FROM activity_logs al

LEFT JOIN instructor_details id ON al.user_id = id.user_id

ORDER BY al.timestamp DESC

LIMIT 4

";

$result = $conn->query($sql);

$log = [];

while ($row = $result->fetch_assoc()) {

    // Concatenate the instructor code into the action description

    $instructorDetails = "";

    if (!empty($row['instructor_code'])) {

        $instructorDetails = " [Faculty: {".$row['instructor_code'].}]";

    }

}

```

```

$logs[] = [

    'action_type' => $row['action_type'],

    'action_description' => $row['action_description'] . $instructorDetails,

    'timestamp' => $row['timestamp'],

];

}

return $logs;

}

```

```

public function resetSchedules()

```

```

{

    $conn = $this->db->getConnection(); // Get the database connection

    try {

        $conn->begin_transaction();

        $resetSchedulesQuery = "DELETE FROM schedules";

        $stmt = $conn->prepare($resetSchedulesQuery);

        $stmt->execute();
    }
}

```



```

        $conn->commit();

        return true;

    } catch (Exception $e) {

        $conn->rollback();

        return false;

    }

}

}

}

facultyLoading.js:

$(document).ready(function () {

    let selectedCurriculumCode;

    let programCode;

    const table = $('#facultyTable').DataTable({

        "searching": true,

        "bInfo": false,

        "responsive": true,

        "bLengthChange": false,

        "ajax": {

            "url": "",

            "dataSrc": function () {

```

```

        return [];

    }

},

"columns": [{

    "data": "section",

    "className": "text-center",

    "width": "10%",

    "orderable": true

},

{

    "data": "curriculum_code",

    "className": "text-center",

    "orderable": false,

    "width": "5%"

},

{

    "data": "year_term",

    "orderable": false,

    "className": "text-center",

    "width": "10%"

```

```

    },

    {

        "data": "course_code",

        "className": "text-center",

        "orderable": false,

        "width": "20%"

    },

    {

        "data": "course_title",

        "orderable": false,

        "width": "25%"

    },

    {

        "data": "curriculum_lec",

        "className": "text-center",

        "orderable": false,

        "width": "5%"

    },

    {

        "data": "curriculum_lab",

```

```

        "className": "text-center",

        "orderable": false,

        "width": "5%"

    },

    {

        "data": "curriculum_total",

        "className": "text-center",

        "orderable": false,

        "width": "5%"

    },

    {

        "data": "faculty",

        "className": "text-center",

        "orderable": false,

        "width": "5%",

        "render": function (data, type, row) {

            return data ? data : 'Unassigned';

        }

    },

    {

```

```

"data": null,

"width": "10%",

"orderable": false,

"render": function (data, type, row) {

    if (row.faculty && row.faculty !== 'Unassigned') {

        return `<button class="btn btn-danger btn-sm unassign-faculty w-100"

            data-id="${row.faculty_loads_id}"

            data-section="${row.section}"

            data-curriculum_code="${row.curriculum_code}"

            data-year_term="${row.year_term}"

            data-course_code="${row.course_code}"

            data-course_title="${row.course_title}"

            data-lec="${row.curriculum_lec}"

            data-lab="${row.curriculum_lab}"

            data-total="${row.curriculum_total}">

                Unassign

            </button>`;

    } else {

        return `<button class="custom-button p-1 assign-faculty w-100"

            data-id="${row.faculty_loads_id}"

```

```

data-section="{row.section}"

data-curriculum_code="{row.curriculum_code}"

data-year_term="{row.year_term}"

data-course_code="{row.course_code}"

data-course_title="{row.course_title}"

data-lec="{row.curriculum_lec}"

data-lab="{row.curriculum_lab}"

data-total="{row.curriculum_total}"

data-bs-toggle="modal"

data-bs-target="#facultyModal">

```

Assign

```

</button>`;

    }

}

}

]

});

$('#yearTermSelect').change(function () {

    let yearTerm = $(this).val();

```

```

if (yearTerm) {

    $('#yearTermSelect option[value="default"]').prop('disabled', true);

} else {

    $('#yearTermSelect option[value="default"]').prop('disabled', false);

}

$.fn.dataTable.ext.search.length = 0;

$.fn.dataTable.ext.search.push(

    function (settings, data, dataIndex) {

        let rowYearTerm = data[2];

        if (yearTerm && yearTerm !== "default") {

            return rowYearTerm === yearTerm;

        }

        return true;

    }

);

table.draw();

table.rows({

    search: 'applied'

}).every(function () {

```

```

        let rowData = this.data();

    });

});

// Fetch and populate program codes

$.ajax({

    url: '../shared/scripts/get-programs.php',

    dataType: 'json',

    success: function (response) {

        if (response.data) {

            let uniqueProgramCodes = new Set();

            response.data.forEach(function (program) {

                if (!uniqueProgramCodes.has(program.program_code)) {

                    uniqueProgramCodes.add(program.program_code);

                    $('#programCode').append(

                        '<option

value="{program.program_code}">{program.program_code}</option>'

                    );

                }

            });

        }
    }
});

```



```

    },

    error: function (xhr, status, error) {

        console.error(error);

        alertify.error('Failed to fetch program codes.');
```

}

```

    });

// Populate curriculum codes when a program code is selected

$('#programCode').change(function () {

    let selectedProgramCode = $(this).val();

    if (selectedProgramCode) {

        $('#curriculumCode').prop('disabled', false);

        $('#curriculumCode').empty().append(

            '<option value="" disabled selected>Select Curriculum Code</option>'

        );

        $.ajax({

            url: '../shared/scripts/get-curriculumCode.php',

            type: 'GET',

            data: {

                program_code: selectedProgramCode

            },
```

```

dataType: 'json',

success: function (response) {

    if (response && response.length > 0) {

        let uniqueCurriculumCodes = new Set();

        response.forEach(function (curriculum) {

            if (curriculum.curriculum_code.startsWith(selectedProgramCode) &&
!uniqueCurriculumCodes.has(curriculum.curriculum_code)) {

                uniqueCurriculumCodes.add(curriculum.curriculum_code);

                $('#curriculumCode').append(

                    '<option

value="{curriculum.curriculum_code}">{curriculum.curriculum_code}</option>'

                );

            }

        });

    } else {

        alertify.warning('No curriculum codes found for the selected program.');
```

```

    }

```

```

},

```

```

error: function (xhr, status, error) {

```

```

    console.error(error);

```

```

    alertify.error('Failed to fetch curriculum codes.');
```

```

        }

    });

} else {

    $('#curriculumCode').prop('disabled', true);

}

});

$('#curriculumCode').change(function () {

    selectedCurriculumCode = $(this).val();

    programCode = $("#programCode").val();

    if (selectedCurriculumCode && programCode) {

        table.ajax.url("../shared/scripts/get-section-by-program.php?program_code=" +
programCode).load();

        $.ajax({

            url: "../shared/scripts/get-section-by-program.php",

            type: "GET",

            data: {

                program_code: programCode

            },

            dataType: 'json',

```

```

success: function (sectionsResponse) {

    let tableData = [];

    if (sectionsResponse && sectionsResponse.data && sectionsResponse.data.length >
0) {

        let sectionRequests = [];

        sectionsResponse.data.forEach(function (section) {

            sectionRequests.push(

                $.ajax({

                    url: "../shared/scripts/get-faculty-loads.php",

                    type: "GET",

                    data: {

                        curriculum_code: selectedCurriculumCode,

                        section: section.section

                    },

                    dataType: 'json',

                    success: function (facultyResponse) {

                        if (facultyResponse && facultyResponse.data &&
facultyResponse.data.length > 0) {

                            facultyResponse.data.forEach(function (facultyLoad) {

```

```

        if (facultyLoad.year_term === section.yearTerm) {

            let existsInTableData = tableData.some(function (item) {

                return item.section === section.section && item.course_code

                === facultyLoad.course_code;

            });

            if (!existsInTableData) {

                tableData.push({

                    section: section.section,

                    curriculum_code: facultyLoad.curriculum_code,

                    year_term: facultyLoad.year_term,

                    course_code: facultyLoad.course_code,

                    course_title: facultyLoad.course_title,

                    curriculum_lec: facultyLoad.curriculum_lec,

                    curriculum_lab: facultyLoad.curriculum_lab,

                    curriculum_total: facultyLoad.curriculum_total,

                    faculty: facultyLoad.faculty ? facultyLoad.faculty :

                    'Unassigned',

                    faculty_loads_id: facultyLoad.faculty_loads_id

                });

            }

```

```

        }

    });

}

},

error: function (xhr, status, error) {

    console.error(error);

    alertify.error('Failed to fetch faculty loads for section ' +
section.section);

}

})

);

});

$.when.apply($, sectionRequests).then(function () {

    table.clear().rows.add(tableData).draw();

});

} else {

    alertify.error('No sections found for the selected program code.');
```

```

    }

},

error: function (xhr, status, error) {

    console.error(error);
```

```

        alertify.error('Failed to fetch sections.');
```

}

```

    });

}

});

$('#facultyTable').on('click', '.assign-faculty', function () {

    const button = $(this);

    const section = button.data('section');

    const curriculumCode = $('#curriculumCode').val();

    const yearTerm = button.data('year_term');

    const courseCode = button.data('course_code');

    const courseTitle = button.data('course_title');

    const lec = button.data('lec');

    const lab = button.data('lab');

    const total = button.data('total');

    $('#facultyModalLabel').text(`Assign Faculty in ${courseTitle}`);

    $.ajax({
```

```

url: '../shared/scripts/get-department.php', // Path to your script

type: 'GET',

success: function (response) {

    try {

        console.log("Response from get-departments.php:", response); // Log the raw
response

        const data = response.data ? response.data : JSON.parse(response).data;

        console.log("Departments:", data); // Log the departments array

        if (Array.isArray(data)) {

            let checkboxesHtml = `

                <label class="form-check form-check-inline">

                    <input class="form-check-input" type="checkbox"
id="selectAllDepartments">

                    <span class="form-check-label">All</span>

                </label>

            `;

            // Add department checkboxes

            data.forEach(department => {

```



```

checkboxesHtml += `

    <label class="form-check form-check-inline">

        <input class="form-check-input department-checkbox" type="checkbox"
value="${department.department_name}">

        <span class="form-check-label">${department.department_name}</span>

    </label>`;

});

$('#departmentCheckboxes').html(checkboxesHtml);

// Handle "All" checkbox toggle

$('#selectAllDepartments').on('change', function () {

    const isChecked = $(this).prop('checked');

    $('.department-checkbox').prop('checked', isChecked);

    console.log("All Departments Checked:", isChecked); // Add this line for
debugging

    const selectedDepartments = $('.department-checkbox:checked').map(function
() {

        return $(this).val();

    }).get();

    filterFacultyBasedOnDepartments(selectedDepartments); // Update faculty
list after selection change

```

```

});

// Handle individual department checkbox change

$('.department-checkbox').on('change', function () {

    const selectedDepartments = $('.department-checkbox:checked').map(function

() {

        return $(this).val();

    }).get();

    console.log("Selected Departments:", selectedDepartments); // Add this line for
debugging

// Uncheck "All" if any department is selected

if (selectedDepartments.length > 0) {

    $('#selectAllDepartments').prop('checked', false);

}

    filterFacultyBasedOnDepartments(selectedDepartments); // Update faculty
list based on selected departments

});

} else {

```

```

        console.error("Departments is not an array:", data);

        alertify.error("Failed to load departments.");

    }

    } catch (error) {

        console.error("Error parsing the response:", error);

        alertify.error("Failed to load departments due to an error.");

    }

},

error: function () {

    console.log("Failed to fetch departments.");

    alertify.error("Unable to load departments. Please try again.");

}

});

$('#facultyModal').modal('show'); // Show the modal

$('#assignFacultyButton').prop('disabled', true);

$('#facultySelect').on('change', function () {

    const faculty = $(this).val();

    if (faculty) {

        $('#assignFacultyButton').prop('disabled', false);

    } else {

```

```

        $('#assignFacultyButton').prop('disabled', true);

    }

});

function filterFacultyBasedOnDepartments(selectedDepartments = []) {

    if (selectedDepartments.length === 0) {

        console.log("No departments selected.");

        alertify.error("Please select at least one department.");

        return; // Exit the function if no departments are selected

    }

    console.log("Selected Departments:", selectedDepartments); // Log the selected
departments

    // AJAX request to get filtered faculty based on selected departments

    $.ajax({

        url: '../shared/scripts/get-instructors-by-department.php', // Path to the script that fetches
faculty

        type: 'GET',

        data: {

            departments: selectedDepartments

        },

```

```

success: function (response) {

    try {

        console.log("Faculty Response:", response); // Log the raw response


        const facultyData = response.data ? response.data : JSON.parse(response).data;


        // Clear current faculty options


        $('#facultySelect').html(<'<option value="" disabled selected>Select
Faculty</option>');


        if (Array.isArray(facultyData)) {

            // Add filtered faculty to the select dropdown


            facultyData.forEach(faculty => {

                // Assuming you have a way to get the instructor's total_loads and
employment_type for each faculty


                let maxLoad = (faculty.employment_type.toLowerCase() === 'full-time') ?
24 : 15;


                let warningLoad = (faculty.employment_type.toLowerCase() === 'full-time')
? 20 : 9;


                // Check if the faculty is within the warning or max load

```

```

        let isWarningLoaded = faculty.total_loads >= warningLoad &&
faculty.total_loads < maxLoad;

        let isMaxLoaded = faculty.total_loads >= maxLoad;

        // Create the option element

        let option = $('<option>', {

            value: faculty.code,

            text: `${faculty.code} (${faculty.department})`,

            css: {

                backgroundColor: isMaxLoaded ? '#bb2124' : isWarningLoaded ?
'#f0ad4e' : "",

                color: isMaxLoaded || isWarningLoaded ? 'white' : ""

            }

        });

        $('#facultySelect').append(option);

    });

} else {

    alertify.error("No faculty found for selected departments.");

}

} catch (error) {

```

```

        console.error("Error parsing faculty response:", error);

        alertify.error("Failed to load faculty.");

    }

},

error: function () {

    console.log("Failed to fetch faculty.");

    alertify.error("Unable to load faculty. Please try again.");

}

});

}

$('#assignFacultyButton').prop('disabled', true).off('click').on('click', function () {

    const faculty = $('#facultySelect').val();

    if (faculty) {

        const programCode = $('#programCode').val();

        if (!programCode || !curriculumCode || !yearTerm) {

            alertify.error('Please complete all required fields before saving.');
            checked

            return;

        }

```

```

$.ajax({

    url: '../shared/scripts/add-faculty-load.php',

    type: 'POST',

    data: {

        section: section,

        curriculum_code: curriculumCode,

        year_term: yearTerm,

        course_code: courseCode,

        course_title: courseTitle,

        lec: lec,

        lab: lab,

        total: total,

        faculty: faculty

    },

    success: function (response) {

        const result = JSON.parse(response);

        if (result.status === "success") {

            alertify.success(faculty + " has been assigned to " + section + ", " + courseTitle)

            $('#curriculumCode').trigger('change');

            $('#facultyModal').modal('hide');

```



```

        $('#assignFacultyButton').prop('disabled', false);

    } else {

        alertify.error(result.message);

        $('#assignFacultyButton').prop('disabled', false);

    }

    },

    error: function () {

        alertify.error("An error occurred while processing the request.");

    }

    });

} else {

    alertify.error('Please select a faculty.');
```



```

    }

    });

});

});

$(document).on('click', '.unassign-faculty', function () {

    const button = $(this);

    const facultyLoadId = button.data('id');

    const section = button.data('section');

    Swal.fire({
```

```

title: 'Are you sure?',

text: "You are about to unassign this faculty",

icon: 'warning',

showCancelButton: true,

confirmButtonText: 'Yes',

confirmButtonColor: '#6D86A1',

cancelButtonText: 'Cancel',

}).then((result) => {

    if (result.isConfirmed) {

        $.ajax({

            url: '../shared/scripts/remove-faculty-load.php',

            type: 'POST',

            data: {

                faculty_loads_id: facultyLoadId,

                section: section

            },

            success: function (response) {

                const data = JSON.parse(response);

                if (data.status === 'success') {

                    $('#curriculumCode').trigger('change');

```

```

        alertify.success("Faculty has been unassigned");

    } else {

        alertify.error(data.message);

    }

},

error: function () {

    alert('An error occurred while unassigning the faculty.');

}

});

} else {

    alertify.warning('Unassignment was canceled.');

}

});

});

});

```

APPENDIX C. EVALUATION TOOL/TEST DOCUMENTS

November 20, 2024

Mr. Ricson M. Ricardo

Academic Head

STI COLLEGE ALABANG

RZB bldg., Interior Montañano St.

Alabang, Muntinlupa City

Dear Mr. Ricson,

First and foremost, I would like to thank you for accommodating the following Students.

- Bienes, Stalen Hitler N.
- Diamla, James Roland M.
- Ibarreta, Richard Rodney T.
- Labaclado, John Simon Lee B.

They are currently taking up **IT Special Project** for this Second Semester, AY 23-24, and part of the requirement of the course in the submission of project documentation and the actual development of the proposed system.

They have expressed their willingness to develop a system for your company entitled “**Chronoschedio: Implementation of a Web-Based Class Scheduling and Faculty Loading Management for STI College Alabang**”. In this regard, may I request your approval to allow our students to make an understudy of your company. May I also request to please grant these students for interviews and other sources of significant information so that they have an overview of the operations, managements, and needs of your company which would be used as basis for the development of the proposed system.

Our institution will greatly appreciate your approval and any help you can extend to them.

More power and God bless!

Sincerely yours,

Jeremiah D. Tablatin

Capstone Project Coordinator

QUESTIONNAIRE FOR ISO 25010: SOFTWARE QUALITY STANDARDS

Name: Mr. King Jasser N. Andaya

Position: IT Head

Instruction:

Please evaluate the project entitled “**CHRONOSCHEDIO: Implementing of a Web-based Class Scheduling and Faculty Loading Management for STI College Alabang**” by rating each item in the questionnaire using the rating scale shown in the table1. Check inside the box which is appropriate that corresponds to your observation.

Table 1. Rating Scale and Its Interpretation

Rating	Interpretation
5	Strongly Agree
4	Agree
3	Neither Agree or Disagree
2	Disagree
1	Strongly Disagree

Table 2. Functionality

Please rate the system functionality according to the capability of the system to provide functions which meet the stated and implied needs of users under the specified conditions of usage	5	4	3	2	1
Does the Chronoshedio meets the requirements in scheduling and faculty loading of STI College Alabang?	✓				
Does it include features such as conflict checker and real time updates to changes in faculty or class requirements?	✓				
Does the system allow faculty members to view their schedules in real time?	✓				
Does the system include features for tracking, such as faculty loading percentages and room utilization?	✓				
Is the system precise in its results	✓				

Table 3. Reliability

Please rate the system's reliability according to the capability of the system product to maintain a specified level of performance when used under specified conditions;	5	4	3	2	1
Does the system maintain data integrity during operations like schedule updates	✓				
Does the system save all data correctly when changes are made?	✓				
Does the system provide consistent performance and accurate results across different devices and browsers used by faculty and administrators?		✓			
The software informs users concerning invalid data entry		✓			

Table 4. Usability

Please rate the system usability according to the capability of the system product to maintain a specified level of performance when used under specified conditions;	5	4	3	2	1
Is it user friendly?		✓			
In your opinion, how helpful are the system's error messages when you make a mistake?	✓				
How clear are the labels and icons used in the system?		✓			
When doing tasks like creating reports, changing schedules, or saving data, how responsive do you think the system is?	✓				

Table 5. Portability

Please rate the system's portability according to the capability of the software product to provide appropriate performance, relative to the amount of resource used under stated conditions	5	4	3	2	1
When accessed using a mobile browser, how responsive is the system?	✓				
How easy it is to access the system from another location away from STI College Alabang?	✓				
Does the system's layout adjust properly when used on different devices?	✓				

Comments/Suggestions:

Recommendations:



Evaluator Signature

Thank you for taking the time to complete this ISO 25010 evaluation form. Your feedback is important to the proponents and will help the proponents to improve the quality of the class scheduling and faculty loading management system.

QUESTIONNAIRE FOR ISO 25010: SOFTWARE QUALITY STANDARDS

Name: Mr. Jerswin E. Balasta

Position: IT Professional

Instruction:

Please evaluate the project entitled “**CHRONOSCHEDIO: Implementing of a Web-based Class Scheduling and Faculty Loading Management for STI College Alabang**” by rating each item in the questionnaire using the rating scale shown in the table1. Check inside the box which is appropriate that corresponds to your observation.

Table 1. Rating Scale and Its Interpretation

Rating	Interpretation
5	Strongly Agree
4	Agree
3	Neither Agree or Disagree
2	Disagree
1	Strongly Disagree

Table 2. Functionality

Please rate the system functionality according to the capability of the system to provide functions which meet the stated and implied needs of users under the specified conditions of usage	5	4	3	2	1
Does the Chronoshedio meets the requirements in scheduling and faculty loading of STI College Alabang?	✓				
Does it include features such as conflict checker and real time updates to changes in faculty or class requirements?	✓				
Does the system allow faculty members to view their schedules in real time?	✓				
Does the system include features for tracking, such as faculty loading percentages and room utilization?	✓				
Is the system precise in its results	✓				

Table 3. Reliability

Please rate the system's reliability according to the capability of the system product to maintain a specified level of performance when used under specified conditions;	5	4	3	2	1
Does the system maintain data integrity during operations like schedule updates	✓				
Does the system save all data correctly when changes are made?	✓				
Does the system provide consistent performance and accurate results across different devices and browsers used by faculty and administrators?		✓			
The software informs users concerning invalid data entry	✓				

Table 4. Usability

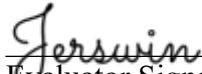
Please rate the system usability according to the capability of the system product to maintain a specified level of performance when used under specified conditions;	5	4	3	2	1
Is it user friendly?	✓				
In your opinion, how helpful are the system's error messages when you make a mistake?	✓				
How clear are the labels and icons used in the system?	✓				
When doing tasks like creating reports, changing schedules, or saving data, how responsive do you think the system is?	✓				

Table 5. Portability

Please rate the system's portability according to the capability of the software product to provide appropriate performance, relative to the amount of resource used under stated conditions	5	4	3	2	1
When accessed using a mobile browser, how responsive is the system?	✓				
How easy it is to access the system from another location away from STI College Alabang?	✓				
Does the system's layout adjust properly when used on different devices?		✓			

Comments/Suggestions:

Recommendations:



Evaluator Signature

Thank you for taking the time to complete this ISO 25010 evaluation form. Your feedback is important to the proponents and will help the proponents to improve the quality of the class scheduling and faculty loading management system.

APPENDIX D. SAMPLE INPUT/OUTPUT/REPORTS

INPUT/OUTPUT/REPORTS

Input Table

The inputs refer to the data and information that users (administrators, faculty, and students) provide to the system. Below is a table detailing the inputs:

1. Course Information

Field	Description	Example
Course Name	Name of the course	"Introduction to Programming"
Course Code	Unique identifier for the course	"CS101"
Number of Credits	Number of credits awarded for the course	"3"
Department	Department to which the course belongs	"Computer Science"
Course Type	Type of the course (Lecture, Lab, Seminar, etc.)	"Lecture"
Preferred Time Slot	Preferred time for the course (optional)	"Mon-Wed 10:00-12:00 PM"

2. Faculty Information

Field	Description	Example
Faculty Name	Name of the faculty member	"Mr. John Doe"
Faculty ID	Unique identifier for the faculty member	"F1234"
Department	Department the faculty member belongs to	"Computer Science"
Teaching Preferences	Faculty's preferred courses or teaching hours	"Math 101, CS 202"

Available Time Slots	Available time slots for teaching	"Mon-Fri 9:00 AM - 3:00 PM"
Maximum Teaching Hours	Maximum number of teaching hours per week	"15 hours"

3. Classroom Information

Field	Description	Example
Classroom ID	Unique identifier for the classroom	"Room 101"
Classroom Capacity	Maximum number of students the classroom can hold	"40"
Available Time Slots	Available time slots for the classroom	"Mon-Wed 9:00 AM - 12:00 PM"
Equipment Available	List of available equipment in the classroom	"Projector, Whiteboard"

4. Student Enrolment Data

Field	Description	Example
Student ID	Unique identifier for the student	"S101"
Student Name	Name of the student	"Jon Doe"
Enrolled Courses	List of courses the student is enrolled in	"CS101, Math 101"
Course Prerequisites	Prerequisite courses (if any) for the enrolled courses	"None"
Time Preferences	Student's preferred class timings (optional)	"Morning, 9:00 AM - 12:00 PM"

5. Academic Calendar

Field	Description	Example
Semester Start Date	Start date of the semester	"2025-02-01"
Semester End Date	End date of the semester	"2025-05-30"
Holidays	List of holidays during the semester	"2025-04-05 (Easter), 2025-05-25 (Memorial Day)"
Exam Dates	Dates for final exams	"2025-05-26 to 2025-05-30"

6. Faculty Constraints

Field	Description	Example
Maximum Teaching Hours	Maximum number of hours a faculty member can teach per week	"15 hours"
Maximum Number of Classes	Maximum number of classes a faculty member can teach per day	"3 classes/day"
Special Requests	Any specific faculty request (e.g., leave, availability)	"Unavailable on Fridays"

Output Table

The outputs are the results generated by the system after processing the inputs. Below are the outputs:

Output Field	Description	Output Type
Faculty Schedule	Personalized schedules for each faculty member, including course assignments.	PDF, Table View
Classroom Schedule	Allocated classroom schedules showing room usage by time slot.	PDF, Table View
Course Schedule	Organized course schedules by course code and batch/year.	PDF, Table View
Faculty Loading Summary	Summary of teaching hours per faculty member.	PDF, Table View
Unresolved Conflicts	List of scheduling conflicts and recommendations for resolution.	Notifications
Summary of Teaching Hours	Breakdown of teaching load distribution among faculty.	Chart, Table View

Report Specifications

The reports generated by the system aim to support decision-making and record-keeping. Below are the report templates:

Report Name	Description	Report Type	Audience
Faculty Loading Report	Comprehensive details on teaching hours, assigned courses, and classrooms per faculty member.	PDF, Printable	Administrators
Course Offering Report	List of all courses offered with corresponding faculty assignments and schedules.	PDF	Administrators
Classroom Utilization Report	Analysis of classroom usage, highlighting underutilized or overbooked rooms.	PDF, Chart	Administrators
Schedule Conflict Report	Detailed list of scheduling conflicts detected by the system with recommendations.	PDF, Table View	Administrators
Overall Academic Schedule Report	Summary of all schedules, including faculty, classrooms, and courses, for the current semester.	PDF, Printable	Administrators, Faculty
Faculty Availability Report	A report highlighting faculty availability and hours worked in comparison to maximum allowed load.	PDF, Table View	Administrators

APPENDIX E. USERS GUIDE

USERS GUIDE

This guide provides detailed instructions for each user role to navigate and efficiently utilize the system. The system is designed to simplify class scheduling, faculty load management, and academic coordination, offering role-based access for streamlined operations.

The system uses Microsoft 365 for secure authentication and integrates functionalities tailored for specific roles: **Superuser**, **Academic Head**, **Program Head**, **Faculty**, **Registrar**, and **Students**.

System Requirements

- A device with internet access (PC, laptop).
- A valid **Microsoft 365 account** for login.
- A compatible web browser (e.g., Microsoft Edge, Google Chrome, Mozilla Firefox)

1. Logging Into the System

1. Launch the System:

Open your web browser and navigate to the system URL
(<https://www.chronoschedio.com/>).

2. Microsoft 365 Login:

On the login page, click "**Sign in with Microsoft 365**".

3. Enter Credentials:

- Input your Microsoft 365 email address (e.g., user@domain.com).
- Enter your password and complete any additional authentication steps (e.g., two-factor authentication).

4. Role-Based Access:

After login, the dashboard and features available will depend on your assigned role.

2. User Roles and Features

A. Superuser

The Superuser has full access to all system functionalities and administrative privileges.

Key Features:

- Manage user accounts for all roles.
- Set system-wide configurations (e.g., academic terms, time slots, classroom capacities).
- Generate comprehensive reports.

How to:

- **Add Users:** Navigate to **User Management** → Click **Add User** → Fill in user details and assign roles.
- **Configure Academic Settings:** Go to **Settings** → Update academic year, semester, and other configurations.

B. Academic Head

The Academic Head oversees overall scheduling, faculty loading, and conflict resolution.

Key Features:

- Review and approve schedules.
- Monitor faculty loads and classroom utilization.
- Resolve conflicts and generate academic reports.

How to:

- **Approve Schedules:** Navigate to **Schedule Management** → Select schedules under **Pending Approval** → Review and click **Approve** or **Reject**.
- **Monitor Faculty Loads:** Go to **Reports** → Select **Faculty Loading Report**.

C. Program Head

The Program Head focuses on managing schedules and loads for their specific program/department.

Key Features:

- Create and edit course schedules for assigned programs.
- Allocate faculty to courses and classrooms.
- Submit schedules for approval.

How to:

- **Create a Schedule:**
 1. Go to **Schedule Management**.
 2. Click **Create Schedule**.
 3. Select the course, faculty member, classroom, and time slot.
 4. Save and submit for Academic Head approval.

D. Faculty

Faculty members can view their schedules, manage their availability, and check assigned loads.

Key Features:

- View teaching schedule and assigned courses.
- Set availability for scheduling.
- Download teaching load reports.

How to:

- **Set Availability:** Navigate to **My Profile** → Click **Set Availability** → Select preferred days and time slots → Save.
- **View Schedule:** Go to **My Schedule** to see your weekly timetable.

E. Registrar

The Registrar manages student enrollment, assigns batch schedules, and ensures timetable alignment.

Key Features:

- Assign students to specific class schedules.
- Monitor and update student batch information

How to:

- **Assign Batch to Schedule:**
 1. Navigate to **Student Management** → Select a batch.
 2. Assign the batch to the desired schedule.
 3. Save changes.

F. Students

Students can view their class schedules and access assigned courses.

Key Features:

- View personalized class schedules.
- Access course details and classroom locations.

How to:

- **View Schedule:** Navigate to **My Schedule** after logging in.
- **Access Course Details:** Click on a course from the schedule to view details such as classroom location and faculty contact.

3. Common Functions

A. Resolving Scheduling Conflicts

1. Navigate to **Conflict Resolution** under **Schedule Management**.
2. Review the list of conflicts (e.g., overlapping schedules).
3. Follow system recommendations or manually adjust schedules.
4. Save the resolved schedule.

B. Generating Reports

1. Go to **Reports**.
2. Select the report type (e.g., Faculty Loading Report, Classroom Utilization Report).
3. Customize parameters if needed (e.g., date range, specific program).
4. Click **Generate** and download in the desired format (PDF/Excel).

C. Updating Personal Information

1. Navigate to **My Profile**.
2. Update your contact details, photo, or preferences.
3. Save changes.

4. Logging Out

1. Click your profile icon in the top-right corner.
2. Select **Logout** to exit the system securely.

5. Troubleshooting and Support

Issue	Solution
Unable to log in	Ensure your Microsoft 365 account is active and try resetting your password.
Missing features	Check if your assigned role matches your expected system access. Contact the administrator.
Schedule conflicts not resolving	Revisit the conflict details and adjust schedules manually or contact the Academic Head.

Support Contact:

- **Email:** support@yourdomain.com
- **Phone:** +123-456-7890

6. Quick Reference Table for Roles and Access

Role	Key Features	Reports Available
Superuser	Full system management	All reports
Academic Head	Schedule approval, faculty load monitoring	Faculty Loading, Schedule Summary
Program Head	Program-specific scheduling and faculty allocation	Program Schedule, Classroom Usage
Faculty	View schedules, set availability	Teaching Load Report
Registrar	Batch management, student schedules	Enrollment Report, Attendance
Students	View schedules	None

APPENDIX F. CURRICULUM VITAE

CURRICULUM VITAE

Curriculum Vitae of

Bianes, Stalen Hitler N.

#0689 Zone 3, Sitio

Pagkakaisa Brgy. Sucat,

Muntinlupa City

bianesstalen@gmail.com

09217483687



EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2021	STI College Alabang
High School	2015-2018	Muntinlupa Business Highschool
Elementary	2009-2014	Sitio Sucat Elementary School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/Job Title	Name and Address of Company or Organization
2021	NSTP	STI College Alabang

Listed in reverse chronological order (most recent first).

AFFILIATIONS

Inclusive Dates	Name of Organization	Position
2023	Student Council	P.R. O
2022	Coders Club	Secretary
2022	Teatro Versatile	President

Listed in reverse chronological order (most recent first).

SKILLS

SKILLS	Level of Competency	Date Acquired
PHP	Beginner	2024
CSS	Beginner	2024
HTML	Beginner	2024
SAP	Advance	2023
SAP	Beginner	2022
JAVA	Advance	2021

TRAININGS, SEMINARS, OR WORKSHOPS

ATTENDED

Inclusive Dates	Title of Training, Seminar, or Workshop
May	TECH HORIZON
Oct	Youth for Leadership Summit
Aug	TEATRO VERSATILE WORKSHOPS

Curriculum Vitae of

Diamla, James Roland M.

Block 8 Lot 12 Elvinda 7

San Antonio, San Pedro,

Laguna

Jamesdiamla27@gmail.com

0929.195.5157



EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2021	STI College Alabang
High School	2014-2017	St. Helena School
Elementary	2008-2014	St. Helena School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization
2021	NSTP	STI College Alabang

Listed in reverse chronological order (most recent first).

AFFILIATIONS

Inclusive Dates	Name of Organization	Position
2023	Computer Society	3 rd Year Representative
2023	Unleash Club	President
2022	Coders Club	Project Manager
2022	Teatro Versatile	Production

Listed in reverse chronological order (most recent first).

SKILLS

SKILLS	Level of Competency	Date Acquired
PHP	Beginner	2021
CSS	Beginner	2021
JavaScript	Beginner	2021
HTML	Beginner	2021
SAP	Beginner	2023
SAP	Beginner	2022
JAVA	Beginner	2021

TRAININGS, SEMINARS, OR WORKSHOPS

ATTENDED

Inclusive Dates	Title of Training, Seminar, or Workshop
May	TECH HORIZON
January	Cybernet: Unmasking Phishing Scheme
January	IOT: Harmony: Orchestrating Voice Command
November	The Era's Tour: How to cope with today Era's
October	Teatro Versatile Workshop
August	Elms Tech Tools

Listed in reverse chronological order (most recent first).

Curriculum Vitae of

Ibarreta, Richard Rodney T.

Block 23 Lot 4D Dama De Noche Street, Doña

Rosario Bayview Subdivision, Brgy. Sucat,

Muntinlupa City

Ibarretarichard@gmail.com

0916.512.0501/0962.603.9062



EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2021-Present	STI College Alabang
Vocational/Technical	N/A	N/A
High School	2000-2005	Jesu-Mariae International School
Elementary	1995 – 2000	Jesu-Mariae International School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization
Oct 2021	NSTP	STI College Alabang
Nov 2019	Senior Customer Representative	GenX Sports and Media Production
Aug 2019	Answering Service	Chat Global Inc.
June 2018	Sales Representative	Telefficare Contact Center

Listed in reverse chronological order (most recent first).

AFFILIATIONS

Inclusive Dates	Name of Organization	Position
2023-Present	Computer Society	President
2022	Coders Club	President
2022	Teatro Versatile	Production
2021	Computer Society	1 st Year Representative

Listed in reverse chronological order (most recent first).

SKILLS

SKILLS	Level of Competency	Date Acquired
PHP	Beginner	2024
CSS	Beginner	2024
HTML	Beginner	2024
SAP	Advanced	2023
SAP	Beginner	2022
JAVA	Advance	2021

TRAININGS, SEMINARS, OR WORKSHOPS ATTENDED

Inclusive Dates	Title of Training, Seminar, or Workshop
May	TECH HORIZON
January	Cybernet: Unmasking Phishing Scheme
January	IOT Harmony: Orchestrating Voice Command
November	The Era's Tour: How to cope with today's Era
October	Teatro Versatile Workshop
August	Elms Tech Tools

Curriculum Vitae of

Labaclado, John Simon Lee B.

Sition Kanluran Zapote V

Bacoor Cavite City

[simonleelabaclado@gmail.co](mailto:simonleelabaclado@gmail.com)

[m 09668129073](tel:09668129073)



EDUCATIONAL BACKGROUND

Level	Inclusive Dates	Name of school/ Institution
Tertiary	2021	STI College Alabang
	N/A	N/A
High School	2009-2018	Mary Mother of God Parochial School
Elementary	2009-2014	Maranatha Christian School

PROFESSIONAL OR VOLUNTEER EXPERIENCE

Inclusive Dates	Nature of Experience/ Job Title	Name and Address of Company or Organization
Feb 2023- Present	Student Assistant	STI College Alabang

Listed in reverse chronological order (most recent first).

AFFILIATIONS

Inclusive Dates	Name of Organization	Position
Present	Computer Society	Tech Officer
Sept 2022	Unleash Club	Musician
Aug 2019 month year	Computer Society	Tech Officer

Listed in reverse chronological order (most recent first).

SKILLS

SKILLS	Level of Competency	Date Acquired
PHP	Beginner	2024
CSS	Beginner	2024
HTML	Beginner	2024
SAP	Advance	2023
SAP	Beginner	2022
JAVA	Advance	2021

TRAININGS, SEMINARS, OR WORKSHOPS ATTENDED

Inclusive Dates	Title of Training, Seminar, or Workshop
May	TECH HORIZON
January	Cybernet: Unmasking Phising Scheme
January	IOT Harmony: Orchestrating Voice Command
November	The Era's Tour: How to cope with today's Era
Aug	Elms Tech Tools

TRANSCRIPT OF INTERVIEW

Interviewer: What is the reason for developing a class schedule?

Interviewee: The reason why I need this class schedule is a consideration for the students because if there's no created schedule and its already enrollment day, it will be difficult for the students to know their schedule or preparation, At the same time, for the transferee or some working students on how they will be scheduled.

Interviewer: Other requirements while creating class schedule.

Interviewee: While making the schedule, it is required to meet the class offered course for the faculty or what my faculty needs.

Interviewer: Are there other things to consider in creating a class schedule?

Interviewee: So, another is considered room availability. if there's an efficient tool that will help me to add the right capacity for each room at the same time.

Interviewer: other resources needed for class scheduling?

Interviewee: Aircons as far as possible there is no space as per head office requirement as much as possible. Utilization rate and room maximization from start to end. I think it's 90%. and if I don't have a good tool that can detect it.

Interviewer: Can someone help you in creating a class schedule?

Interviewee: Aside from me, the Department heads can also help me in the scheduling, as with the manual or use of Excel they can assign teachers.

Interviewer: Is it sustainable or future proof?

Interviewee: But if there are 3000 or more students, it's not doable especially if STIs will be transferred to a bigger school, or it will be expected.

Interviewer: What do you expect from the system?

Interviewee: to have buff numbers. What if sections only are made? Needs a scheduling system number one reason. making the scheduling, conflicting rooms, or if there's a system that eliminates errors that would be great, cause the student will have a difficult time

knowing their schedule or getting lost at school.

Interviewer: Can you manage your time to create a class schedule in a short period of time?

Interviewee: As an academic head, I cannot use my entire week every time there are changes and use a manual, if there's a tool that will help me to easily create or change a schedule that will prevent time consumption. or at least to lessen time. Because it is more clerical work than the academic head's work and eliminating manual process technology.

Interviewer: What are the possible outcomes?

Interviewee: And of course, the change, the schedule, not only the students will have the satisfaction, but even the faculty.

Interviewer: What is the possible outcome if the expectations are not met?

Interviewee: So again, there's a possibility of dissatisfaction for students and faculty and encounter, and every time there are changes because of not detecting immediately the errors that's why every week there are changes in the schedule.

Interviewer: What account do you prefer to use in the system?

Interviewee: I prefer to use Microsoft Office 365 account since I have too many workloads and using the same account within the institution will make it more less hassle for me to use another account.

Interviewer: Why do you think that class scheduling system is a perfect tool to help you create one?

Interviewee: Some working students already filed their schedules. for that week or for that month for that semester but if there are other changes another conflict again so some of the reasons why this scheduling system is Perfect, efficient, and effective for creating class schedules, when the schedule is good and there is no conflict. The teachers will not have problems with their schedules so they can now focus on their requirements on the subject rather than putting their time knowing what their schedule is.

Interviewer: How would you like the flow of the system?

Interviewee: The class schedule, basically, or initially subject to change, but this is based on the usual work of what I'm doing, but it may be subject to addition or modification.

So, number one, we need to consider. instructor the teachers, meaning who are teachers that I goanna load in the subjects. I also considered my current teachers, it's either that is the first thing I do. For higher or needed as I do the class scheduling the assumption needed is information and credentials.

Interviewer: How will you ensure the loaded subject to the faculty is aligned to their specialization?

Interviewee: This will matter in the class scheduling especially since we have something called a matrix of academic qualification which is also part of the instructor, and the instructor needs to have MAQ, to ensure that the loaded subject for a teacher is aligned in specialization and because STI is unique we have certification put from the credentials like MAQ, like DET or FCCE so this will be filtering if the teacher is qualified or not to handle a subject and also his/her 201 files for information cause it's difficult to know something like a birthday or other information about him/her and also considered they have the faculty rank the rank will determine her/hi loading and most of the time some of the teachers is hybrid, hope that will be dynamic like he/her have an identifier that approves to teach for the senior high or tertiary or pure SH or pure tertiary, sooner or later

we will have a separate, I hope that there's a filter that ranks they will determine if the teacher can handle the subject.

Interviewer: Does the faculty have an intent subject to teach?

Interviewee: Intendent to teach subjects they are qualified and intended to teach the subject so it will be considered but not absolute meaning the academic head and the program head can approve or not, but still considered to teach that subject cause his/her qualified another is preferred schedule from part-time their part of instructor if round, Prodi or a part-time full load they cannot choose their preferred schedule, the part-time can only input data that is preferred schedule and it will reflect in class scheduling that for example if he/her put

the name in the schedule that is not preferred the user will receive some kind of notification that is not considered to their availability.

Interviewer: How will you maintain the updated class schedules?

Interviewee: The next module would be maintenance management, dynamic to set rooms, create a room, can add or remove, and considered number of sections, operating room and operating days, and after set. Curriculum management Inside we have the academic programs like courses or programs offered by STI (BIST, BSHM, etc.), the program code, curriculum code, and we have the program curriculum inside is year term, course code, the course title, how many lectures, how many labs, total units, connected to MAQ, if it is possible to add the FCCE, and also can add certifications/credentials.

Interviewer: How will you add load to the faculty?

Interviewee: Faculty Loading the user will upload/select from the created curriculum and after that, the user will add the teachers, also in faculty loading there is part of reports or viewing within the faculty loading I can see if the teacher is fully loaded or still not enough.

Interviewer: After all of those processes can you create a class schedule now?

Interviewee: Class scheduling the user can put for example the section BSIT 111 the user will load in the schedule if it is Monday to Saturday for the rooms, that it will not have a conflict at the same time there is a way to see the room utilization within the schedule the user will see if the resources is maximized or not.

Interviewer: So, will that be efficient class scheduling?

Interviewee: Yes, and that is the reason to have a good system that will notify me of some reports like utilization and after-class scheduling is the reports that I can see the room utilization reports because that is required to submit, and I hope

Interviewer: Is there anything you want to see in the system?

Interviewee: I hope that it will notify me of some reports like utilization and after-class scheduling is the reports that I can see the room utilization reports because that is required

to submit, so that I can see inside the reports room schedule that can be export or viewing and same for the faculty class schedule same for the faculty and faculty inventory and I hope the faculty is connected to the instructor so the teacher can see his/her schedule and so on so I hope this is the flow of the system because that is how it's done when it comes to excel.

Interviewer: What is the reason why do you need 201 files?

Interviewee: The reason why I need 201 files is for ranking, HO Submission, and CHED submission also to know the basic information about the faculty, because that' one of my functions as academic head.

Interviewer: Is that under of faculty loading management sir and since there's Oracle how will be the processes?

Interviewee: In Oracle input schedules like for example. Manual Schedule then AH Made Schedule, and then Via Manual. So, after I finish doing the manual schedule, it will proceed to Registrar and finally Oracle, then if they want to change the oracle, it should be returned to me. Secondly how to they manage faculty load without proper information because how will you load that teacher if don't know the credentials that is the reason why I need also the 201 files, that's where the MAQ will come in.

Interviewer: What if there's not an available teacher sir?

Interviewee: First time, I hope that action that can bypass, that's why need the FCCE for example there's an FCCE exam and the I failed that, so basically, he/she cannot teach that subject, that is why in FCCE that's where they put the failed subject so meaning aside from MAQ all that FCCE like course code, once is passed in FCCE I hope there's an action that can update.

INTERVIEWERS:

Bianes, Stalen Hitler N.

Diamla, James Roland M.

Ibarreta, Richard Rodney T.

Labaclado, John Simon Lee B.

INTERVIEWEE:

Mr. Ricson M. Ricardo
Academic Head, STI College Alabang

TRANSCRIPT OF INTERVIEW

Interviewer: What is the possible outcome if you will receive the schedule late?

Interviewee: In Senior High School (SHS) there is no problem in the schedule because the units is fixed. The tertiary students are different because of regular and irregular students. If the schedule received upon enrollment will be change by the academic head due to different reason, sudden changes will happen in the schedule of the enrolled student on the schedule he/she saw during enrollment.

Interviewer: What is the current problem you faced in regards of class schedules.

Interviewee: Prone of changes in schedules and Teachers. In present days overloading ng faculty is prohibited unlike before. That is the main problem the availability of the teacher.

Interviewer: What is the current process of transferring the class schedule from the academic head to registrar.

Interviewee: Through outlook/ Office 365 account. Not unless urgent and I can't wait for his schedule I would chat him personally.

Interviewer: Is the current process sustainable?

Interviewee: Yes, if it is done right and in time. But it will be harder since the population of STICA is growing.

Interviewer: What do you expect in the system Chronoschedio.

Interviewee: The process of getting and updating the schedule will be faster. As well as the conflict in time, and room will be avoided. Also, the real time monitoring in the schedules is accessible to keep the schedules updated.

INTERVIEWERS:

Bianes, Stalen Hitler N.

Diamla, James Roland M.

Ibarreta, Richard Rodney T.

Labaclado, John Simon Lee B.

INTERVIEWEE:

Mrs. Lyla M. Baybay
Registrar, STI College Alabang